Harrington's Insights

Improvements, Concepts, Strategies, Successes and Failures in Data Compression

Michael Harrington
October 2026

Contents

Introduction

Part I — Framework

Chapter 1 — Scope, Claims and Prior Art
Chapter 2 — Evaluation Rules
Chapter 3 — Methodology

Part II — Strongest Results on Standard Formats

Chapter 4 — Automatic Record Detection and Record Delta
Chapter 5 — No-Pain Encoder Tuning
Chapter 6 — PDF Documents
Chapter 7 — Combary
Chapter 8 — The Combary Activity Map

Part III — The Harrington Compression Method

Chapter 9 — HCM Architecture
Chapter 10 — HCM Improvements: v3.1 to v3.3
Chapter 11 — hcm-trit and the Ternary Pointer
Chapter 12 — HCM-Bin
Chapter 13 — HCM Against the Field

Part IV — Further Methods and Data Types

Chapter 14 — Harrington Header Methods
Chapter 15 — From Quad Exchange to the Harrington Exchange
Chapter 16 — Images and Audio
Chapter 17 — Columnar Formats (Exploratory)
Chapter 18 — Modern Log and Export Data

Part V — Cost and Economic Potential

Chapter 19 — Energy and Cost Model
Chapter 20 — Economic Potential and Commercial Applications

Part VI — Limits, Lessons and Licensing

Chapter 21 — HaBiTS and Binary–Ternary Conversion
Chapter 22 — Negative Results
Chapter 23 — What Generalizes, and Open Questions
Chapter 24 — Licensing and Use

Appendices

Appendix A — Full Results Tables
Appendix B — Specifications
Appendix C — Commands and Versions
Appendix D — Scripts, Tools and Repository
Appendix E — References
Appendix F — Random Re-verification of Wins (28 October 2026)
Appendix G — Best Results Charts

Contact the Author

Introduction

Lossless compression is a mature field. The tools most organisations rely on — gzip, zstd, xz, 7-Zip, bzip2, the PDF and image formats — have been refined for decades, and it is reasonable to ask whether anything worthwhile remains to be gained. This book's answer is *yes, sometimes a great deal, sometimes very little, and it is possible to tell in advance which*.

Headline result: a new compression king for biased, packed data

The most significant result in this book is the **Harrington Exchange** (Chapter 15). It is a reversible transform that renames each symbol by how often it occurs, then splits the renamed symbol into two streams, at any bit width from 2 to 24. It was designed for random data with a bias, the kind produced by hardware noise sources, sensors and packed measurements, and that is exactly where it wins.

It was tested against eight outside compressors suggested by independent AI reviewers, including the strongest available: zstd at its maximum setting (-22), Brotli 11, 7-Zip ultra, Blosc2, Facebook's OpenZL, and the context-mixing compressors ZPAQ -m5 and paq8px. Every result was decompressed and checked byte for byte.

Biased file (original bytes)	Harrington Exchange	Best practical rival	paq8px (strongest tested)
Synthetic 12-bit noise (524,288)	**324,507 B**, decodes in 5.1 ms	360,045 B (Blosc2)	323,153 B, decodes in 189 s
NIST biased bits (125,000)	**19,722 B**, 1.5 ms	21,640 B (Blosc2)	18,064 B, 27 s
NIST biased bytes (1,000,000)	**321,128 B**, 7.4 ms	321,626 B (Blosc2)	290,460 B, 270 s
Synthetic 5-bit skewed (524,289)	**346,370 B**, 11.3 ms	388,457 B (Brotli 11)	351,007 B, 180 s

- **It is the smallest practical compressor tested on every biased file.** Its decode speed places it in Tier 3 (M ≈ 38 against LZ4), and no rival in that tier or any faster tier matched its size on any of the four files. That makes it the Tier 3 compression king for this class of data (Section 15.13).
- **It beat the context-mixing giants outright on some files.** On the 5-bit file it was smaller than paq8px, one of the strongest compressors in existence, while decoding about 16,000 times faster. It was also smaller and faster than ZPAQ -m5 on both the 5-bit and 12-bit files.
- **Where it loses on size, it gives up very little.** On the 12-bit file it came within 1,354 bytes of paq8px, which took over three minutes to decode against 5 milliseconds for the Exchange.
- **It can be made fast as well as small.** Feeding the Exchange's renaming into Blosc2, the closest practical rival, gave a hybrid that beats stock Blosc2 on both counts on the 12-bit file: 324,765 bytes against 360,045, decoding in about 0.7 ms against 0.76 ms. On the other three biased files the hybrid is smaller than Blosc2 but 1.3–2.5× slower, and on the 5-bit file it is still smaller than paq8px while decoding in under a millisecond (Section 15.14).
- **It wins on real radio noise too.** On public software-defined-radio captures (8-bit receiver samples from the rtl_433 test set), the Exchange won the two noisiest of six. On the Fine Offset WH51 capture it was a No-Pain win over xz and the author's own tuned xz-hi: 1,000,000 bytes down to 753,163, against 759,512 for xz-hi, decoding in 11.8 ms against 56.8 ms. On the Acurite 606TX capture, the Exchange's split with an xz back end gave the smallest file of any practical compressor (Section 15.15).

The limits are stated just as plainly. The crown covers biased, near-memoryless data packed at non-byte widths. On real medical images, EEG recordings and flight telemetry, renaming hurts, and Blosc2 with bytedelta and the context-mixing compressors do better. Two of the four winning files are synthetic, and cmix and Pcodec were not tested (Section 15.12).

A second original method: Harrington Sets, multi-core for single-core compressors

The second major original contribution is **Harrington Sets** (Section 10.9), a way to let compressors that have always run on a single processor core use every core in the machine. Instead of cutting a file into equal chunks, the method reads the file in small windows, measures each one on the spot, and gathers windows of the same kind from anywhere in the file into their own streams. Windows that are random or already compressed are excluded to a raw set through a layered exclusion map (set one or excluded, then set two or set three among the excluded) that keeps every window's exact position. Each stream gets its own model and runs on its own core.

This is the author's own method. It has loose ties to earlier ideas (parallel blocks, grouping similar data, storing incompressible data raw), but it works differently from all of them: other tools cut files into fixed equal chunks, group whole files by name, need a format description written in advance

(OpenZL, Parquet and ORC schemas), or work only on one kind of data (Logzip, for logs). Harrington Sets parses on the spot, with no format knowledge, on any data. To the author's knowledge, no existing tool does this.

- **HCM on samba:** 1.73 times faster to decode and 1.5 times faster to compress, for only 0.16% more size, beating plain equal-chunk blocks on both size and speed.
- **On three randomly picked medium files:** 1.5 to 1.9 times faster decoding on just two cores, with more speed expected on more cores.
- **It carries to other slow tools:** lpaq1-h decoded 1.6 to 1.9 times faster with the same treatment.

The aim is that almost any slow, single-core compressor can be made multi-core this way. Tools that already decode in milliseconds gain little and are left as they are.

The range of results: from fractional to colossal

The improvements reported here vary enormously in size, and the book does not hide that.

At the small end: tuning the lossless WebP encoder on the 24-image Kodak photo set reduced the combined output from 11,204,040 to 11,202,034 bytes. That is a saving of 2,006 bytes, or 0.018%. It is real and measurable, and it decodes with the standard WebP decoder, but it is small. (An earlier summary described this as 71 bytes; that figure was a difference in normalised units, not bytes.) Fractional gains like this matter only at scale: 0.018% of an exabyte-scale photo archive is still about 180 terabytes.

In the middle:

- **7-Zip (Silesia corpus):** the No-Pain search made every file in the 211.9 MB corpus smaller, from 48,757,480 to about 47,333,000 bytes (−2.9%), and total decoding got about 5% faster (Chapter 5).
- **PDF:** on 139 test files, the detector-chosen PDF row filter took qpdf's already optimised output from 81,681,005 to 81,173,302 bytes (−0.6%), with rendering time not slower (Chapter 6).

At the large end:

File / data	Standard result	Harrington result	Change	Decode speed	Chapter
kennedy.xls (spreadsheet, 1,029,744 B) with zstd -22	64,818 B	26,401 B	−59.3%	1,359 → 520 µs, **2.6× faster**	4
kennedy.xls with gzip -9	209,721 B	42,971 B (record delta)	−79.5%	2,179 → 1,054 µs, **2.1× faster**	4
kennedy.xls with gzip -9	209,721 B	35,558 B (Combary activity map)	−83.0%	2,148 → ≈1,400 µs, **1.5× faster**	8
kennedy.xls with xz -9e (stock xz decodes it)	51,868 B	23,656 B	−54.4%	faster	5
Gradient-image PDF (colorspace_cos.pdf)	140,249 B (qpdf)	1,900 B	−98.6%	render time equal	6
x-ray (medical image, 8,474,240 B) stored in ORC (zlib)	≈6,169,000 B (one column)	≈4,695,000 B (detected record columns)	−23.9%	read 46.1 → 25.6 ms, **1.8× faster**	17
x-ray (8,474,240 B) with zstd -19	5,129,823 B	4,508,494 B (record delta, prefix-sum inverse)	−12.1%	24.6 → 20.2 ms median of 8 runs, faster in 7 of 8	4
Packed 12-bit biased noise (524,288 B) with zstd -19	430,664 B	324,507 B (Harrington Exchange)	−24.6%	0.9 → 5.1 ms (slower than zstd; 3.8× faster than xz)	15
kennedy.xls with HCM-Bin vs ZPAQ -m5	16,822 B (ZPAQ)	16,797 B	−0.1%	6.35 → 0.79 s, **8× faster**	12
Medical radiograph RG1 with JPEG XL (stock decoder)	effort-7 default	+2.9% size	—	**7.5× faster**	16

Sizes throughout the book are in bytes. A value marked ≈ was derived from a normalised measurement because the raw byte count of that run was not kept; such values are accurate to within about 1 byte per 40 KB of original data.

A note on compression time

Some readers will reasonably ask about compression time, because several of these methods take longer to compress than the standard tools. That cost is real, and the book reports it. But it is paid once, when a file is written, while the benefits are collected every time the file is read. Consider a file that takes 200 time units to compress and 100 to decompress. Suppose a method doubles the compression time to 400 but cuts decompression to 90. The extra 200 units are repaid after 20 reads, and every read after that is a net saving.

Many systems read the same data far more often than that. Content delivery networks, cloud archives and AI services that repeatedly retrieve the same kinds of data are examples. Smaller files also save storage, and memory and storage are expensive. Fewer bytes also pass through every network link and device along the way, whether a data-centre link, a Wi-Fi router or a Bluetooth connection, so each of them spends less time transmitting. The trade only runs the other way for data that is written once and rarely read, and for those cases the faster standard settings remain the better choice.

Several of the most valuable results are about **decoding speed** rather than size: a file that opens two to eight times faster at the same or smaller size saves CPU time and energy every time it is read. Most of the book's methods also run a strict *No-Pain* test (Chapter 2), so a change is kept only if it is smaller, faster, or both, and worse at neither.

On modern log and export data (Chapter 18), HCM was 22–41% smaller than xz on server logs but far slower to decode, and stock-decodable xz tuning gave a small 0.3% gain with faster decoding. In October 2026 a random re-test of the recorded wins was added (Appendix F). It recorded compression time as well, confirmed most wins, withdrew two, and fixed one decoder weakness. Section 1.7 classifies every contribution by how original it is. Not every method succeeds on every dataset. On plain English text, record detection finds nothing to exploit and switches itself off. Several early ideas failed outright, and those results are collected in Part VI rather than hidden.

Where the gains come from

The results come from four broad approaches, each covered in its own chapter:

1. **Structural detection.** A cheap detector finds fixed-size records in a file automatically (Chapter 4).
2. **Transformations.** The data is reorganised before a standard compressor sees it — record delta, the Combary activity map, the Harrington Exchange (Chapters 4, 8 and 15).
3. **Header optimisation.** The tables that describe a transform are made smaller (Chapter 14).
4. **Modifying existing compressors** so that their standard decoders still read the output (Chapter 5), and the author's own context-mixing compressor, HCM (Chapters 9–13).

The detector finds real structure, not codec quirks

One finding deserves special emphasis because it suggests the methods will travel beyond the test files. The same four files — kennedy.xls, the mr and x-ray medical images, and the sao star catalogue — responded to record-based methods in **every** format and codec tested: zstd, gzip, LZ4, xz, 7-Zip, Snappy and LZFSE, and also the Parquet and ORC column formats. Files without that structure responded in none of them. A trick that exploited one compressor's weakness would not show up consistently across seven unrelated compressors and two storage formats. That consistency is strong evidence that the detector identifies genuine structure in the data (Section 17.4).

Knowing in advance when a method will help

The research also offers a practical way to predict success. In these experiments, the record methods helped when:

1. the byte-spacing histogram shows a **strong peak** at one record length;
2. most bytes **repeat the byte one row above**, which shows up as a low fraction of "active" bytes in the activity map;
3. the file is **binary**, with more than 10% control bytes;
4. the file is **not machine code**, which is better served by branch filters.

A fast pre-pass that checks these four signals, followed by a size-and-time fallback, tells a system whether to apply a method before spending time on it. For compression researchers this is a hypothesis to test on new data. For developers and organisations with large archives, it is a cheap screening rule. These predictors were derived from the files studied here and have not yet been validated on a separate dataset (Section 23.3).

How the book is organised

- **Part I (Chapters 1–3)** sets out what is claimed, the evaluation rules and the methodology.
- **Part II (Chapters 4–8)** presents the strongest results on standard formats.
- **Part III (Chapters 9–13)** documents HCM.
- **Part IV (Chapters 14–18)** covers header methods, the Harrington Exchange, images and audio, columnar formats, and modern log and export data.
- **Part V (Chapters 19–20)** examines cost and economic potential.
- **Part VI (Chapters 21–24)** records what did not work, what generalises, and licensing.

Chapter 1 — Scope, Claims and Prior Art

> **Executive summary.** This book reports a body of lossless-compression work carried out by Michael Harrington: a context-mixing compressor (HCM) whose design closely follows Matt Mahoney's lpaq1, with the author's additions on top, a family of structural transforms that make existing compressors smaller and often faster, and encoder-side tuning of widely deployed tools whose output still opens with the stock software. For an organisation, the practical outcome is lower storage and transfer cost on structured data — spreadsheets, fixed-record logs, medical images — without changing the software its customers already run. This chapter states precisely what is claimed, what is borrowed, and where the claims stop.

1.1 Objectives

The research pursues three objectives, in order of priority:

1. **Size without decode penalty.** A change is accepted only if it reduces compressed size, decode time, or both, and regresses neither (the *No-Pain* rule, formalised in Chapter 2).
2. **Format compatibility where possible.** Improvements that leave the output decodable by unmodified, publicly distributed decoders are preferred over improvements that require a new decoder, because they can be adopted without any change on the reading side.
3. **Originality.** A result is credited to this work only if it is not already the substance of a published tool or method. Gains obtained by invoking a third party's known technique (for example Google's zopfli for Deflate, or optipng-style filter search for PNG) are reported but not claimed.

1.2 What is claimed

Each contribution is classified using the scheme defined in Chapter 2: **Original** (the method or code is the author's), **Mutated** (partly the author's, partly third-party, and requiring a new decoder), and **Modified** (a third-party tool with the author's changes, still decoded by the stock tool).

The strongest results, in conventional units (compressed bytes; smaller is better):

- **Record detection + record delta** cut zstd's output on kennedy.xls by 59% and made it decode 2.6× faster.
- **The Combary activity map** cut gzip's output on the same file by 83% and decoded 1.5× faster.
- **HCM** beats ZPAQ -m5 on the Canterbury corpus total while decoding about six times faster.
- **No-Pain 7-Zip** makes every Silesia file smaller with no decode slowdown.

Contribution	Class	Decoder	Headline result	Chapter
Automatic record-length detection driving a record delta	Mutated	stock + inverse step	kennedy.xls (1,029,744 B) with zstd -22: 64,818 → 26,401 bytes (−59.3%); decode 1,359 → 520 µs (2.6× faster)	4
No-Pain 7-Zip (7z-hi)	Modified	stock 7-Zip	Silesia (211,938,580 B): 48,757,480 → ≈47,333,000 bytes (−2.9%); total decode 3,483 → 3,314 ms (5% faster)	5
No-Pain xz (xz-hi)	Modified	stock xz	Canterbury (2,810,784 B): 493,080 → 462,832 bytes (−6.1%); decode faster in first session, unconfirmed on re-timing	5
No-Pain bzip2 (bzip2-hi)	Modified	stock bzip2	Canterbury: −2.0% in size; decode 59.7 → 55.4 ms (7% faster)	5
Detector-chosen PDF row filters (No-Pain pick)	Modified	standard PDF readers (verified with poppler only)	139 PDFs (95,872,155 B): qpdf 81,681,005 → 81,173,302 bytes (−0.6% vs qpdf; −15.3% vs the originals); render time not slower	6
Combary activity map	Original / Mutated	stock + inverse step	kennedy.xls with gzip -9: 209,721 → 35,558 bytes (−83.0%); decode 2,148 → ≈1,400 µs (1.5× faster)	8

Contribution	Class	Decoder	Headline result	Chapter
HCM v3.3 context-mixing compressor (built on the lpaq1 design)	Mutated	HCM	Canterbury: 353,494 bytes (fast), 350,434 (max), vs ZPAQ -m5 ≈361,800; about 6× faster decode than ZPAQ -m5	9–10
hcm-trit v2 ternary pointer, rebuilt as *hcmx*	Original addition to the lpaq1-based HCM engine	HCM	Canterbury: 347,696 bytes (max, best-of), −0.8% vs HCM v3.3 max	11
HCM-Bin v1 record contexts (related to the paq8 record model)	Mutated	HCM	kennedy.xls 16,797 bytes (rebuilt; original build 16,763) vs ZPAQ -m5 16,822; decode 0.79 s vs 6.35 s (8× faster)	12
Harrington shrinking and permutation headers	Original	custom	256-entry table: 256 → 225 bytes (−12%)	14
Detected-record column layout for Parquet and ORC	Mutated	stock readers	x-ray, ORC zlib: −24% in size, read 46.1 → 25.6 ms (1.8× faster)	17
Harrington Sets (on-the-spot type grouping into parallel streams, layered exclusion map)	Original	HCM / lpaq1-h	samba (21,606,400 B): decode 16.8 → 9.7 s (1.73× faster) for +0.16% size, on 2 cores	10

Figures marked ≈ are derived from normalised measurements, because the raw byte totals of that run were not preserved; they are accurate to within a few bytes per megabyte.

Every figure in this table is restated, with its full per-file breakdown, in the chapter cited and in Appendix A.

1.3 What is not claimed

- **Context mixing itself.** HCM belongs to the context-mixing family introduced by Matt Mahoney's PAQ series and continued in lpaq, paq8px, ZPAQ and cmix. Its novelty lies in specific components — the ternary pointer, the word-position mixer selection, the line-position context, the binary-only record swap — not in the paradigm. The core engine is close to the work of others and is credited to them in Section 1.8.
- **Record modelling in general.** paq8 includes a record model that infers record length from repeated byte spacing. The contribution here is narrower: using an inexpensive detector of the same kind to *configure stock compressors* — choosing a delta distance for xz, 7-Zip, zstd, gzip or LZ4, or a column count for Parquet and ORC — and verifying that the result is No-Pain.
- **Delta filtering.** Delta filters exist in xz, 7-Zip and RAR. RAR additionally detects tabular data automatically, which is why record delta added almost nothing to RAR (Chapter 22).
- **Permutation coding.** Encoding a full permutation as a single factorial index is the Lehmer code. The author's shrinking header is an independent, slightly less compact variant; its value lies in simplicity and stream-friendliness, not in beating the theoretical bound.
- **Parameter search on stock encoders.** Searching encoder options per file is common practice. It is credited only where the search space or the selection rule is the author's — for example record-driven filter choice, or the strict decode-time acceptance test.
- **Superiority over whole compression systems.** Every comparison is against a named configuration of a named tool on a named corpus. A win over xz -9e on Canterbury is not a claim about xz in general.

1.4 Prior art at a glance

Area	Established work	Relationship to this book
Context mixing	PAQ, lpaq, paq8px, ZPAQ, cmix	HCM's core design follows lpaq1; the author's additions are built on it
Block sorting	Burrows–Wheeler transform, move-to-front, bzip2	Used in early HCM routes and in bzip2-hi
Record and table modelling	paq8 record model; RAR automatic delta; xz and 7-Zip delta filters	Record detector reused to drive stock tools
Byte and stream splitting	Byte-shuffle filters (for example Blosc)	Related to the nibble juggle of the Harrington Exchange
Deflate optimisation	zopfli	Reported, excluded from claims
PNG optimisation	optipng, pngcrush, zopflipng	Reported, excluded from claims
Permutation coding	Lehmer code, factorial number system	Compared against the shrinking header
Columnar storage	Parquet and ORC encodings and codecs	Record detector chooses the column layout
Parallel and type-aware compression	Equal-chunk blocks (pigz, pbzip2, xz -T, zstd -T, ZPAQ); file grouping by extension (FreeArc, 7-Zip, RAR); format-described stream splitting (OpenZL, Parquet, ORC); log clustering (Logzip)	Harrington Sets groups windows found on the spot, with no format knowledge, into parallel streams
PDF optimisation	qpdf, Ghostscript, pdfsizeopt (PNG predictors on images)	Baseline is qpdf; detector chooses predictor width for any decodable stream

Full citations appear in Appendix E.

1.5 Limitations that apply throughout

- **Corpora.** The core evidence comes from the Canterbury and Silesia corpora, with enwik8 for large text and small speech, photo and medical-image sets. These are standard but small. Results on proprietary data may differ.
- **One machine.** All timings come from a single cloud Linux environment. Ratios between compressors measured in the same session are reliable; absolute times are not portable.
- **Timing method. Warning; some results will vary.** Most decode times are in-process medians. Where only a command-line tool was available (RAR, FLAC, WebP), times include process start-up and vary by roughly 5–10%; those cases are flagged. In some cases the timing noise is large, and multiple runs will be needed to fully account for speed.
- **Outliers.** kennedy.xls responds to record-based methods far more than any other file. Wherever it dominates a total, the total is also given without it.
- **No independent replication yet.** All numbers are the author's own measurements. Scripts and tools for reproducing most of them are provided; Appendix D lists the results whose original sources were lost and can only be reproduced approximately, through rebuilt tools.

1.6 How to read this book

- **The Introduction** shows the range of results, from fractional to very large, and what predicts success.
- **Part I (Chapters 1–3)** sets out the rules and methods.
- **Part II (Chapters 4–8)** presents the strongest results on standard formats: record detection, No-Pain encoder tuning, PDF, Combary and the activity map.
- **Part III (Chapters 9–13)** documents HCM.
- **Part IV (Chapters 14–18)** covers further methods and data types: header methods, the Harrington Exchange, images and audio, columnar formats, and modern log and export data.
- **Part V (Chapters 19–20)** covers cost and economic potential.
- **Part VI (Chapters 21–24)** collects what did not work — HaBiTS conversion and other negative results — together with what generalises, and licensing.

A reader who uses a single tool can go directly to that tool's section and find every result.

1.7 Originality classification (28 October 2026)

Two separate scales are used. The **novelty scale** asks how new an idea is relative to the prior art we checked. The **authorship labels** (Original, Mutated, Modified; Chapter 2) say whose code it is and whether a stock decoder reads the output. An item can be "mostly original" in novelty yet labelled Original because the code is the author's. "Original" means no prior match was found in our searches; **a formal literature and patent search has not been done.**

Original (no prior match found)

- **Combary** (Chapter 7): a value-free positional symbol, and the design rule derived from it. That rule is a conceptual model, not a proven law. The claim rests on the representation and the mechanisms derived from it; positional and enumerative coding in general have substantial prior art.
- **HaBiTS** (Chapter 21): a headerless, streamable binary↔ternary conversion. Original, but a negative result for compression.
- **The No-Pain rule and the Compression Tier Standard** (Chapter 2): a distinctive evaluation framework. Its underlying principle, Pareto dominance, is established.
- **Harrington Sets** (Section 10.9): on-the-spot measurement of windows inside any file, grouping of like windows from anywhere in the file into separate streams, a layered exclusion map for random or already-compressed data, and parallel decoding of the streams inside a context-mixing compressor. Loose ties only: equal-chunk parallel blocks, file grouping by extension, format-described stream splitting (OpenZL) and log clustering (Logzip) work differently. No tool doing the same was found.

Mostly original (new mechanism; known neighbours)

- **Harrington Exchange** (Chapter 15). Neighbours: bitshuffle, byte shuffle, symbol ranking, slot codes. On biased data the renaming adds about 12% beyond splitting.

- **Ternary pointer** (Chapter 11). Neighbours: LZP, PPM exclusion, the paq8 match model. An independent control (Section 11.8) shows its gain in HCM comes from the refinement mixer, which is secondary estimation, not from the flag itself.
- **Combary activity map** (Chapter 8). Neighbours: change masks, JBIG row skipping.
- **hcm-trit v1 ternary digit coding** (Chapter 11).

Hybrid (known tools used in an unexpected way)

- Record detection driving stock compressors, with fast inverses and the No-Pain fallback (Chapter 4)
- HCM and its v3.1–v3.3 changes (Chapters 9–10); core design follows lpaq1 (Section 1.8)
- HCM-Bin record contexts (Chapter 12)
- Detector-chosen PDF row filters (Chapter 6)
- Detected column layout for Parquet and ORC (Chapter 17)
- No-Pain encoder search (Chapter 5)
- The pointer on a BWT back end (Chapter 11)
- Sample-tile screening (Chapter 16)
- The shrinking header and omitted-suffix permutation index (Chapter 14): an independent reinvention close to the known Lehmer code, so not claimed

Excluded from claims: choosing stock options already in someone else's tool (JPEG-LS thresholds, JPEG XL -P/-g/-I, zstd/Brotli/FLAC/WebP settings, RAR -mcd-/-mcx+, JBIG2 TPGD); third-party work (zopfli, optipng, qpdf, cmix's dictionary); bug fixes.

Win types. These are kept separate throughout the book:

- **No-Pain win:** smaller or faster to decode, with neither metric worse.
- **Size win with a decode cost:** for example the ternary pointer (decode +11–13%), or the Exchange against plain zstd.
- **Head-to-head win against a named rival:** for example the Exchange against xz on the 12-bit source, where it is smaller and faster.

1.8 Credit to the work HCM is built on

HCM is close to the work of others, and the author gladly credits it. The overall design follows Matt Mahoney's **lpaq1**: hashed contexts of the previous one to eight bytes, word contexts, a long-match model, a mixer whose weights are chosen by the partial byte and the match length, and an APM refinement stage. The record contexts added in v3.3 and HCM-Bin are related to the **paq8 record model**, which detects fixed-length records from a byte recurring at equal spacing and adds contexts from the rows above. The hash-table layout (64-byte buckets with check values) and the adaptive counters, which adapt at a rate of 1/(n + 1.6), follow **lpaq and ZPAQ** (ZPAQ uses 1/(n + 1.5)). The arithmetic coder follows **Witten, Neal and Cleary (1987)**. The exp() and log() functions used to build HCM's startup tables come from **Arm Optimized Routines**. The author's contribution to HCM is the set of additions described in Chapters 10 to 12 and the Harrington Sets method, and every improvement is offered back free under the MIT License for use in the original tools. The name "HCM" is a working label for this combined build; it is not a claim to the underlying design.

Chapter 2 — Evaluation Rules

> **Executive summary.** Compression results are easy to overstate: a file can be made smaller by making it slower to open, or a compressor can be "beaten" by comparing it at a weak setting. This chapter fixes the rules before any result is shown. A change counts as an improvement only if it gives up nothing — the No-Pain rule. Larger trade-offs are graded on a fixed scale, the Big Gain Standard, and every compressor is placed in a speed tier so that it competes only against tools of comparable decoding speed.

2.1 Classification of contributions

Class	Definition	Decoder required
Original	The method and code are the author's own, or a third-party base altered so extensively that the result is substantively new.	The author's decoder
Mutated	Partly the author's, partly third-party, with a format or pipeline change that the stock decoder cannot read unaided.	Stock decoder plus the author's inverse step, or a new decoder
Modified	A third-party tool with the author's changes confined to the encoder, so the stock decoder reads the output unchanged.	Stock decoder

Record delta in front of zstd, for example, is *Mutated*: zstd decodes the stream, and a short inverse pass restores the file. xz-hi is *Modified*: any copy of xz decodes its output.

2.2 The No-Pain rule

> *No loss of compression or decode time is allowed, only gains to either or both.*

Formally, let a baseline configuration have compressed size S_0 and median decode time T_0 on a file, and a candidate have S_1 and T_1. The candidate is **No-Pain** on that file if and only if

$S_1 \le S_0$ and $T_1 \le T_0$, with at least one inequality strict.

Four operational details follow:

1. **Per-file fallback.** An encoder may try several configurations and keep, per file, the best one that satisfies the rule, falling back to the baseline otherwise. Corpus totals are reported with this fallback applied.
2. **Strict timing.** Decode times are medians of repeated, interleaved runs. Where two times differ by less than the measured noise, the candidate is re-timed and accepted only if it still does not lose.
3. **Strict comparison in No-Pain picks.** The 3% equal-speed band applies only to Tier BGS rankings (Section 2.4). No-Pain picks use a strict "not slower" test on medians, and results whose timing is noisy are flagged as such.
4. **Decoder memory.** From 27 October 2026 onward, peak decoder memory is recorded alongside size and time. Results earlier than that date did not measure it, and are marked accordingly.

2.3 The Compression Tier Standard (v1.1)

Compressors differ in decode speed by four to five orders of magnitude, so a single league table is meaningless. Each configuration is placed in a tier by its **decode multiple M**: the sum of its per-file median decode times on a corpus, divided by the same sum for LZ4 at its default level, measured on the same machine in the same session.

Tier	Decode multiple M	Members measured on Canterbury
1 — Speedsters	$M \le 2$	LZ4 -12, zstd -22 ($M \approx 1.5$)
2	$2 < M \le 10$	gzip -9 ($M \approx 4$)
3	$10 < M \le 100$	xz, LZMA, 7-Zip, bzip2
4	$100 < M \le 3{,}000$	HCM fast, ZPAQ -m4
5 — Insane Compression	$M > 3{,}000$	ZPAQ -m5, paq8px, cmix

Tool placements outside these measured cases are given in the chapter for each tool.

Rules attached to the tiers:

- **M decides membership; rivals decide awards.** A configuration is judged only against the tested rivals in its own tier.
- **Re-tiering.** A modified compressor competes in the tier its own M lands in, not the tier of the tool it came from.
- **Kings.** Each of Tiers 1–4 has a *speed king* (fastest decode) and a *compression king* (smallest output). Defeating the kings, not merely any rival, is the research target. Tier 5 is considered out of reach for practical deployment.

2.4 The Big Gain Standard (BGS)

When a change is not No-Pain but buys a large improvement on one axis, it is graded on the following scale.

Grade	Size gain *or* decode speed-up	Allowed regression on the other metric
BGS-1	≥ 10% smaller *or* ≥ 1.25× faster	up to 5%
BGS-2	≥ 25% smaller *or* ≥ 2× faster	up to 15%
BGS-3	≥ 50% smaller *or* ≥ 10× faster	no fixed limit

An award is one of two kinds:

- **Baseline BGS** — measured against a single named compressor and setting.
- **Tier BGS** — the configuration is not dominated by any tested rival in its tier. Decode speeds within 3% of each other are treated as equal.

2.5 Coverage

Each award also states how widely it holds.

Coverage	Meaning
Local	Holds on one file or a small group of related files
Class	Holds across a recognisable class of data, for example radiographs or fixed-record binaries
General	Holds on the whole corpus total, with per-file fallback

The record format is **Award / Coverage / Corpus or file** — for example *No-Pain / General / Silesia* for 7z-hi, or *BGS-1 / Local / ooffice*.

2.6 Baseline and originality rules

- **Own units only.** Comparing one level or version against another earns an award only for the author's own tools (HCM v3.3 against HCM v3, for example). Choosing a different stock level of someone else's compressor — Brotli 5 instead of Brotli 2 — earns nothing.
- **Known work earns nothing.** If a gain comes from a published third-party method (zopfli, optipng-style PNG search, RAR's own exhaustive-search switch), it is reported for completeness and excluded from claims. A minor variation on a known idea is acceptable; repeating someone else's work is not.

2.7 Reporting conventions

- **Size unit.** Sizes are reported in conventional units: the original size in bytes, each competitor's compressed size in bytes, ours, and the absolute and percentage difference. Where the raw byte counts of a run were not preserved, sizes are derived from the normalised figure and marked ≈; for files up to 2 MB the derivation is exact to the byte, and for larger files it is accurate to within about 1 byte per 40 KB of original. A few early experiments on unpublished inputs (Chapters 9 and 21) are kept as *1,000,000 bits down to X bits* — the compressed size scaled to a one-megabit original — and labelled as normalised.
- **Speed.** Decompression speed is reported as time, and where useful as a speed-up factor (old time ÷ new time) or percentage change. Every results table is preceded by a short summary of the size and decode-speed changes.
- **Outliers.** kennedy.xls is an extreme outlier for record-based methods. Corpus totals are given both with and without it whenever it changes the conclusion.
- **Per-file disclosure.** Every corpus total is accompanied by its per-file breakdown, losses included.

2.8 Relation to established practice

The No-Pain rule is a strict form of Pareto dominance on the two axes of size and decode time: a No-Pain candidate dominates its baseline. The tier system resembles the speed classes used informally in public benchmarks, such as Matt Mahoney's Large Text Compression Benchmark and the lzbench suite. It makes the boundaries explicit and ties them to a reproducible reference, LZ4's own decode time on the same machine. The BGS grades formalise trade-offs that a pure Pareto analysis would neither accept nor rank.

Chapter 3 — Methodology

> **Executive summary.** Every number in this book comes from public test files, was checked by decompressing it back to the exact original, and was timed the same way for every competitor in a comparison. Where that was not possible — because a tool could only be run from its command line — the result is flagged as less precise. The goal is that anyone with the published scripts can reproduce the tables. Most can be; Appendix D lists the exceptions, where early laboratory sources were lost.

3.1 Corpora

Corpus	Files	Size	Character	Role
Canterbury (large-file set excluded)	11	2,810,784 bytes	English text, source code, HTML, a spreadsheet (kennedy.xls), a fax bitmap (ptt5), a SPARC executable (sum)	Primary small-file corpus
Silesia	12	211,938,580 bytes	Text, XML, databases, executables, medical images (mr, x-ray), astronomical catalogue (sao), chemical database (nci)	Primary large-file corpus
enwik8	1	100,000,000 bytes	First 10^8 bytes of an English Wikipedia dump; SHA-1 verified	Large-text race (Chapter 13)
Kodak	24	768 × 512 RGB each	Photographic images	WebP and PNG (Chapter 16)
Free Spoken Digit Dataset	6	2.8–4.6 MB each	8 kHz, 16-bit mono speech, concatenated per speaker	FLAC (Chapter 16)
Medical images	11	Various	Silesia x-ray and mr, plus nine images from pydicom's test data	JPEG XL and JPEG-LS (Chapter 16)

The download mirror used adds a checksum file, SHA1SUM, that is not part of the corpus. It is excluded from every table. (Some interim reports during the research called the remaining set "10 files"; it is the standard 11.)

3.2 Baselines

Each experiment names one stock configuration as its baseline, normally the tool's strongest commonly used setting:

Tool	Baseline
gzip	`gzip -9n`
bzip2	`bzip2 -9`
xz	`xz -9e`
zstd	`zstd --ultra -22` (also -19, -9, -3, -1 in Chapter 5)
LZ4	`lz4 -12`
Brotli	quality 11 (also 2, 5, 8)
LZMA / 7-Zip	LZMA SDK -9; 7-Zip Ultra `-mx9`
RAR	`rar -m5 -ma5`
Snappy, LZFSE	Library defaults (no levels)
FLAC	`flac -8`
WebP	`cwebp -lossless -z 9`
PNG	Pillow `optimize=True`
JPEG XL	`cjxl` lossless, effort 7
ZPAQ	`-m4`, `-m5`
HCM	v3 (for its own improvements); v3.3 thereafter

3.3 Lossless verification

Every compressed output reported in this book was decompressed and compared byte for byte with the original. For *Modified* results, decompression used the **stock** decoder — the unmodified public build of the tool — so that compatibility is demonstrated, not assumed. For *Mutated* results, the stock decoder was followed by the author's inverse step, and the final output was compared. No result with a mismatch was retained.

3.4 Decode-time measurement

Three methods were used, in descending order of precision:

1. **In-process timing.** A small C or C++ harness links the decoder library, decodes from memory to memory, and records the median of 9–41 runs. Baseline and candidate runs are interleaved to cancel drift. This method was used for gzip/zlib, zstd, LZ4, xz/liblzma, bzip2/libbz2, Snappy, LZFSE and ORC.
2. **In-process via a runtime.** Python's built-in `lzma` and `bz2` modules (thin wrappers over the C libraries) and Pillow for PNG. These timings carry a small, constant overhead.
3. **Command-line timing.** RAR (`unrar`), FLAC (`flac -d`), WebP (`dwebp`) and DuckDB were timed as processes. Process start-up adds roughly 2–4 ms per run, and repeatability is about ±5–10%. Conclusions drawn from these timings are marked as provisional.

3.5 Decoder memory

Peak decoder memory has been recorded since 27 October 2026. Earlier results report size and time only. HCM's memory use is documented separately in Chapter 9: 42 MB in fast mode and 68 MB in max mode.

3.6 Test environment

All measurements were made in Linux cloud containers with a shared Intel Xeon processor (2 virtual CPUs at 2.30 GHz, 7 GB RAM at the time of writing), using single-threaded decoding. Containers can differ slightly between sessions, so **comparisons are only made between runs taken in the same session**. Absolute times should be read as relative indicators, not as hardware specifications.

3.7 Software provenance

Third-party tools were built from their latest public source where possible, so that comparisons are made against the current state of each project:

Tool	Source / version
zstd	GitHub, 1.6.0 development
LZ4, zlib, bzip2, Brotli 1.2.0, Snappy, LZFSE, libwebp, FLAC, Apache ORC	GitHub, latest at test date
xz / liblzma	v5.4.5
LZMA SDK / 7-Zip	Igor Pavlov's SDK
RAR	7.30 beta 2 for Linux (official trial build)
DuckDB	1.3.2
gzip	1.12
ZPAQ, paq8px v217, cmix v21, bsc, bzip3	Public source

Command lines are listed in Appendix C. Exact commit identifiers were not recorded for every build; Appendix C states which.

3.8 Individual versus aggregate results

Per-file results are always the primary evidence; totals are derived from them. Because a single file can dominate a total — kennedy.xls being the clearest case — the conclusion of each chapter is stated per file first, and only then as a total, with and without the outlier.

3.9 Threats to validity

- **Corpus selection bias.** Canterbury and Silesia are old and small by modern standards; modern data such as JSON logs, columnar exports and container images is under-represented.
- **Tuning on the test set.** Per-file parameter searches are tuned on the very files they are scored on. For the No-Pain tools this is the intended deployment model — the search runs on every new file at compression time, so it is not overfitting in the machine-learning sense. It does mean

the results describe the search procedure, not any fixed setting; a fixed setting chosen from these results should not be expected to perform as well on unseen data.

- **Timing noise.** Command-line and short-file timings are near the noise floor. Results within the noise are not counted as wins.
- **Single author, single environment.** Independent reproduction has not yet occurred.

Chapter 4 — Automatic Record Detection and Record Delta

Executive summary. Many business and scientific files are tables of fixed-size records, and neighbouring rows differ only slightly. HCM includes a cheap detector that finds the record size automatically. Each byte can then be stored as its difference from the byte one row above. Mostly-zero differences compress far better, and with a vectorised reverse step they often decode *faster* too. On the Canterbury spreadsheet this more than halved the output of zstd, LZ4, gzip, Snappy, LZFSE and xz; RAR, which already detects tables, gained nothing. On files without record structure the method switches itself off.

4.1 Method

Detector. For the first 200 KB of the input, the distance since the previous occurrence of each byte value is counted. The most frequent distance between 2 and 512 among the top 20 is taken as the record length *R*; if none qualifies, *R* = 1 and the transform is disabled. HCM's in-model version (v3.1) requires a byte value to repeat at equal spacing three or more times.

Transform. $y_i = x_i - x_{i-R} \pmod{256}$, for $i \geq R$.

Inverse. $x_i = y_i + x_{i-R}$. Two inverse implementations are used in this book; Section 4.3a explains why the second was needed. A scalar loop is slow when R < 16, because of the dependency chain. The author's decoder reconstructs min(R, 16) bytes per step with 16-byte SIMD additions into a separate output buffer. Lanes beyond R are overwritten correctly on the next step. This vectorised inverse is what makes the method **No-Pain**: with a plain byte loop, LZ4 decoding of kennedy.xls became slower (1,176 µs against 770 µs).

Header. One flag byte and the record length.

Fallback. The encoder keeps whichever of plain and delta is smaller and not slower to decode.

Class. *Mutated*: stock decoder plus the author's inverse step. In xz and 7-Zip, which provide a built-in delta filter, the detected distance simply configures that filter, giving *Modified* output that stock tools decode unaided (Chapter 5).

Prior art. paq8's record model and RAR's automatic delta detection infer record structure internally. The contribution here is applying an inexpensive detector to configure stock compressors, and verifying the No-Pain property.

4.2 Canterbury — kennedy.xls (R = 13)

Summary. On kennedy.xls (1,029,744 bytes) record delta cut compressed size by 52–87% with every compressor that lacks its own table detection, and decoding got faster in every case: 2.6× faster with zstd, 2.1× with gzip, and 1.3–1.4× with Snappy and LZFSE. LZ4 was 6% faster, but only with the SIMD inverse. RAR gained nothing because it already detects tables.

Compressor	Plain (bytes)	With record delta (bytes)	Size change	Decode plain → delta	Verdict
zstd -22	64,818	**26,401**	−59.3%	1,359 → 520 µs (2.6× faster)	No-Pain
LZ4 -12	324,136	**43,653**	−86.5%	770 → 725 µs (6% faster)	No-Pain (SIMD inverse only)
gzip -9	209,721	**42,971**	−79.5%	2,179 → 1,054 µs (2.1× faster)	No-Pain
Snappy	422,640	**129,763**	−69.3%	966 → 717 µs (1.35× faster)	No-Pain
LZFSE	136,282	**54,844**	−59.8%	1,064 → 767 µs (1.39× faster)	No-Pain
xz -9e (via built-in delta filter)	51,868	**23,656**	−54.4%	faster	No-Pain, stock xz
RAR -m5	≈35,555	no gain	—	—	RAR already detects tables

All other Canterbury files fall back to plain: the detector finds record lengths such as 3–5 on text, but the delta output is larger, so it is rejected. **The Canterbury totals therefore change only because of kennedy.xls.**

Summary (corpus totals). Over the whole corpus (2,810,784 bytes) the gain is 7.4% for zstd, 12.2% for LZFSE, 22.8% for gzip, 23.2% for Snappy and 30.0% for LZ4. Every file decodes at least as fast as before, because any file where delta is slower is kept plain.

Compressor	Plain total (bytes)	With fallback (bytes)	Saved (bytes)	Change
zstd -22	516,140	477,733	38,407	−7.4%
LZ4 -12	935,419	654,947	280,472	−30.0%

Compressor	Plain total (bytes)	With fallback (bytes)	Saved (bytes)	Change
gzip -9	730,625	563,886	166,739	−22.8%
Snappy	1,264,594	971,717	292,877	−23.2%
LZFSE	670,001	588,563	81,438	−12.2%

4.3 Silesia — high-compression levels

Record delta was applied with gzip -9, zstd -19, LZ4 -12 and xz -9e, timed in-process, with exact round trips.

Summary. Only the two 16-bit medical images gain. x-ray shrinks by 15.6% with gzip, 6.3% with zstd and 5.3% with xz. mr shrinks by 9.7% with gzip and 5.1% with xz. On the 211,938,580-byte corpus total this gives 1.9% (gzip), 0.6% (zstd) and 0.8% (xz), with no file decoding slower. Figures are ≈ because this run kept only normalised sizes.

File (bytes)	gzip -9	zstd -19	xz -9e
mr (9,970,564; R = 2)	≈3,681,000 → **≈3,323,000** (−9.7%)	no gain	≈2,752,000 → **≈2,611,000** (−5.1%)
x-ray (8,474,240; R = 2)	≈6,038,000 → **≈5,094,000** (−15.6%)	≈5,130,000 → **≈4,806,000** (−6.3%)	≈4,491,000 → **≈4,252,000** (−5.3%)
Text, x86 executables, osdb, sao	worse — fallback	worse — fallback	worse — fallback
Silesia total with fallback	≈67,766,000 → **≈66,464,000** (−1.9%)	≈52,892,000 → **≈52,568,000** (−0.6%)	≈48,456,000 → **≈48,075,000** (−0.8%)

LZ4 -12 gained nothing that passed the timing test. Machine code does not benefit.

4.3a Re-verification and the prefix-sum inverse (28 October 2026)

A random re-test of recorded wins (Appendix F) drew x-ray with zstd -19. The sizes reproduced, but the decode-time win did not. With the original SIMD inverse, delta decoding was 4–15% *slower* than plain zstd in every run (22.4 → 27.0 ms in the first snapshot).

Cause. The original inverse rebuilds only *R* bytes per 16-byte step. At R = 2 it makes eight times as many passes as at R = 16, and loses to zstd's own speed.

Fix. For R ∈ {1, 2, 4, 8}, a new inverse computes the running sum inside one 16-byte register:

- each 16-byte block is added to itself shifted by R, 2R, 4R and 8R bytes;
- the last R bytes of the previous block are then added to every lane.

Larger or odd R keeps the original inverse.

Confirmation. Record delta (prefix-sum inverse) + zstd -19 with minMatch = 5, against stock zstd -19, x-ray (8,474,240 bytes), 8 paired in-process runs:

Summary. The output is 12.1% smaller and the median decode is 18% faster (24.6 → 20.2 ms). The candidate was faster in 7 of the 8 runs; that split has a probability of about 3.5% if the two decoded equally fast. Compression is about 30% slower (≈4.7 → ≈6.1 s).

	Stock zstd -19	Record delta + zstd -19 (minMatch 5)
Size (bytes)	5,129,823	**4,508,494**
Decode, median of 8 runs	24.6 ms	**20.2 ms**
Runs in which the candidate was faster	—	7 of 8
Compression time	≈4.7 s	≈6.1 s

The earlier x-ray zstd -19 result in the table above should be read with this correction: the size gain stands, and the No-Pain decode claim depends on the prefix-sum inverse.

Speed-tier snapshot (single runs, in-process; record delta or activity map against stock, with fallback):

- **LZ4 (Tier 1):** no win on any file. LZ4 decodes at near memory-copy speed, so a separate reverse pass always costs more than it saves. kennedy.xls was 370,976 → 112,465 bytes, but decoding was twice as slow (0.32 → 0.65 ms). Winning Tier 1 would need the inverse fused into LZ4's copy loop.
- **zstd -1 (Tier 2):** wins on kennedy.xls (115,391 → 51,551 bytes; 0.96 → 0.78 ms) and x-ray (6,772,118 → 5,025,009 bytes; 9.84 → 8.61 ms). It fails on mr and sao.
- **Eight randomly drawn other files** (seed 2320821518; logs, JSON, Parquet, osdb, samba, xml): all fell back to plain.

4.4 Silesia — fast levels (complete per-file record)

Summary. At fast levels the gains come from mr and x-ray. The best single result is x-ray with zstd -1: 1,747,111 bytes smaller (6,772,118 → 5,025,007, −25.8%) and decoding 17% faster (9.3 → 7.7 ms). Corpus totals with fallback: zstd -1 −2.4% (73,229,468 → 71,482,357 bytes), xz -0 −1.1% (62,590,540 → 61,887,888), gzip -1 −0.9% (77,622,399 → 76,910,692). Total decode time fell slightly in each case (0.3–0.8%). zstd -3 and LZ4 fast never pass: LZ4's plain decode is already near memory speed.

Bold marks cells where the delta is smaller and not slower (No-Pain); all other cells fall back to plain. Sizes are compressed bytes; times are decode medians.

File (bytes, R)	gzip-1	zstd-1	zstd-3	lz4-fast	xz-0
dickens (10,192,446, 69)	4,620,004 → 8,622,194 (48.9 → 80.4 ms)	4,261,734 → 8,530,518 (8.6 → 11.3 ms)	3,663,504 → 8,490,008 (9.8 → 12.2 ms)	6,403,052 → 10,231,269 (3.9 → 1.9 ms)	4,162,208 → 8,517,060 (250.9 → 512.1 ms)
mozilla (51,220,480, 96)	20,689,153 → 32,628,521 (242.5 → 289.4 ms)	19,968,503 → 32,133,199 (57.2 → 64.3 ms)	18,227,296 → 30,605,723 (59.8 → 75.9 ms)	26,374,675 → 37,122,499 (18.4 → 16.4 ms)	15,653,780 → 28,342,948 (1123.0 → 1808.5 ms)
mr (9,970,564, 2)	3,849,267 → 3,563,254 (41.0 → 43.0 ms)	3,813,409 → 3,298,618 (8.5 → 9.9 ms)	3,547,526 → 3,273,937 (9.3 → 10.7 ms)	5,425,551 → 5,050,190 (3.1 → 5.5 ms)	**3,291,020 → 3,115,428 (217.6 → 215.3 ms)**
nci (33,553,445, 70)	4,683,017 → 6,470,054 (68.5 → 94.4 ms)	2,848,578 → 4,306,530 (18.1 → 27.3 ms)	2,833,843 → 3,978,300 (19.5 → 28.2 ms)	5,427,457 → 7,487,191 (7.5 → 13.4 ms)	2,702,488 → 3,848,984 (198.5 → 279.8 ms)
ooffice (6,152,192, 48)	3,291,205 → 5,312,087 (43.1 → 36.6 ms)	3,593,224 → 5,433,439 (7.9 → 4.6 ms)	3,126,146 → 5,288,337 (8.6 → 4.0 ms)	4,338,588 → 5,628,192 (2.1 → 1.2 ms)	2,785,576 → 5,090,540 (188.6 → 355.7 ms)
osdb (10,085,684, 2)	4,082,115 → 4,966,574 (42.4 → 43.4 ms)	3,729,819 → 5,038,658 (8.2 → 11.3 ms)	3,501,457 → 4,426,380 (8.2 → 11.5 ms)	5,241,663 → 6,452,704 (3.4 → 4.7 ms)	3,480,512 → 4,415,384 (222.0 → 282.6 ms)
reymont (6,627,202, 7)	2,395,111 → 4,149,987 (28.6 → 48.0 ms)	2,149,170 → 4,320,091 (5.4 → 10.2 ms)	1,937,973 → 3,534,120 (5.8 → 10.1 ms)	3,157,660 → 5,526,116 (2.8 → 4.9 ms)	2,025,012 → 3,407,028 (132.7 → 212.8 ms)
samba (21,606,400, 38)	6,354,529 → 12,748,718 (75.8 → 114.5 ms)	5,487,600 → 12,084,610 (15.3 → 24.8 ms)	4,955,117 → 11,351,649 (15.6 → 27.0 ms)	7,631,657 → 14,141,406 (6.6 → 9.4 ms)	4,932,992 → 11,187,888 (352.1 → 743.5 ms)
sao (7,251,944, 28)	5,564,114 → 5,951,004 (43.4 → 50.8 ms)	6,250,010 → 6,235,116 (9.8 → 10.3 ms)	5,527,504 → 6,013,570 (11.1 → 14.0 ms)	6,790,191 → 7,209,590 (1.6 → 3.9 ms)	4,966,836 → 5,340,092 (353.1 → 357.8 ms)
webster (41,458,703, 11)	15,095,757 → 28,321,426 (176.0 → 280.5 ms)	13,661,986 → 28,494,510 (36.8 → 69.2 ms)	12,096,850 → 26,171,276 (42.3 → 70.5 ms)	19,952,826 → 33,396,378 (16.9 → 28.1 ms)	12,748,808 → 25,529,924 (800.4 → 1648.3 ms)
x-ray (8,474,240, 2)	**6,025,572 → 5,313,865 (58.9 → 56.3 ms)**	**6,772,118 → 5,025,007 (9.3 → 7.7 ms)**	6,085,208 → 4,897,376 (12.1 → 12.2 ms)	8,377,118 → 7,886,120 (0.9 → 3.8 ms)	**5,193,904 → 4,666,844 (331.0 → 301.6 ms)**
xml (5,345,280, 7)	972,555 → 1,490,648 (16.3 → 22.2 ms)	693,317 → 1,281,541 (3.1 → 6.2 ms)	635,299 → 1,069,959 (3.2 → 6.1 ms)	1,176,288 → 1,729,448 (1.9 → 4.9 ms)	647,404 → 1,020,368 (45.9 → 73.0 ms)

4.5 Columnar formats

The same detector chooses the number of columns when unknown binary data is imported into Parquet or ORC (Chapter 17). It wins on kennedy.xls, mr, x-ray and sao, the same four files that respond to record delta.

4.6 What predicts success

Signal	Files where record delta wins	Files where it loses
Fixed-size numeric records	kennedy.xls (13), mr (2), x-ray (2)	—
Smooth sample-to-sample values	mr and x-ray (16-bit medical images)	—
Text or markup	—	every text file
Machine code	—	sum, ooffice, mozilla, samba
Spurious detection on text (R = 3–7)	—	rejected by the fallback

Detection alone is not sufficient: the detector reports a "record length" for almost every file. The size-and-time fallback is what makes the method safe.

Chapter 5 — No-Pain Encoder Tuning

> **Executive summary.** The cheapest improvement to deploy is one that needs no new software on the reading side. Every result in this chapter produces files that the standard, unmodified tool opens exactly as before, only smaller, and in most cases faster to open. The encoder simply searches, file by file, for the settings — and in some cases the pre-filters chosen by HCM's record detector — that the stock format already supports but the default command does not use. An organisation can adopt these results by changing how it *writes* files, with no change to anyone who reads them.

5.1 Common method

For each file, the encoder:

1. tries a set of valid stock-format settings;
2. verifies an exact round trip with the **stock** decoder;
3. times stock decoding in-process against the baseline;
4. keeps the smallest candidate that is not slower, or falls back to the baseline.

Where the record detector contributes — choosing the delta distance for xz, 7-Zip or LZMA2 — the result is credited to this work. Pure level selection of a third party's tool is not credited (Chapter 2).

5.2 xz-hi and lzma-hi

xz-hi searches, per file:

- lc, lp and pb;
- nice_len and match-finder depth;
- an optional pre-filter: delta at the detected record length, or BCJ for executables;
- a dictionary sized to the file;
- a CRC-32 check.

lzma-hi applies the same search in raw LZMA (7-Zip SDK) form.

Summary. On Canterbury (2,810,784 bytes), xz-hi is 6.1% smaller than xz -9e (493,080 → 462,832 bytes) and its total decode is 13% faster in this table (33.0 → 28.8 ms). lzma-hi is 1.4% smaller than LZMA -9 (493,423 → 486,372) and 1% faster to decode. Most of the xz gain is kennedy.xls: 51,868 → 23,656 bytes (−54.4%), decoding 2.2× faster (6.48 → 2.90 ms).

Table 5.A — LZMA SDK -9 vs lzma-hi; xz -9e vs xz-hi (Canterbury). Compressed bytes / decode ms

File	LZMA -9	lzma-hi	xz -9e	xz-hi
alice29.txt	48,464 / 2.60	48,456 / 2.59	48,528 / 2.83	48,472 / 2.73
asyoulik.txt	44,542 / 2.42	44,491 / 2.38	44,592 / 2.59	44,448 / 2.57
cp.html	7,610 / 0.45	7,610 / 0.45	7,652 / 0.49	7,616 / 0.48
fields.c	2,969 / 0.15	2,969 / 0.15	3,032 / 0.19	2,996 / 0.17
grammar.lsp	1,241 / 0.02	1,218 / 0.02	1,292 / 0.04	1,264 / 0.03
kennedy.xls	50,395 / 5.19	46,557 / 5.12	51,868 / 6.48	23,656 / 2.90
lcet10.txt	119,461 / 6.24	119,461 / 6.24	119,488 / 7.29	118,892 / 6.75
plrabn12.txt	165,475 / 8.09	165,456 / 7.96	165,456 / 9.20	165,276 / 8.83
ptt5	42,103 / 2.80	39,025 / 2.71	39,860 / 3.09	39,052 / 3.57
sum	9,401 / 0.62	9,401 / 0.62	9,500 / 0.76	9,384 / 0.71
xargs.1	1,762 / 0.06	1,728 / 0.05	1,812 / 0.07	1,776 / 0.07
Total	**493,423 / 28.6**	**486,372 / 28.3**	**493,080 / 33.0**	**462,832 / 28.8**

Summary. On Silesia (211,938,580 bytes), lzma-hi is 1.2% smaller than LZMA -9 (≈48,752,900 → ≈48,191,700 bytes) and decodes 3.9% faster (2,939 → 2,825 ms). xz-hi is 2.1% smaller than xz -9e and 1.7% faster overall, but it is slower on dickens, mozilla and xml, so only lzma-hi earns the General

award. Byte figures are ≈ (±50 bytes) because this run kept normalised sizes.

Table 5.B — LZMA -9 vs lzma-hi; xz -9e vs xz-hi (Silesia). Compressed bytes (≈) / decode ms

File	LZMA -9	lzma-hi	xz -9e	xz-hi
dickens	≈2,830,500 / 123.0	≈2,827,400 / 122.9	≈2,831,200 / 133.3	≈2,825,600 / 146.2
mozilla	≈13,341,400 / 882.5	≈13,191,000 / 842.0	≈13,376,200 / 982.6	≈13,232,900 / 1022.9
mr	≈2,747,700 / 156.4	≈2,747,700 / 156.4	≈2,751,900 / 172.8	≈2,594,600 / 163.2
nci	≈1,741,100 / 131.0	≈1,443,900 / 107.7	≈1,449,300 / 131.1	≈1,440,100 / 124.0
ooffice	≈2,425,000 / 144.7	≈2,416,300 / 141.0	≈2,427,200 / 166.1	≈2,077,700 / 143.3
osdb	≈2,851,300 / 166.5	≈2,851,300 / 166.5	≈2,844,600 / 195.6	≈2,835,800 / 188.4
reymont	≈1,318,000 / 64.1	≈1,318,000 / 64.1	≈1,315,600 / 82.7	≈1,313,900 / 82.1
samba	≈3,759,600 / 232.4	≈3,726,600 / 225.1	≈3,739,500 / 264.5	≈3,728,000 / 254.0
sao	≈4,416,800 / 283.7	≈4,415,700 / 279.7	≈4,425,700 / 322.7	≈4,417,500 / 323.0
webster	≈8,387,300 / 433.3	≈8,349,100 / 419.0	≈8,368,700 / 482.7	≈8,346,700 / 480.3
x-ray	≈4,479,100 / 292.0	≈4,470,400 / 275.6	≈4,491,300 / 342.2	≈4,164,500 / 292.3
xml	≈454,900 / 29.7	≈434,200 / 25.1	≈434,900 / 30.1	≈433,900 / 30.4
Total	**≈48,752,900 / 2939**	**≈48,191,700 / 2825**	**≈48,455,900 / 3306**	**≈47,411,300 / 3250**

Results.

- **xz-hi:** every Canterbury file is smaller *and* decodes faster than with xz -9e: 493,080 → 462,832 bytes (−6.1%), and 33.0 → 28.8 ms in this table. The first measurement session gave 32.1 → 24.8 ms. kennedy.xls falls from 51,868 to 23,656 bytes, using a delta filter at distance 13. sum falls from 9,500 to 9,384 bytes with the SPARC branch filter. **Award:** *No-Pain / General / Canterbury* in the first measurement session only. In the later Tier 3 re-timing (next bullet), the result held only with kennedy.xls included, so the award is treated as **unconfirmed** until re-timed again.
- **xz-hi without kennedy.xls** decodes about 10.5% slower than xz -9e, so its General award depends on kennedy.xls. lzma-hi passes with or without it: without kennedy.xls, the remaining 1,781,040 bytes compress from about 443,000 to 439,700 bytes (−0.7%).
- **Silesia:** lzma-hi makes every file smaller (≈48,752,900 → ≈48,191,700 bytes, −1.2%, under the strict per-run rule; ≈48,136,800 in the confirmation run), with total decode faster. **Award:** *No-Pain / General / Silesia*. Taking the better of lzma-hi and xz-hi per file reaches ≈47,488,200 bytes (−2.6% vs LZMA -9) at 2,924 ms.
- **Standouts:** nci (33,553,445 bytes) with lzma-hi falls from ≈1,741,100 to ≈1,443,900 bytes (−17.1%) and decodes 1.22× faster (*BGS-1 / Local*). ooffice with xz-hi and the x86 filter falls from ≈2,425,000 to ≈2,077,700 bytes (−14.3%) (*BGS-1 size / Local*).

5.3 7z-hi

7z-hi searches, per file:

- LZMA2 lc, lp, pb and fb;
- the filter: automatic, off, BCJ, BCJ2, ARM64, or Delta at the detected record length.

Stock 7-Zip extracts the output.

Summary. 7z-hi makes every file in both corpora smaller and none slower. Canterbury: 494,902 → 464,853 bytes (−6.1%), decode 64 → 59 ms (8% faster). Silesia: 48,757,480 → ≈47,333,100 bytes (−2.9%, about 1.42 MB saved), decode 3,483 → 3,300 ms (5.3% faster). The biggest single gains are kennedy.xls (−53%, 1.5× faster), ooffice (−16%, 1.25× faster), nci (−17%, 1.20× faster), and x-ray and mr (−6%, 1.1–1.2× faster).

Table 5.C — 7-Zip Ultra -mx9 vs 7z-hi. Compressed bytes / decode ms; chosen 7z-hi setting

File	-mx9	7z-hi	Setting
alice29.txt	48,588 / 5.6	48,588 / 5.6	`-mx9`
asyoulik.txt	44,666 / 5.2	44,612 / 5.1	`-m0=LZMA2:d=64m:fb=273:lc=3:lp=0:pb=0`
cp.html	7,726 / 2.8	7,699 / 2.8	`-m0=LZMA2:d=64m:fb=273:lc=3:lp=0:pb=0`
fields.c	3,085 / 2.6	3,068 / 2.5	`-m0=LZMA2:d=64m:fb=64:lc=1:lp=0:pb=0`
grammar.lsp	1,365 / 2.4	1,365 / 2.4	`-mx9`

File	-mx9	7z-hi	Setting
kennedy.xls	50,529 / 9.9	23,764 / 6.5	`-m0=LZMA2:d=64m:fb=273:lc=1:lp=0:pb=0 -mf=Delta:13`
lcet10.txt	119,592 / 9.8	119,592 / 9.8	`-mx9`
plrabn12.txt	165,653 / 13.0	165,571 / 11.8	`-m0=LZMA2:d=64m:fb=273:lc=3:lp=0:pb=0`
ptt5	42,211 / 6.7	39,289 / 6.3	`-m0=LZMA2:d=64m:fb=273:lc=0:lp=0:pb=0`
sum	9,609 / 3.5	9,461 / 3.4	`-m0=LZMA2:d=64m:fb=273:lc=3:lp=0:pb=2`
xargs.1	1,878 / 2.5	1,844 / 2.5	`-m0=LZMA2:d=64m:fb=273:lc=0:lp=0:pb=0`
Total cant	**494,902 / 64**	**464,853 / 59**	
dickens	≈2,831,100 / 152.8	≈2,828,100 / 149.4	`-m0=LZMA2:d=64m:fb=273:lc=3:lp=0:pb=0`
mozilla	≈13,342,800 / 987.5	≈13,165,100 / 973.4	`-m0=LZMA2:d=64m:fb=273:lc=0:lp=2:pb=2`
mr	≈2,748,300 / 192.9	≈2,592,100 / 161.1	`-m0=LZMA2:d=64m:fb=273:lc=2:lp=1:pb=1 -mf=Delta:2`
nci	≈1,741,400 / 208.0	≈1,444,400 / 173.5	`-m0=LZMA2:d=64m:fb=273:lc=4:lp=0:pb=1`
ooffice	≈2,425,600 / 170.4	≈2,026,300 / 136.6	`-m0=LZMA2:d=64m:fb=273:lc=4:lp=0:pb=2 -mf=BCJ2`
osdb	≈2,851,800 / 194.8	≈2,839,300 / 192.0	`-m0=LZMA2:d=64m:fb=273:lc=4:lp=0:pb=2`
reymont	≈1,318,400 / 82.9	≈1,314,200 / 81.1	`-m0=LZMA2:d=64m:fb=273:lc=4:lp=0:pb=1`
samba	≈3,760,500 / 279.2	≈3,728,100 / 275.2	`-m0=LZMA2:d=64m:fb=273:lc=4:lp=0:pb=0`
sao	≈4,413,900 / 301.6	≈4,408,000 / 292.8	`-m0=LZMA2:d=64m:fb=273:lc=2:lp=2:pb=2 -mf=ARM64`
webster	≈8,388,800 / 552.3	≈8,355,900 / 536.9	`-m0=LZMA2:d=64m:fb=273:lc=4:lp=0:pb=0`
x-ray	≈4,479,900 / 315.9	≈4,197,400 / 287.6	`-m0=LZMA2:d=64m:fb=64:lc=3:lp=1:pb=1 -mf=Delta:2`
xml	≈455,000 / 44.2	≈434,300 / 40.3	`-m0=LZMA2:d=64m:fb=273:lc=3:lp=0:pb=0`
Total sil	**≈48,757,500 / 3483**	**≈47,333,100 / 3300**	

Every file is smaller and none is meaningfully slower: Canterbury 494,902 → 464,853 bytes (464,423 in the confirmation run), and Silesia 48,757,480 → ≈47,333,100 bytes. Without kennedy.xls, the rest of Canterbury is 444,373 → 440,660 bytes (−0.8%). **Award:** *No-Pain / General* on both corpora.

Standouts:

- ooffice falls from ≈2,425,600 to ≈2,026,300 bytes (−16.5%) and decodes 1.25× faster, using BCJ2. 7-Zip's -mx9 skips that filter when the file has no .exe or .dll name. *BGS-1 / Local*.
- nci falls from ≈1,741,400 to ≈1,444,400 bytes (−17.1%) and decodes 1.20× faster.
- mr falls from ≈2,748,300 to ≈2,592,100 bytes (−5.7%, 1.20× faster) with Delta 2; x-ray falls from ≈4,479,900 to ≈4,197,400 bytes (−6.3%, 1.10× faster) with Delta 2.

5.4 zstd-hi and Brotli-hi

Built from current source: zstd 1.6.0-dev and Brotli 1.2.0. The search covers documented advanced parameters, such as minimum match length, block splitting, window and search log for zstd, and window, mode and literal-context modelling for Brotli. Stock decoders read the output.

Summary. zstd-hi gains are small: 1.1% at level 1 (686,780 → 678,908 bytes), 0.4–0.6% at levels 3 and 9, and 0.04–0.12% at levels 19 and 22. Decoding is not slower at any level. kennedy.xls gains most at level 22 (64,814 → 64,205 bytes).

Table 5.D — zstd-hi on Canterbury, per level. Compressed bytes (stock → hi; a single value means no No-Pain gain)

File	L=1	L=3	L=9	L=19	L=22
alice29.txt	59,657	56,970 → 55,736	52,188	49,211	49,211
asyoulik.txt	54,512	50,324	47,542 → 47,523	45,137	45,137
cp.html	8,820	8,464	7,929 → 7,916	7,713	7,713
fields.c	3,556	3,380	3,122	3,015	3,015
grammar.lsp	1,337	1,291	1,241	1,211	1,211
kennedy.xls	115,391	111,827	111,058 → 110,465	64,825 → 64,644	64,814 → 64,205
lcet10.txt	158,520	141,024	130,048 → 128,450	121,336	121,336
plrabn12.txt	217,841 → 209,969	191,684	179,363 → 178,587	167,438 → 167,414	167,438
ptt5	51,666	54,420 → 52,908	49,956 → 49,412	43,640 → 43,633	43,537
sum	13,620	13,375	12,514	11,107	11,101
xargs.1	1,860	1,799	1,747	1,724	1,724
Total	**686,780 → 678,908**	**634,558 → 631,812**	**596,708 → 593,165**	**516,357 → 516,145**	**516,237 → 515,628**

Summary. Brotli-hi gains 0.9% at quality 11 (490,662 → 486,199 bytes), mostly from kennedy.xls (61,498 → 57,075, −7.2%). At quality 2 it gains nothing; at 5 and 8 the gains are tens to hundreds of bytes.

Table 5.E — Brotli-hi on Canterbury, per level. Compressed bytes (stock → hi; a single value means no No-Pain gain)

File	q=2	q=5	q=8	q=11
alice29.txt	57,383	52,809	51,187	46,487 → 46,473
asyoulik.txt	50,507	47,583 → 47,582	46,715 → 46,714	42,712
cp.html	8,443	7,758	7,675	6,894
fields.c	3,445	3,012 → 3,008	2,968 → 2,964	2,717
grammar.lsp	1,352	1,187	1,186	1,124 → 1,121
kennedy.xls	126,306	71,496	68,099 → 67,834	61,498 → 57,075
lcet10.txt	148,449	133,991	128,078	113,416
plrabn12.txt	198,935	186,038 → 186,032	178,163	163,267
ptt5	53,645	48,407	48,409	40,939
sum	13,260	11,562	11,531 → 11,424	10,144
xargs.1	1,876	1,655	1,649	1,464 → 1,441
Total	**663,601 → 663,601**	**565,498 → 565,487**	**545,660 → 545,283**	**490,662 → 486,199**

The gains are small. Confirmation runs with repeated timing gave (Canterbury total bytes, ≈ within 3 bytes): zstd -1 686,780 → 678,886; -3 634,558 → 630,189; -9 596,708 → 591,597; -19 516,357 → 516,113; -22 516,237 → 515,627; Brotli 5 565,498 → 564,495; 8 545,660 → 545,095; 11 490,662 → 485,917. Brotli quality 2 gains nothing, because its encoder ignores those parameters. An earlier apparent speed-up came from parallel runs and was discarded.

5.5 bzip2-hi

These are encoder-only changes; stock bzip2 decodes the output.

- **Per block**, the encoder tries 2–6 Huffman tables × 4, 8, 16 or 32 refinement passes × maximum code length 17 or 20 × 3 starting splits, including contiguous stretches of the block, and keeps the fewest bits.
- **Per file**, it picks the block size (-1 to -9) that is smallest without decoding slower, using strict interleaved timing.

Summary. bzip2-hi is 2.0% smaller on Canterbury (≈542,700 → ≈531,900 bytes) and decodes 7% faster (59.7 → 55.4 ms); kennedy.xls is 7.5% smaller.

Corpus	bzip2 -9	bzip2-hi	Decode (in-process stock libbz2)
Canterbury (bytes)	≈542,700	**≈531,900**	59.7 → 55.4 ms (7% faster)
kennedy.xls (bytes)	130,280	**120,482** (100 KB blocks)	faster

Every file is smaller, and every file decodes faster. *Caveat:* part of the gain comes from choosing the block size, which is a stock level; under the baseline rule that part is not credited. A recheck at a fixed block size is on the open-work list. The per-file table for this run is held in the original laboratory workspace, which is no longer available, and will be regenerated for Appendix A.

5.6 RAR (closed source, switch search only)

RAR 7.30 beta 2, `-m5 -ma5 -mt1`, stock unrar. Decode timing is command-line (±5–10%).

Summary. Most of RAR's gains come from its own `-mcx+` switch and are not credited. Two are credited: sao, where turning off RAR's automatic table filter saves 10.7% (≈5,542,600 → ≈4,949,900 bytes) and is faster; and mr, where the detected delta saves 2.0%. Silesia decode time fell 3.8% (1,059 → 1,019 ms). Table values are compressed bytes (Silesia ≈).

File	RAR -m5	Best No-Pain	Switch	Credited?
alice29.txt	51,189	50,268	-mcx+	No (RAR's own)
asyoulik.txt	46,929	46,046	-mcx+	No
cp.html	8,079	7,982	-mcx+	No (decode not confirmed faster)
fields.c	3,160	no gain	—	—
grammar.lsp	1,325	1,315	-mcx+	No
kennedy.xls	35,555	35,048	-mcx+	No
lcet10.txt	126,437	123,932	-mcx+	No
plrabn12.txt	175,598	171,643	-mcx+	No
ptt5	49,260	47,213	-mcx+	No (decode not confirmed faster)
sum	11,601	11,398	-mcx+	No (decode not confirmed faster)
xargs.1	1,827	1,810	-mcx+	No (decode not confirmed faster)
Canterbury total	**510,961**	**499,814**		
dickens	≈3,032,300	≈2,929,700	-mcx+	No
mozilla	≈15,155,400	≈15,151,500	-mcl+	No
mr	≈2,769,900	**≈2,715,700**	detected delta 2 + -mcx+	**Yes**
nci	≈1,858,800	≈1,724,600	-mcx+	No
ooffice	≈2,302,300	no gain	—	—
osdb	≈3,237,000	no gain	—	—
reymont	≈1,444,800	≈1,389,300	-mcx+	No
samba	≈4,106,500	no gain	—	—
sao	≈5,542,600	**≈4,949,900**	automatic delta *disabled* (-mcd-)	**Yes** — search found RAR's filter misfiring
webster	≈9,035,000	no gain	—	—
xml	≈484,600	≈469,300	-mcx+	No
x-ray	≈4,160,600	≈4,148,400 (first run)	detected delta 2	Within timing noise
Silesia total	**≈53,129,800**	**≈52,171,400** (decode 1,059 → 1,019 ms)		

RAR already detects tabular data internally, which is why record delta adds little. The notable finding is sao: RAR's own table filter costs about 592,700 bytes (12% more than without it) on that file, and turning it off is both smaller and faster.

Chapter 6 — PDF Documents

Executive summary. PDF is the most widely exchanged document format in the world, and its standard already defines a small “row filter” for compressed data. Using HCM’s record detector to choose that filter’s row width for each piece of a PDF — and keeping it only where it helps — made 139 PDFs from a public test collection — the bug-test suite of Mozilla’s pdf.js reader, rich in unusual and damaged files — smaller than the widely used free optimiser qpdf. In the No-Pain configuration, it is also not slower to render than qpdf’s output or the original files in our test. The first three pages of every file rendered pixel-for-pixel identically at the resolution tested, and the files use only standard PDF features. Compatibility was verified with one reader (poppler), not across all readers.

6.1 Background

A PDF stores page descriptions, fonts and images as *streams*, usually compressed with Deflate (`/FlateDecode`), the same method as gzip. The PDF standard lets a Flate stream declare a **predictor** in its `/DecodeParms`. Predictor 12 is the PNG “Up” filter: each byte is stored as its difference from the byte one row above, with the row width given by `/Columns`. This is exactly *record delta* (Chapter 4), and conforming readers must decode it. Writers rarely use it except on images, and when they do, the row width is fixed by the image description.

The established free tool **qpdf** optimises PDFs losslessly by packing objects into compressed object streams and recompressing every Flate stream at level 9. That is the baseline here, and it is known work.

6.2 Method

For every stream that qpdf can decode — Flate, LZW, RunLength, ASCII filters, or none — and that is at least 64 bytes long:

1. **Decode** the stream to raw bytes.
2. **Candidates:**
    - Deflate level 9 with the default and the “filtered” zlib strategies;
    - when the record detector (Chapter 4) finds a record length *R* between 2 and 512 that divides the stream length, the same two strategies applied after the PNG Up filter with `/Columns R`.
3. **Keep the smallest** and write it with the matching `/Filter` and `/DecodeParms`.
4. **Leave untouched** streams in formats that are already compressed images (DCT/JPEG, JPX, JBIG2, CCITT).
5. **Write** the file with qpdf’s object-stream packing at level 9, without re-decoding the rewritten streams.

Verification.

- Each output passes `qpdf --check`.
- The first three pages of the original, the qpdf version and our version are rendered with poppler (`pdftoppm`, 36 dpi, greyscale), and the pixel output must be identical.
- Render time is the median of three runs.

No-Pain pick. Per file, our version is used only if it is smaller than qpdf’s *and* its measured render time is not slower. Otherwise qpdf’s version is used. Because this choice uses measured render times, it is an experimental selection; a production tool would need a cheap predictor of render cost (Section 6.7).

Class. *Modified* — stock readers, standard format; the row width is chosen by the author’s detector.

6.3 Test set

The PDF test suite of Mozilla’s pdf.js reader: 1,012 PDFs, of which the 146 larger than 100 KB were used. Of these:

- 2 failed qpdf’s own processing (144 remain);
- 2 more failed our rewrite step — `issue3188.pdf` and `poppler-90-0-fuzzed.pdf`, a deliberately corrupted file (142 remain);
- 1 decompression-bomb test file, `bomb_giant.pdf`, was excluded from rendering because it exhausts memory by design (141 remain);
- 2 render differently even after qpdf’s own optimisation, being nondeterministic or damaged, and are excluded from the comparison.

139 files, 95,872,155 bytes, remain. The set is a reader's bug collection and is unusually rich in damaged and edge-case files. Results on an ordinary office document collection may differ.

6.4 Results

Summary. Against qpdf's output (81,681,005 bytes), the No-Pain pick saves 507,703 bytes (−0.6%) with total render time not slower (11.35 → 11.20 s). Against the original files (95,872,155 bytes) it saves 14.7 MB (−15.3%). Always using the row filter saves 1.7% against qpdf (1,404,000 bytes), but renders about 2% slower, so it fails the No-Pain test.

Version	Bytes (139 files)	vs original	vs qpdf	Render time (first 3 pages)
Original files	95,872,155	—	—	11.22 s
qpdf optimisation (known)	81,681,005	−14.8%	—	11.35 s
Ours, smallest always	≈80,277,600	−16.3%	−1.7%	11.60 s (2.2% slower than qpdf)
Ours, No-Pain pick	**81,173,302**	**−15.3%**	**−0.6%**	**11.20 s** (not slower)

- **Smallest-always** is 1.7% smaller than qpdf: smaller on 86 files, larger on 41, tied on the rest. The row filter costs about 2% in render time, because the reader must undo the filter.
- **The No-Pain pick** uses our version on 52 files. It is smaller than qpdf, and its total render time (11.20 s) is not slower than qpdf's (11.35 s) or the originals' (11.22 s). These differences are within the few-percent timing noise, so the result is stated as *not slower*, not as faster. **Award: No-Pain (Baseline, vs qpdf) / General / pdf.js test set**, with the caveat that render times are command-line medians of three runs.

6.5 Notable files

File	Original (bytes)	qpdf (bytes)	Ours (bytes)	Why
colorspace_cos.pdf	140,325	140,249	**1,900**	Smooth gradient image; the detector finds R = 3 (one RGB pixel). The image stream falls from 138,960 to 575 bytes.
colorspace_sin.pdf	140,325	140,250	**1,901**	Same
colorspace_atan.pdf	140,340	140,257	**1,908**	Same
bug1473809.pdf	178,088	171,081	**104,391**	Image data with a row structure
issue14814.pdf	419,768	506,692	**355,817**	qpdf's output is *larger* than the original; ours is 30% smaller than qpdf
issue14200.pdf	679,384	638,002	**481,359**	Row-structured stream data
basicapi.pdf	105,779	100,203	**84,724**	—

6.6 A correctness issue found and fixed

The rewrite passes each PDF through qpdf's JSON representation. In the first run, one file — `filled-background.pdf`, whose catalogue stores π to 100,001 digits — came out 99% smaller. Investigation showed the JSON reader had rounded that number to double precision. The pages rendered identically, so a render check alone would **not** have caught it. Numbers are now carried through as exact text; the file comes out at qpdf's size, with all 100,001 digits intact; and the whole test set was re-run. The episode is recorded because it shows why pixel comparison is necessary but not sufficient for "lossless" claims about PDF.

6.7 Limitations

- **Render verification and timing** cover the first three pages of each file at 36 dpi in greyscale, using the command-line renderer. Timing is accurate to a few percent, and later pages and higher resolutions were not checked.
- **Selection.** The No-Pain pick uses measured render times per file, so it is an upper bound on what an automatic selector could achieve. A production tool needs a rule decided at write time, such as using the row filter only above a size-saving threshold.
- **Untouched content.** JPEG and other already-compressed images are left as they are; lossless JPEG repacking is known work.
- **Reader compatibility.** Predictor 12 is defined in PDF 1.2 and later, but a standard feature is not proof that every reader handles every legal parameter combination, such as unusual `/Columns` values on non-image streams. Only poppler was tested. Testing with pdf.js, MuPDF, Adobe Acrobat and the readers built into Chrome and macOS is needed before claiming broad compatibility.

- **Non-visual content.** Rendering checks cover page appearance only. Annotations, form fields, embedded files, metadata, bookmarks and JavaScript were carried through qpdf's object model but were not individually compared. The π-precision bug (Section 6.6) shows that identical pages do not prove a faithful document; a full object-by-object comparison is planned.
- **Digital signatures.** Any rewrite of a signed PDF invalidates its signatures. Signed files must be excluded or re-signed; this test set was not screened for signatures.
- **Test set.** A bug collection, not a typical office archive.

6.8 Complete per-file results

Compressed bytes. Bold marks the smaller of qpdf and ours. Render times are in milliseconds.

File	Original (bytes)	qpdf (bytes)	Ours (bytes)	Ours vs qpdf	No-Pain pick	Row-filter streams	Render ms: orig / qpdf / ours
160F-2019.pdf	328,400	**312,362**	313,953	+0.5%	956,008	qpdf	0
22060_A1_01_Plans.pdf	6,264,322	6,262,899	**6,262,636**	-0.0%	**999,731**	qpdf	0
basicapi.pdf	105,779	100,203	**84,724**	-15.4%	**800,953**	qpdf	2
bug1200096.pdf	117,519	**59,610**	59,763	+0.3%	508,539	qpdf	0
bug1245391_reduced.pdf	351,793	317,002	**316,997**	-0.0%	**901,090**	qpdf	0
bug1473809.pdf	178,088	171,081	**104,391**	-39.0%	**586,176**	qpdf	1
bug1627427_reduced.pdf	264,448	**226,228**	226,698	+0.2%	857,250	qpdf	0
bug1669099.pdf	187,596	**167,887**	170,217	+1.4%	907,359	qpdf	0
bug1703683_page2_reduced.pdf	330,131	326,511	**326,009**	-0.2%	**987,514**	ours	0
bug1721218_reduced.pdf	825,329	784,895	**771,243**	-1.7%	**934,467**	qpdf	10
bug1727053.pdf	420,987	**418,063**	420,644	+0.6%	999,185	qpdf	0
bug1743245.pdf	271,984	270,999	**264,935**	-2.2%	**974,083**	ours	0
bug1755507.pdf	498,287	495,086	**494,592**	-0.1%	**992,585**	ours	1
bug1795263.pdf	355,886	306,076	**306,069**	-0.0%	**860,020**	ours	0
bug1815476.pdf	134,495	132,707	**131,708**	-0.8%	**979,278**	qpdf	0
bug1885505.pdf	109,851	109,057	**109,003**	-0.0%	**992,280**	ours	0
bug1888084.pdf	111,656	**107,747**	110,207	+2.3%	987,023	qpdf	0
bug1978317.pdf	1,535,205	1,498,726	**1,495,685**	-0.2%	974,258	excluded (render differs even with qpdf)	1
bug1992868.pdf	1,017,948	867,210	**833,850**	-3.8%	**819,148**	qpdf	57
bug1997343.pdf	189,463	144,812	**144,094**	-0.5%	**760,539**	ours	0
bug854315.pdf	107,532	**106,266**	106,266	+0.0%	988,227	qpdf	0
bug886717.pdf	133,458	**122,478**	123,451	+0.8%	925,018	qpdf	0
bug900822.pdf	166,510	**162,431**	163,746	+0.8%	983,400	qpdf	0
bug920426.pdf	392,084	**203,157**	204,266	+0.5%	520,975	qpdf	0
calrgb.pdf	322,053	19,804	**19,774**	-0.2%	**61,400**	ours	0
canvas.pdf	150,904	149,407	**145,106**	-2.9%	**961,578**	qpdf	12
chrome-text-selection-markedContent.pdf	137,291	132,843	**132,837**	-0.0%	**967,558**	ours	0
cmykjpeg.pdf	374,080	337,084	**331,659**	-1.6%	**886,599**	ours	1
colorkeymask.pdf	161,533	**2,117**	2,117	+0.0%	13,106	qpdf	0
colorspace_atan.pdf	140,340	140,257	**1,908**	-98.6%	**13,596**	ours	1
colorspace_cos.pdf	140,325	140,249	**1,900**	-98.6%	**13,540**	qpdf	1
colorspace_sin.pdf	140,325	140,250	**1,901**	-98.6%	**13,547**	qpdf	1
comments.pdf	1,065,760	913,980	**880,535**	-3.7%	**826,204**	ours	58
file_pdfjs_test.pdf	150,611	128,464	**128,452**	-0.0%	**852,873**	qpdf	0
filled-background.pdf	401,129	**143,065**	143,065	+0.0%	356,656	qpdf	0
firefox_logo.pdf	234,678	**232,774**	232,774	+0.0%	991,887	qpdf	0
freeculture.pdf	2,598,727	**2,387,377**	2,387,441	+0.0%	918,696	qpdf	0
highlights.pdf	969,276	874,436	**840,985**	-3.8%	**867,642**	ours	58
images.pdf	1,494,439	1,463,731	**1,361,381**	-7.0%	**910,965**	qpdf	2
inks.pdf	108,628	106,216	**105,283**	-0.9%	**969,207**	qpdf	0
issue10529.pdf	120,681	**116,520**	117,088	+0.5%	970,227	qpdf	0

File	Original (bytes)	qpdf (bytes)	Ours (bytes)	Ours vs qpdf	No-Pain pick	Row-filter streams	Render ms: orig / qpdf / ours
issue11922_reduced.pdf	1,001,563	**521,456**	524,256	+0.5%	523,438	qpdf	0
issue12007_reduced.pdf	253,720	252,191	**252,190**	-0.0%	**993,970**	ours	0
issue12295.pdf	375,776	**377,626**	379,757	+0.6%	1,010,594	qpdf	0
issue12337.pdf	1,016,953	866,479	**833,101**	-3.9%	**819,213**	qpdf	57
issue12798_page1_reduced.pdf	128,707	126,472	**126,228**	-0.2%	**980,739**	ours	0
issue12810.pdf	253,783	**250,982**	252,825	+0.7%	996,225	qpdf	0
issue12841_reduced.pdf	5,852,805	**5,852,622**	5,852,622	+0.0%	999,969	qpdf	0
issue12963.pdf	603,342	480,345	**479,926**	-0.1%	**795,446**	ours	0
issue13193.pdf	990,389	**988,353**	995,795	+0.8%	1,005,458	qpdf	0
issue13201.pdf	149,306	**145,621**	145,820	+0.1%	976,652	qpdf	0
issue13211.pdf	107,307	**97,597**	97,980	+0.4%	913,081	qpdf	0
issue1350.pdf	118,396	90,268	**90,240**	-0.0%	**762,188**	ours	0
issue13520.pdf	754,579	430,534	**428,666**	-0.4%	**568,086**	qpdf	0
issue13931.pdf	303,176	301,122	**301,105**	-0.0%	**993,169**	ours	0
issue14200.pdf	679,384	638,002	**481,359**	-24.6%	**708,523**	qpdf	1
issue14297.pdf	242,940	**234,493**	235,935	+0.6%	971,166	qpdf	0
issue14438.pdf	130,683	109,143	**109,090**	-0.0%	**834,768**	ours	0
issue14618.pdf	159,855	**96,766**	96,972	+0.2%	606,625	qpdf	0
issue14814.pdf	419,768	506,692	**355,817**	-29.8%	**847,652**	qpdf	2
issue14821.pdf	312,363	**229,219**	230,908	+0.7%	739,230	qpdf	1
issue15012.pdf	108,381	98,548	**98,498**	-0.1%	**908,812**	ours	0
issue15053.pdf	108,405	**88,316**	88,525	+0.2%	816,614	qpdf	0
issue15096.pdf	203,730	97,897	**86,928**	-11.2%	**426,682**	ours	32
issue15372.pdf	109,219	**96,262**	96,692	+0.4%	885,304	qpdf	0
issue15629.pdf	159,165	33,750	**33,748**	-0.0%	**212,032**	ours	0
issue16091.pdf	107,355	10,441	**10,366**	-0.7%	**96,558**	qpdf	1
issue16316.pdf	109,744	101,831	**101,782**	-0.0%	**927,449**	ours	0
issue16553.pdf	718,356	203,607	**197,260**	-3.1%	**274,599**	qpdf	2
issue17069.pdf	706,317	**639,634**	640,155	+0.1%	906,328	qpdf	0
issue17730.pdf	102,596	**101,361**	101,493	+0.1%	989,249	qpdf	0
issue17808.pdf	979,045	**486,488**	488,645	+0.4%	499,104	qpdf	0
issue18032.pdf	103,353	100,235	**100,153**	-0.1%	**969,038**	ours	0
issue18911.pdf	1,018,050	867,035	**833,647**	-3.9%	**818,866**	ours	57
issue1905.pdf	918,339	865,283	**865,251**	-0.0%	**942,191**	ours	0
issue19182.pdf	175,528	170,650	**136,361**	-20.1%	**776,862**	qpdf	1
issue19239.pdf	1,060,670	909,821	**876,443**	-3.7%	**826,311**	ours	57
issue19360.pdf	1,223,311	1,156,500	**1,140,333**	-1.4%	**932,169**	ours	0
issue19517.pdf	6,262,577	6,260,802	**6,260,765**	-0.0%	**999,711**	ours	0
issue19633.pdf	312,652	311,308	**311,253**	-0.0%	**995,525**	ours	0
issue19971.pdf	1,729,186	1,785,141	**1,737,610**	-2.7%	**1,004,872**	ours	1
issue20513.pdf	1,157,253	**607,300**	610,396	+0.5%	527,453	qpdf	1
issue20930.pdf	141,480	**91,155**	91,255	+0.1%	645,003	qpdf	0
issue2462.pdf	139,946	138,430	**138,406**	-0.0%	**988,996**	qpdf	0
issue269_2.pdf	271,024	240,977	**240,971**	-0.0%	**889,113**	ours	0
issue2956.pdf	103,556	99,945	**85,173**	-14.8%	**822,483**	qpdf	2
issue3205r.pdf	126,273	**78,271**	78,357	+0.1%	620,536	qpdf	0
issue4061.pdf	115,240	**112,104**	113,512	+1.3%	985,005	qpdf	0
issue4875.pdf	202,841	**199,582**	200,680	+0.6%	989,346	qpdf	0
issue5475.pdf	166,026	**165,621**	165,621	+0.0%	997,561	qpdf	0
issue5481.pdf	1,720,095	**1,718,487**	1,718,487	+0.0%	999,065	qpdf	0
issue5549.pdf	830,412	**829,695**	829,695	+0.0%	999,137	qpdf	0
issue5567.pdf	2,169,844	**2,169,126**	2,169,126	+0.0%	999,669	qpdf	0
issue5677.pdf	194,851	30,156	**30,151**	-0.0%	**154,739**	ours	0
issue5747.pdf	112,018	105,927	**105,863**	-0.1%	**945,053**	qpdf	0

File	Original (bytes)	qpdf (bytes)	Ours (bytes)	Ours vs qpdf	No-Pain pick	Row-filter streams	Render ms: orig / qpdf / ours
issue5874.pdf	214,331	214,449	**214,364**	-0.0%	**1,000,154**	ours	0
issue5946.pdf	239,220	**229,811**	231,278	+0.6%	966,800	qpdf	0
issue5994.pdf	1,248,986	766,581	**765,386**	-0.2%	**612,806**	ours	0
issue6068.pdf	432,895	**430,515**	433,287	+0.6%	1,000,906	qpdf	0
issue6127.pdf	1,446,634	667,387	**665,603**	-0.3%	**460,105**	qpdf	0
issue6132.pdf	119,101	105,091	**104,893**	-0.2%	**880,706**	qpdf	0
issue6286.pdf	1,909,445	1,833,814	**1,832,383**	-0.1%	**959,642**	qpdf	0
issue6410.pdf	211,633	**205,366**	206,427	+0.5%	975,401	qpdf	0
issue6605.pdf	174,718	**58,895**	58,895	+0.0%	337,086	qpdf	0
issue6721_reduced.pdf	103,344	102,594	**102,577**	-0.0%	**992,578**	ours	0
issue6889.pdf	513,863	**511,454**	513,894	+0.5%	1,000,060	qpdf	0
issue6894.pdf	1,372,448	**1,362,536**	1,369,250	+0.5%	997,670	qpdf	0
issue6961.pdf	2,156,587	**2,037,115**	2,038,962	+0.1%	945,458	excluded (render differs even with qpdf)	0
issue7014.pdf	114,757	103,187	**103,128**	-0.1%	**898,664**	ours	0
issue7229.pdf	1,153,154	**1,152,585**	1,152,585	+0.0%	999,507	qpdf	0
issue7426.pdf	117,531	58,421	**57,438**	-1.7%	**488,705**	ours	0
issue7454.pdf	101,777	**96,692**	96,928	+0.2%	952,357	qpdf	0
issue840.pdf	152,333	71,657	**71,579**	-0.1%	**469,885**	ours	0
issue8570.pdf	229,717	**227,644**	232,726	+2.2%	1,013,099	qpdf	0
issue8795_reduced.pdf	4,244,537	3,762,698	**3,758,976**	-0.1%	**885,603**	qpdf	0
issue9262_reduced.pdf	5,536,057	**5,431,281**	5,438,537	+0.1%	982,385	qpdf	0
issue9418.pdf	151,088	552	**521**	-5.6%	**3,448**	ours	0
issue9458.pdf	106,412	99,689	**99,605**	-0.1%	**936,032**	qpdf	1
issue9940.pdf	394,734	**383,110**	384,596	+0.4%	974,317	qpdf	0
issue9949.pdf	844,757	584,759	**584,483**	-0.0%	**691,895**	ours	0
large_jpeg_downscale.pdf	121,794	**121,946**	121,946	+0.0%	1,001,248	qpdf	0
mixedfonts.pdf	405,432	388,855	**384,113**	-1.2%	**947,417**	qpdf	1
multimedia_annotations.pdf	115,662	113,392	**113,118**	-0.2%	**978,005**	qpdf	0
PDFJS-9279-reduced.pdf	3,090,252	1,435,337	**1,435,265**	-0.0%	**464,449**	ours	0
pdfjs_wikipedia.pdf	207,960	140,684	**140,394**	-0.2%	**675,101**	ours	0
pdkids.pdf	546,144	524,414	**524,198**	-0.0%	**959,816**	ours	0
personwithdog.pdf	521,565	408,750	**402,106**	-1.6%	**770,960**	ours	0
pr12564.pdf	147,420	107,003	**106,299**	-0.7%	**721,062**	qpdf	1
prefilled_f1040.pdf	306,416	126,458	**126,124**	-0.3%	**411,610**	qpdf	0
S2.pdf	478,490	432,616	**429,682**	-0.7%	**897,996**	ours	0
stamps.pdf	107,372	104,417	**104,320**	-0.1%	**971,575**	ours	0
TAMReview.pdf	674,629	626,817	**619,389**	-1.2%	**918,118**	qpdf	0
ThuluthFeatures.pdf	117,677	**112,213**	112,390	+0.2%	955,072	qpdf	0
tracemonkey.pdf	1,016,315	866,294	**832,925**	-3.9%	**819,554**	qpdf	57
tracemonkey_a11y.pdf	110,108	99,404	**99,353**	-0.1%	**902,323**	ours	0
tracemonkey_annotation_on_page_8.pdf	1,052,147	901,612	**868,247**	-3.7%	**825,215**	ours	57
tracemonkey_freetext.pdf	1,017,319	866,831	**833,457**	-3.9%	**819,268**	ours	57
tracemonkey_with_annotations.pdf	1,021,717	870,180	**836,829**	-3.8%	**819,042**	ours	57
tracemonkey_with_editable_annotations.pdf	1,212,647	1,104,235	**1,070,809**	-3.0%	**883,034**	ours	58
transparency_group.pdf	347,690	320,406	**320,305**	-0.0%	**921,237**	ours	0
xfa_filled_imm1344e.pdf	3,023,968	**377,363**	379,116	+0.5%	125,370	qpdf	0

Chapter 7 — Combary

> **Executive summary.** Combary is the author's ternary representation. Its name joins the beginning of *combination* to the end of *binary*. It grew out of work on combinations and decimal representations, and it splits ternary information into two distinct components: ordinary binary values (0 and 1), and a *combination* — the positions held by a third symbol, 2. Its original purpose was to let a single ternary language carry two different kinds of data, or information separated into two representations, for storage and transmission. It was not designed as a compression method. Compression uses came later, and they produced two of the author's most successful mechanisms: the ternary pointer inside HCM and the activity map in front of standard compressors.

7.1 Definition

A Combary string is a sequence over the alphabet {0, 1, 2} with distinct roles for the symbols:

- **0 and 1** carry binary value in the ordinary way.
- **2** is a *location marker*. It does not contribute a value; its position is itself the information. It can stand for "here", "the boundary is at this point", "this position refers elsewhere", or "this position is active".

The system therefore splits a ternary stream into two kinds of information: a binary payload, and a *combination*, meaning the set of positions held by the 2s. Because the positions of k markers among n slots can be any of C(n, k) arrangements, a marker pattern can carry up to $\log_2 C(n, k)$ bits on top of the binary payload. Whether that capacity is a net gain depends entirely on how much the marker positions are already predictable from the data. Section 7.4 shows this is the deciding factor in practice.

7.2 Origin and original purpose

Combary began with combinations and decimal representations, not with compression. Its central idea is that ternary information can be separated into two parts that travel together: the binary values carried by 0 and 1, and the combination formed by where the 2s are placed. This lets one ternary stream carry two different data types at once, or one body of information split into two representations.

The original applications are data storage, data transmission and data encryption. A 2 need not be a written symbol: it can be a **physical location** on a medium, or a **variation in time** within a transmission. The author was granted patents that apply to Combary in these areas; one, U.S. 11,636,878, remains in force (Section 24.4). They concern storage, transmission and encryption, not the compression methods in this book; see Chapter 24. Those applications are outside the scope of this book and are not evaluated here.

Only later was the same idea applied to compression, where the "2" became a marker that points, flags or cuts (Section 7.3).

7.3 Combary in compression

Three compression mechanisms were derived directly from Combary. Each maps the 2 to a different role.

Role of the 2	Mechanism	Where	Outcome
Points elsewhere	**Ternary pointer:** before a byte is coded, a single "2" decision asks whether it equals the byte predicted by a match or context; on "no", that byte is excluded from the remaining coding	Inside HCM (hcm-trit v2)	Success. Every Canterbury file improved; best HCM result to date (Chapter 11)
Position is active	**Activity map:** within each detected record, 2s mark the bytes that differ from the record above; only those bytes are stored	In front of gzip, zstd, LZ4, Snappy, LZFSE	Success on fixed-record data with gzip, where it is No-Pain; elsewhere smaller but slower (Chapter 8)
Cut here	**Segment marker:** 2s mark boundaries where the transform choice changes from block to block	In front of zstd, LZ4, gzip	Failure. The cost of the markers exceeded the gain (Chapter 22)

7.4 Why two roles succeeded and one failed

The three outcomes follow a single principle. A Combary marker is profitable only when **its position is cheap to code relative to the information it unlocks.**

- **Ternary pointer.** The “is it the pointed-to byte?” decision is coded with a probability taken from the model’s own prediction. *Correction (29 October 2026):* with a well-calibrated model, coding the flag and then the byte with the candidate excluded costs exactly what coding the byte directly would, so the flag by itself cannot save bits. An independent control by Andrew Goodwin, re-run and confirmed here (Section 11.8), shows the pointer’s gain comes entirely from its refinement mixer, which re-estimates the candidate’s probability from flag history, match length and an order-2 context (secondary estimation).
- **Activity map.** On fixed-record data most positions repeat the record above. The flag stream is long runs of “same” that general-purpose compressors reduce to almost nothing, while the “active” positions isolate exactly the bytes that carry new information. On kennedy.xls (1,029,744 bytes), gzip output fell from 209,721 to 35,558 bytes, an 83.0% reduction, while decoding 1.5× faster.
- **Segment marker.** Block boundaries in the Canterbury and Silesia files do not coincide with real changes in data character often enough. The markers cost a few bytes each, the per-block transform choice gained less, and every configuration came out larger than choosing one transform for the whole file. Silesia-scale files with genuinely mixed content, such as archives of heterogeneous files, remain an untested case where this role could succeed.

7.5 Relation to existing ideas

The pointer role resembles the *escape* and *exclusion* mechanisms of PPM and the match models of context-mixing compressors. The activity map resembles sparse-difference and run-length encodings of change masks used in database page compression. What Combary contributes is a single organising principle — a value-free location symbol — from which all three mechanisms were derived. In hindsight, that principle accounts for which of them worked; it was not used to predict the outcomes before the experiments.

7.6 Open questions

1. A formal account of when a marker pattern’s positional entropy is lower than the information it isolates, giving a test that predicts profitability before any experiment is run.
2. Variable-length and multi-level markers, such as a 2 that also selects between several references.
3. The segment-marker role on heterogeneous archives, where boundaries are real.

Chapter 8 — The Combary Activity Map

> **Executive summary.** The activity map is the author's Combary idea applied to tables. Within each detected record, it marks which bytes changed since the row above and stores only those bytes; unchanged positions are copied back on decoding. Paired with gzip, the most widely deployed compressor in existence, it shrank the Canterbury spreadsheet to one sixth of gzip's normal output while also decoding faster. With faster compressors the extra decoding pass usually costs more time than it saves, so the method is offered as a size option there, not a free gain.

8.1 Method

Given record length *R* from the detector (Chapter 4), the activity map produces two streams:

- **Flags** — one bit per input byte: 1 if $x_i \neq x_{i-R}$ (the position is *active*, the Combary "2"); 0 otherwise. Packed 8 per byte.
- **Literals** — the active bytes, in order.

Each stream is compressed separately by the chosen back-end compressor. To decode, both streams are decompressed; where a flag is 0, the byte is copied from one record back, and where it is 1, the next literal is taken. The method is exactly lossless. The header holds the record length and stream sizes (about 6 bytes).

Class: Original (method) / Mutated (pipeline).

8.2 Results with standard compressors — kennedy.xls (R = 13)

Summary. On kennedy.xls (1,029,744 bytes) the activity map is the smallest result for gzip, LZ4, Snappy and LZFSE: −83.0% with gzip (209,721 → 35,558 bytes), −87.7% with LZ4, −83.5% with Snappy and −69.7% with LZFSE. Only with gzip is it also faster to decode: about 1.5× faster than plain gzip (2,148 → ≈1,400 µs), although record delta decodes faster still (1,054 µs). With zstd, record delta is both smaller and faster.

Compressed bytes; decode time in-process.

Back end	Plain	Record delta	Activity map	Decode: plain / delta / activity	Activity verdict
gzip -9	209,721	42,971	**35,558**	2,148 / 1,054 / ≈1,400 µs	**No-Pain** vs gzip; smallest gzip result
zstd -22	64,818	**26,401**	30,340	1,359 / 520 / — µs	Record delta better
LZ4 -12	324,136	43,653	**39,997**	739 / 725 / ≈1,000 µs	Smaller but slower — fails
Snappy	422,640	129,763	**69,635**	966 / 717 / 1,030 µs	Smaller but slower — fails
LZFSE	136,282	54,844	**41,227**	1,064 / 767 / 1,098 µs	Smaller but slower — fails

Award: *No-Pain / Local / kennedy.xls* with gzip. Decode times for the gzip activity map varied between 1,313 and 1,476 µs across runs, always below plain gzip's 2,148 µs.

8.3 Results on other files

Summary. Outside kennedy.xls the activity map often shrinks output — by 27% on mr and 39% on x-ray with Snappy — but it always decodes slower, by 28% to about 10×, so it fails the No-Pain test on every other file. Silesia sizes are ≈ (derived from normalised data).

File	Back end	Plain	Activity map	Decode plain → activity	Verdict
ptt5	gzip -9	52,377	52,035	faster → slower	Fails
ptt5	Snappy	92,869	80,081	365 → 882 µs	Fails
ptt5	LZFSE	56,334	53,136	562 → 801 µs	Fails
mr	Snappy	≈5,399,500	≈3,956,400	9.5 → 15.5 ms	Fails (BGS candidate, size −27%)
mr	LZFSE	≈3,580,200	≈3,154,200	23.1 → 31.9 ms	Fails
x-ray	Snappy	≈8,457,900	≈5,181,600	1.1 → 11.4 ms	Fails (BGS-2 size, ×10 slower)
x-ray	LZFSE	≈5,988,100	≈4,937,200	17.5 → 22.4 ms	Fails (−18% size, +28% time)

File	Back end	Plain	Activity map	Decode plain → activity	Verdict
sao	Snappy	≈6,437,600	≈6,085,800	8.1 → 17.4 ms	Fails
sao	LZFSE	≈5,441,000	≈5,165,500	17.5 → 30.1 ms	Fails
Text files	all	—	larger	—	Fallback

The activity map consistently gives the largest size reduction of any transform on record data. Its weakness is the per-byte branch in the reconstruction loop. A branch-free version — expanding flags with SIMD masks — is the obvious engineering fix (Chapter 23).

8.4 Variants tested

Variant	kennedy.xls flag stream (gzip / LZ4 / zstd, bytes)	Verdict
Packed flags, 1 bit per byte (adopted)	**12,324 / 13,533 / 9,640**	Best
13-bit mask per record, stored in 2 bytes	14,088 / 15,797 / 10,660	Worse
Mask split into low and high bytes	14,089 / 15,749 / 11,099	Worse
Frequency-ranked mask codes (361 distinct of 8,192), with table	12,832 / 14,306 / 10,678	Worse
Literals stored as deltas instead of raw bytes	larger on every file	Worse
Optimised branch-reduced rebuild (8 flags at a time)	—	Slower on this CPU

Padding 13 bits to 16 wastes 3 bits per record. General-purpose compressors already find the recurring patterns in the contiguous bit stream.

8.5 Activity map in front of HCM

Summary. In front of HCM the activity map always makes the output larger (by 5–35%), so it earns no award. On structured files it does cut decode time sharply: kennedy.xls 0.71 → 0.27 s (2.6× faster), ptt5 0.33 → 0.13 s (2.5×), x-ray 11.4 → 7.4 s (1.6×), nci 21.5 → 13.6 s (1.6×).

Compressed bytes (Silesia ≈), decode time.

File (R)	HCM fast plain	+ activity map	HCM max plain	+ activity map
alice29.txt (4)	36,636 (0.16 s)	47,527 (0.20 s)	36,267 (0.21 s)	47,183 (0.22 s)
asyoulik.txt (4)	34,348 (0.15 s)	42,397 (0.17 s)	34,138 (0.19 s)	42,172 (0.23 s)
cp.html (5)	6,437 (0.05 s)	7,700 (0.05 s)	6,388 (0.05 s)	7,671 (0.06 s)
fields.c (11)	2,485 (0.03 s)	3,332 (0.02 s)	2,477 (0.03 s)	3,327 (0.03 s)
grammar.lsp (2)	1,047 (0.01 s)	1,222 (0.01 s)	1,038 (0.01 s)	1,216 (0.01 s)
kennedy.xls (13)	18,680 (0.71 s)	22,030 (0.27 s)	18,861 (0.91 s)	21,725 (0.26 s)
lcet10.txt (29)	89,345 (0.45 s)	131,473 (0.46 s)	88,378 (0.52 s)	130,472 (0.55 s)
plrabn12.txt (5)	125,047 (0.55 s)	162,111 (0.65 s)	124,161 (0.63 s)	161,022 (0.73 s)
ptt5 (216)	28,125 (0.33 s)	39,368 (0.13 s)	27,511 (0.36 s)	39,273 (0.25 s)
sum (40)	9,822 (0.07 s)	12,571 (0.08 s)	9,726 (0.10 s)	12,508 (0.10 s)
xargs.1 (0)	1,464 (0.01 s)	—	1,463 (0.01 s)	—
dickens (4)	≈2,224,500 (10.09 s)	≈2,896,900 (10.91 s)	≈2,181,800 (11.79 s)	≈2,831,300 (12.86 s)
mozilla (3)	≈13,410,600 (55.33 s)	≈14,461,500 (58.10 s)	≈13,166,400 (65.91 s)	≈14,225,500 (66.84 s)
mr (2)	≈2,230,500 (7.82 s)	≈2,349,100 (5.25 s)	≈2,206,500 (8.57 s)	≈2,322,200 (6.08 s)
nci (3)	≈1,428,400 (21.51 s)	≈1,609,100 (13.59 s)	≈1,382,400 (24.13 s)	≈1,546,200 (16.10 s)
ooffice (16)	≈2,288,200 (8.28 s)	≈2,582,200 (7.91 s)	≈2,257,100 (9.45 s)	≈2,550,100 (9.06 s)
osdb (2)	≈2,376,600 (11.76 s)	≈2,613,900 (12.68 s)	≈2,339,500 (12.93 s)	≈2,559,200 (14.62 s)
reymont (5)	≈1,093,100 (6.48 s)	≈1,422,500 (7.59 s)	≈1,063,800 (6.69 s)	≈1,383,100 (8.27 s)
samba (4)	≈3,405,600 (20.17 s)	≈4,080,300 (21.65 s)	≈3,362,100 (25.23 s)	≈4,031,100 (22.24 s)
sao (28)	≈3,921,700 (10.33 s)	≈4,509,900 (9.97 s)	≈3,898,200 (12.84 s)	≈4,483,900 (11.47 s)
webster (6)	≈6,442,800 (40.93 s)	≈8,608,900 (46.32 s)	≈6,271,700 (45.19 s)	≈8,399,600 (54.91 s)
x-ray (2)	≈3,751,400 (11.44 s)	≈4,190,400 (7.36 s)	≈3,734,200 (13.73 s)	≈4,159,000 (10.81 s)
xml (28)	≈352,600 (4.58 s)	≈530,200 (5.57 s)	≈351,400 (5.32 s)	≈524,600 (5.76 s)

The activity map never shrinks HCM's output (5–35% larger), because HCM's own record contexts already model the same structure more precisely. On structured files, however, it **cuts decode time sharply**: kennedy.xls 0.71 → 0.27 s, ptt5 0.33 → 0.13 s, x-ray 11.4 → 7.4 s, nci 21.5 → 13.6 s. The size loss is too large for any award under the standard. A future HCM mode could pass unchanged runs through the map while modelling only the literals.

Chapter 9 — HCM Architecture

> **Executive summary.** The Harrington Compression Method (HCM) is a context-mixing compressor: for every bit of a file, it lets a set of specialised models vote on whether the next bit is 0 or 1, learns online which models to trust, and codes the bit with an arithmetic coder at almost exactly the cost of that prediction. Its design closely follows Matt Mahoney's lpaq1 and borrows from paq8, ZPAQ and the Witten–Neal–Cleary arithmetic coder (full credits in Section 1.8). HCM v3 is a compact, deterministic C++ implementation that runs about four times faster than its own predecessor and six times faster than ZPAQ's strongest mode, while matching or beating it on most text. It is the base for every later HCM result.

9.1 Lineage

Version	Date	Form	Key step
Route 2 ("restore point")	24 Oct 2026	Python	BWT > move-to-front > zero-run tokens > adaptive arithmetic coder with previous-token context
Context-balance coder	24 Oct 2026	Python	First context-mixing design: hashed bit-level contexts, a long-match predictor, a logistic mixer and a secondary estimate
v2	24–25 Oct 2026	C++ port	Byte-identical to the Python coder
v3	25 Oct 2026	C++ (`hcm.cpp`)	Integer-only, cache-aligned redesign described in this chapter
v3.1–v3.3	25 Oct 2026	C++	Chapter 10

Early route results (200 KB English text; 140 KB access-control gate log)

Route	Text, 1,000,000 bits down to (normalised)	Gate log, 1,000,000 bits down to (normalised)
Route 2 (BWT > MTF > zero runs > arithmetic)	176,890	86,293
Context-balance coder	148,625	60,336
xz	185,240	104,914
bzip2	168,850	79,464
ZPAQ -m5	140,505	60,493

Context-balance coder ablation (bytes added when a component is removed or changed; reference 29,725 for text and 8,447 for the gate log)

Change	Text	Gate log
Remove long-match predictor	+2,688	+315
Freeze mixer learning	+1,969	+1,413
Remove word context	+808	+20
Hash tables 2^{18}	+614	+47
Remove refinement stage	+403	+210
Hash tables 2^{20} (a quarter of the memory)	+145	+5
Refinement blend 50/50	+56	+41
Verified matches	−36	+5
Refinement blend 90/10	−7	−28
Verified matches + 90/10 blend	−41 (29,684)	−19 (8,428)

The ablation established the priorities carried into v3: the long-match model and mixer learning dominate, the word context matters on text, and table size can be cut by three quarters at little cost.

9.2 v3 model

Contexts. Each bit is predicted from:

- hashed contexts of the previous 1, 2, 3, 4, 6 and 8 bytes;

- the current word and the previous word pair;
- a direct order-0 context.

Hashes are cumulative across orders.

Bit-history state. Each context slot holds a 16-bit probability and a count. The probability is updated at rate 32768 / (n + 1.6), with the count capped at 60 for order 1 and at 255 otherwise. This is an adaptive-rate estimator that learns quickly at first and stabilises with evidence.

Memory layout. The 15 nodes of a nibble's binary tree share one 64-byte block per context. Slot 0 holds a 16-bit check value, which detects hash collisions. Lookup is two-way: of a pair of candidate blocks, the matching one is used; failing a match, the one with the lower first-node count is recycled. Table size per context is min(16 in fast mode or 17 in max mode, $\lfloor \log_2 n \rfloor + 2$, at least 10) bits, so small files use small tables.

Long-match model. A 6-byte hash locates the most recent occurrence of the current context. A match is used only when at least 6 bytes are verified. The predicted bit is weighted by a confidence learned per match length. Its table size is min(20, $\lfloor \log_2 n \rfloor + 3$) bits.

Mixer. An integer logistic mixer combines the stretched predictions (stretch × 256). Weights are 16.16 fixed-point, initialised to 19661, and updated by (x · err + 1024) >> 11. The weight set is selected by partial byte × match-length bucket × whether the previous byte is a letter. Max mode adds a second mixer, selected by partial byte × the classes of the last two bytes (letter, digit, space, control/high, other); the two stretched outputs are averaged.

Adaptive probability map (APM). A secondary estimate indexed by partial byte and previous byte, with 33 interpolation bins updated at rate >> 13. The final probability is (p_mixer · 26 + p_APM · 230) >> 8.

Coder. A 32-bit binary arithmetic coder. The decoder uses no division.

Determinism. The entire coding loop uses integer arithmetic, so output is bit-identical across compilers, operating systems and CPUs. This fixed the known weakness of the floating-point Python prototype.

Container. `"HCM3"` + mode byte + 32-bit length + CRC-32 + coded bits. The decoder reads v2 files and reports damaged input.

9.3 Resources

Mode	Peak memory	Encode / decode speed relative to v2	Decode speed relative to ZPAQ -m5
Fast	42 MB	3.96× / 4.10× faster	about 6.3–6.5× faster
Max	68 MB	about 3.2× faster	—
v2 (for reference)	121 MB	1×	—

Compression and decompression take the same time, as in every symmetric context-mixing design.

9.4 v3 results — Canterbury

Summary. HCM v3 fast is 0.6% smaller than v2 on the whole corpus (416,994 → 414,602 bytes) and decodes about 4× faster (Section 9.5). It beats ZPAQ -m5 on the 8 text and code files (304,688 against 306,082 bytes) but is 14.6% larger on the whole corpus, because ZPAQ is far stronger on the three binaries.

Configuration	All 11 files (bytes)	8 text and code files (bytes)	alice29.txt (bytes)
HCM v2	416,994	304,800	37,664
HCM v3 fast	**414,602**	**304,688**	**37,342**
HCM v3 max	412,328	301,571	37,001
ZPAQ -m5	361,793	306,082	37,600

v3 fast beat ZPAQ -m5, xz and bzip2 on 6 of the 11 files — alice29.txt, asyoulik.txt, cp.html, fields.c, grammar.lsp and xargs.1 — and beat ZPAQ on the text total. ZPAQ's overall lead came almost entirely from the binary files (kennedy.xls, ptt5, sum), which motivated HCM v3.1 and HCM-Bin (Chapters 10 and 12).

Measurement note: an earlier comparison that appeared to give v2 six wins was invalidated, because the ZPAQ archive stored a long folder path (about 100 bytes). All ZPAQ figures here are from fair runs with short stored names.

9.5 Design observations

1. **Integer determinism is not optional.** A compressor whose output depends on floating-point rounding cannot be trusted across machines; v3's integer loop removed that risk at no cost in size.
2. **Locality beats raw table size.** Packing a nibble's nodes into one cache line, with a check value, gave most of the 4× speed-up over v2 and cut memory by two thirds.
3. **Small-file scaling.** Sizing tables from the input length avoids the fixed start-up cost that penalises large-table context mixers on small files. It is one reason HCM wins the smallest Canterbury files.

Chapter 10 — HCM Improvements: v3.1 to v3.3

> **Executive summary.** Three targeted changes took HCM from 414,602 to 353,494 bytes on Canterbury (2,810,784 bytes), a 14.7% reduction, without slowing decoding. One fixed the weakness on binary files; two sharpened the prediction of English text. With these changes, HCM's fast mode produces the smallest output of all tested rivals on 9 of the 11 Canterbury files.

10.1 Summary of versions

Summary. From v3 to v3.3 the Canterbury total fell 14.7% in fast mode (414,602 → 353,494 bytes) and 15.0% in max mode (412,328 → 350,434 bytes), with decode speed unchanged. Most of the gain (12.9%) came from v3.1's record swap on binary files; v3.2 and v3.3 cut the 8 text files by 2.6% (304,688 → 296,867 bytes).

Version	Change	Canterbury bytes (fast / max)	Text files (8), bytes	Speed relative to v3
v3	Baseline (Chapter 9)	414,602 / 412,328	304,688	1.00
v3.1	Binary-only record swap	361,093 / 357,467	unchanged	1.00
v3.2	+ mixer selected by word position	358,884 / 356,284	302,243	1.00
v3.3	+ line-position context, retuned refinement and match rate	**353,494 / 350,434**	**296,867**	1.00

10.2 v3.1 — binary-only record swap

Detection.

- A file is flagged as binary when more than 1 byte in 10 is a control byte (0–8, 14–31, 127). The flag is stored in the header's mode byte.
- A record length is detected when a byte value repeats at equal spacing three or more times. kennedy.xls yields 13; ptt5 yields 216, the fax scan line in bytes.

Swap. In binary files only, once a record length is known, three context slots change role. The two word-context slots and the order-8 slot become record contexts:

1. row above + above-right byte;
2. rows above 1–3;
3. row above + last byte.

The number of contexts and the speed are unchanged, and text output is bit-identical to v3.

Effect. kennedy.xls fell from 53,899 to 18,674 bytes in fast mode, and ptt5 to 27,973 bytes, beating ZPAQ -m5 at 28,563. The Canterbury total fell by 13%.

10.3 v3.2 — word-position mixer selection

v3 selects mixer weights partly by "is the previous byte a letter". v3.2 replaces that bit with a finer description of position in a word:

- **Outside a word:** the class of the last byte — space, digit, control/high, or other.
- **Inside a word:** letters so far — 1, 2, 3–4, or 5 and more.

The number of weight sets rises from 2,048 to 8,192, which adds 0.3 MB of RAM at the same speed. Text improved from 304,688 to 302,243 bytes.

Rejected variants (all worse): a word context including punctuation; several previous-word variants; alternative order sets; a minimum match length of 7; APM selection by match state or word state.

10.4 v3.3 — line-position context

In text files the order-6 slot becomes a **line-position context**: a hash of the column in the current line, letters so far in the word, and the class of the last byte. Two constants were retuned at the same time:

- refinement weight (APM share) 230 → 170;
- match-model learning rate halved (update shift of 5 bits changed to 6 bits).

Text improved to 296,867 bytes. Column position alone carries most of the gain; adding it to *mixer selection* instead made results worse.

10.5 Strict No-Pain check

Against v3, three small files were slightly larger under v3.3: fields.c, grammar.lsp and xargs.1. With per-file fallback at compression time — the decoder supports both v3 and v3.3 streams — no file is ever larger than v3. Median decode time over all 11 files was 1.85–1.87 s against 1.88 s for v3, so there is no measurable loss. Instruction counts rose by 0.2–0.4% on text and by up to 2% on binaries.

Award: *No-Pain / General / Canterbury* against HCM v3.

10.6 Per-file results — HCM v3.3

Summary. HCM v3.3 fast is the smallest of the tested Tier 1–4 rivals on 9 of 11 Canterbury files. It loses kennedy.xls to ZPAQ -m5 (18,680 against 16,822 bytes, 11% larger) and sum to xz -9e (9,822 against 9,500 bytes, 3% larger). Decode times range from 10 ms to 0.75 s per file.

Compressed bytes, with median decode time.

File (bytes)	v3.3 fast	Decode	v3.3 max	Decode	Best rival (bytes)
alice29.txt (152,089)	36,636	200 ms	36,267	271 ms	ZPAQ -m5 37,600 vs HCM fast 36,636
asyoulik.txt (125,179)	34,348	181 ms	34,138	256 ms	HCM wins
cp.html (24,603)	6,437	55 ms	6,388	69 ms	HCM wins
fields.c (11,150)	2,522	26 ms	2,493	35 ms	HCM wins
grammar.lsp (3,721)	1,048	10 ms	1,038	12 ms	HCM wins
kennedy.xls (1,029,744)	18,680	747 ms	18,861	954 ms	ZPAQ -m5 16,822 bytes wins
lcet10.txt (426,754)	89,345	467 ms	88,378	579 ms	ZPAQ 90,021 vs HCM 89,345 bytes — HCM wins
plrabn12.txt (481,861)	125,047	625 ms	124,161	598 ms	ZPAQ 127,578 vs HCM 125,047 bytes — HCM wins
ptt5 (513,216)	28,125	406 ms	27,511	388 ms	HCM wins (ZPAQ 28,563 bytes)
sum (38,240)	9,822	69 ms	9,726	98 ms	xz -9e 9,500 bytes wins
xargs.1 (4,227)	1,484	16 ms	1,473	12 ms	HCM wins
Total (2,810,784)	**353,494**		**350,434**		HCM wins 9 of 11

10.7 Dictionary pre-step (experimental, not part of HCM)

cmix's English word-replacement dictionary (412 KB) was applied to the 8 text files before HCM.

Summary. The dictionary cut HCM v3.3 fast's text output by 5.4% (296,867 → 280,696 bytes) and the Canterbury total by 4.6%, with decoding not slower. paq8px and cmix remain 9–12% smaller than the best HCM result with the dictionary (hcm-trit v2 max, 332,468 bytes).

Configuration	Text bytes (8 files)	All 11 files (bytes)
HCM v3.3 fast	296,867	353,494
HCM v3.3 fast + dictionary	280,696	337,322
hcm-trit v2 max + dictionary	—	332,468
paq8px v217 -8	—	*302,019*
cmix v21 (published)	—	*292,617*

alice29.txt fell from 36,636 to 34,648 bytes, and lcet10.txt from 89,345 to 82,898 bytes. Decoding was the same speed or faster, because the reverse step for all 8 text files takes about 0.15 s and leaves fewer bytes to model. **Licensing and scope:** the dictionary and its transform code are GPL-3 and English-only. They are excluded from HCM's MIT release; an MIT-licensed HCM would need its own dictionary.

10.8 v3.4 — match recovery, numeric contexts and raw store

HCM v3.4 (29 October 2026) answered points raised in an independent review. A match of 24 bytes or more now survives one wrong byte instead of being dropped, binary files get a numeric context that predicts 16-bit samples, and incompressible files are stored unchanged with a 13-byte header. Every older .hcm file decodes exactly as before. On the Canterbury corpus HCM v3.4 max gave 350,326 bytes, decoding in 3.4 s, against ZPAQ -m5's 350,542 bytes in 11.5 s on the same machine.

10.9 v3.5 and v3.6 — multi-core: parallel blocks and Harrington Sets

HCM decodes at the same speed it compresses, so a large file can take tens of seconds to open. Two versions added multi-core processing.

v3.5, parallel blocks. A large file is cut into equal blocks (16 MB by default), and each block is compressed and decoded on its own processor core. Each block starts with an empty model, so the file grows slightly. A second v3.5 feature, switching between text and binary contexts at section boundaries inside one model, gained almost nothing (samba unchanged, mozilla 3,263 bytes smaller) and is reported here as a negative result.

v3.6, Harrington Sets. This is the author's own method, built on his exclusion idea. The file is read in 64 KB windows. Windows that are random or already compressed (byte entropy above 7.99 bits per byte) are excluded to a raw set that is stored and never modelled. The remaining windows are grouped into up to three sets by their byte statistics (k-means on the share of control bytes, letters, digits, spaces, high bytes and zeros, plus entropy). Runs shorter than 512 KB join their neighbour, and sets under 1 MB fold into the largest set. Each set's windows are joined in file order and compressed with their own model; large sets are split into blocks; every block runs on its own core. A run-length map of the set labels, five bytes per run, restores every window to its place.

Originality. Harrington Sets is an original work by the author with loose ties to earlier ideas. Other approaches either cut a file into equal chunks and process each on its own (pigz, pbzip2, xz -T, zstd -T, ZPAQ blocks, pcompress), group whole files by name (FreeArc, 7-Zip, RAR), split data into streams from a format description written in advance (OpenZL, Parquet and ORC schemas), or cluster one specific kind of data (Logzip, for text logs). Harrington Sets differs in how it operates: it parses on the spot, measuring each window of an unknown file, regroups like windows from anywhere in the file into shared streams, excludes incompressible windows through a layered map that keeps exact positions, and decodes the streams in parallel inside a context-mixing compressor. To the author's knowledge no existing tool does this. Its purpose is to let slow compressors that have only ever used one core use them all; tools that already decode in milliseconds are left alone, because they gain little.

Results, fast mode, on a 2-core machine, one run, wall clock; every file decoded exactly. Sizes are 1,000,000 bits down to X.

File (bytes)	Version	1,000,000 bits down to	Compress	Decode
samba (21,606,400)	v3.4 single stream	157,393	62.9 s	16.8 s
	v3.5 blocks (16 MB)	159,390	59.2 s	13.9 s
	v3.6 sets	**157,643**	**42.5 s**	**9.7 s**
mozilla (51,220,480)	v3.4 single stream	260,739	180.8 s	47.0 s
	v3.5 blocks (16 MB)	261,866	130.2 s	32.7 s
	v3.6 sets	261,999	135.9 s	35.4 s
dickens (10,192,446)	v3.4 / v3.5 2 MB / v3.6 2 MB	218,246 / 225,691 / 227,565	33.1 / 20.4 / 19.9 s	8.3 / 5.6 / 5.5 s
x-ray (8,474,240)	v3.4 / v3.5 2 MB / v3.6 2 MB	432,649 / 434,707 / 437,870	34.2 / 17.8 / 17.5 s	8.6 / 5.0 / 4.5 s
reymont (6,627,202)	v3.4 / v3.5 2 MB / v3.6 2 MB	164,946 / 169,264 / 170,469	18.7 / 12.7 / 10.2 s	5.1 / 3.3 / 2.8 s

On samba, which mixes source text and binaries, Harrington sets decoded 1.73 times faster than one stream for 0.16% more size, and beat plain blocks on both size and speed. On mozilla, which is mostly one kind of data, sets and plain blocks were about equal. On the three medium files (picked at random) both methods decoded 1.5 to 1.9 times faster for 0.5% to 4.3% more size; smaller blocks cost more size because each block starts empty. Small files that fit in one block gain nothing. Only two cores were available for these tests; with four to eight cores the speed-up grows with the number of blocks decoded at once. v3.5 and v3.6 are speed improvements, not size improvements.

The same method on other slow tools. The two slowest tools in this book after HCM were tested the same way on the three medium files, with 2 MB blocks. lpaq1-h (the lpaq1 mutant, Chapter 13) was given a parallel-block wrapper. zpaq-hi (the ZPAQ modification) needs no change to its format at all: ZPAQ already decodes separate blocks on separate cores, so zpaq-hi only writes smaller blocks, and the stock zpaq 7.15 decoder read every archive with two threads.

File	Tool	1,000,000 bits down to (one / multi-core)	Compress (one / multi)	Decode (one / multi)
reymont	lpaq1-h	157,486 / 163,049	38.9 / 28.2 s	6.7 / 4.1 s
x-ray	lpaq1-h	438,616 / 444,765	69.0 / 36.6 s	12.5 / 6.5 s

File	Tool	1,000,000 bits down to (one / multi-core)	Compress (one / multi)	Decode (one / multi)
dickens	lpaq1-h	215,212 / 227,103	69.7 / 41.6 s	12.4 / 7.0 s
reymont	zpaq-hi	142,773 / 151,508	81.6 / 43.2 s	41.5 / 22.9 s
x-ray	zpaq-hi	432,631 / 438,089	58.9 / 23.7 s	44.9 / 22.0 s
dickens	zpaq-hi	205,223 / 222,108	62.0 / 30.4 s	51.1 / 25.3 s

Both tools decoded 1.6 to 2.0 times faster on two cores. The size cost depends on the data: under 1.5% on the x-ray image, 3.5% to 8% on the two text files, because text models gain most from a long history and every 2 MB block starts empty. Larger blocks would lose less on text. Every result decoded exactly.

Stock ZPAQ on the same machine. The zpaq-hi speed-up turned out not to be its own. Stock ZPAQ given the same 2 MB blocks decoded just as fast, because splitting into blocks is ZPAQ's own feature. Same machine, same files, two threads (1,000,000 bits down to / decode):

File	Stock, one block	zpaq-hi, one block	Stock, 2 MB blocks	zpaq-hi, 2 MB blocks
reymont	144,330 / 30.2 s	142,773 / 41.5 s	153,200 / 16.7 s	151,508 / 22.9 s
x-ray	433,042 / 41.1 s	432,631 / 44.9 s	438,503 / 18.1 s	438,089 / 22.0 s
dickens	205,520 / 50.2 s	205,223 / 51.1 s	222,373 / 25.0 s	222,108 / 25.3 s

zpaq-hi stayed 0.1% to 1.1% smaller than stock at the same block size, but decoded slower in all six comparisons (from about 1% on dickens to 37% on reymont, where its larger model was chosen) and compressed about twice as slowly. It is a size gain in Tier 5, not a No-Pain win, and not a speed gain. For HCM and lpaq1-h, which had no multi-core option, the speed-up is genuinely new to those tools.

Chapter 11 — hcm-trit and the Ternary Pointer

> **Executive summary.** The ternary pointer is the author's Combary idea applied inside a context-mixing compressor. Before coding a byte, HCM asks one cheap question: is it the byte the model already expects? A "yes" skips the byte almost for free; a "no" removes that guess from the remaining possibilities. It improved every Canterbury file and produced the best HCM result to date: 347,696 bytes on the 2,810,784-byte Canterbury corpus in max mode, 0.8% smaller than HCM v3.3 max.

11.1 hcm-trit v1 — ternary digit coding

hcm-trit v1 codes each byte as six base-3 digits of (byte XOR flip). Each trit is coded as two binary decisions:

1. Is it the *split-off value* S?
2. If not, which of the other two is it?

S is chosen per digit position by a *swap pattern*. Impossible digit values are skipped, since $3^6 = 729$ exceeds 256. Two digits share one 64-byte block per context. The encoder tries five candidates per file and keeps the smallest:

Pattern	Flip
020000	255
020211	255
022222	255
012111	255
000000	0

The container is `"HCMT"` + version + pattern (2 bytes, base 3) + flip + length + CRC-32.

Summary. Coding bytes as ternary digits saved 3.0% on ptt5 and 1.7% on kennedy.xls, but lost 0.4–1% on text, so the Canterbury total rose by 301 bytes (+0.07%). Decoding ran at v3 speed; encoding was about five times slower.

File (bytes)	HCM v3 fast	hcm-trit v1	Difference
ptt5	45,083	43,748	−1,335
kennedy.xls	53,899	53,007	−892
sum	10,930	10,910	−20
alice29.txt	37,342	37,564	+222
Other text files	—	0.4–1% worse	—
All 11 files	414,602	414,903	+301

Ternary digit coding helps binary files and costs a little on text. Compression is about five times slower, because five patterns are tried; decompression runs at v3 speed. Later versions with the record swap: hcm-trit v1.1 361,711 bytes and v1.2 359,294 bytes (Canterbury total).

11.2 The ternary pointer — design

The pointer adds one decision before each byte for which a candidate exists. The candidate is:

- **with a match:** the byte predicted by the long-match model, at any match length;
- **without a match:** the last byte that followed the same 4-byte context, with its repeat count kept.

Then:

1. **Flag probability.** The model's own probability for the candidate is computed as the product of its 8 bit predictions along the candidate's path, *without learning*. It is refined by a small four-input mixer whose inputs are:
    - that probability;
    - an escape table indexed by match length and the last two pointer outcomes;
    - a table keyed by order-2 context and candidate;

    - a bias.

  The mixer weights are kept per escape-table cell.
2. **On “yes”** (the “2”), the byte’s bits are learned by the model but not coded.
3. **On “no”**, the byte is coded with the candidate **excluded**: the candidate’s share is removed from each bit’s probability, and the last bit is forced when only one value remains.
4. **Reuse.** The 8 predictions computed for the candidate are reused while the actual byte follows the same path, so the pointer costs little extra work.

Because the model learns exactly the same bits as without the pointer, the pointer never degrades the underlying model. It can only change how cheaply bytes are coded.

11.3 Development record (Canterbury total, bytes)

Summary. The working pointer cut HCM v3 fast’s Canterbury total by 1.5% (414,602 → 408,427 bytes). A speed-recovered version kept a 1.1% gain at about 4% slower decoding. Without exclusion the pointer made output 8.3% larger, so exclusion is essential.

Step	Result	Speed relative to v3 fast
HCM v3 fast (reference)	414,602	1.00
Simple pointer: fixed escape table, matches of 16 or more only	418,135 (worse)	—
Pointer with model-informed flag and 3-input mixer	410,096	15–20% slower
+ context candidate when no match, + 4th mixer input, prediction reuse	408,427	about 29% slower
Same, *without* exclusion on “no”	449,073 (much worse)	—
Speed-recovered: orders 1, 2, 3, 4, 6 (order 8 dropped), APM-row prefetch	409,888	about 4% slower
Max mode + pointer	406,695	—
Lazy second-nibble trick	427,846 (rejected)	—

Key findings. Exclusion is essential: without it the pointer is 10% worse than no pointer at all. A model-informed flag probability is essential too: a fixed escape table loses.

11.4 hcm-trit v2 across the HCM versions

Summary. On every base version, the pointer with six orders plus max mode is the smallest result: 1.4–1.7% smaller than the same version’s fast mode without the pointer. On v3.3 it reaches 348,135 bytes (Canterbury total).

Base	Without pointer (bytes)	With pointer, 5 orders	With pointer, 6 orders	Max + pointer
v3.1	361,093	360,075	358,229	354,878
v3.2	358,884	358,268	356,489	353,881
v3.3	353,494	353,105	351,179	348,135

11.5 Rebuilt as *hcmx* on the published v3.3 code

The experimental sources were lost with an earlier workspace, so the pointer and HCM-Bin were re-implemented on the published v3.3 code as one program, *hcmx*. Per file it tries v3, v3.3, trit2 and, for binaries, Bin1, and keeps the smallest. New mode bits mark the choice: PTR = 64, XREC = 128.

Summary. In the rebuilt hcmx, the pointer (trit2) makes 9 of 11 Canterbury files smaller and none larger: the total falls 0.6% in both modes (fast 353,494 → 351,302 bytes; max 350,434 → 348,244). The cost is decode time: the total rises 11% in fast mode (2.80 → 3.11 s) and 13% in max mode (3.27 → 3.70 s).

Compressed bytes, with median decode time:

File (bytes)	v3.3 fast	trit2 fast	v3.3 max	trit2 max
alice29.txt (152,089)	36,636 (200 ms)	**36,571** (236 ms)	36,267 (271 ms)	**36,216** (267 ms)
asyoulik.txt (125,179)	34,348 (181 ms)	**34,279** (165 ms)	34,138 (256 ms)	**34,078** (310 ms)
cp.html (24,603)	6,437 (55 ms)	**6,430** (61 ms)	6,388 (69 ms)	**6,383** (82 ms)
fields.c (11,150)	2,522 (26 ms)	**2,507** (29 ms)	2,493 (35 ms)	**2,482** (41 ms)

File (bytes)	v3.3 fast	trit2 fast	v3.3 max	trit2 max
grammar.lsp (3,721)	1,048 (10 ms)	**1,047** (10 ms)	1,038 (12 ms)	1,038 (14 ms)
kennedy.xls (1,029,744)	18,680 (747 ms)	**17,308** (838 ms)	18,861 (954 ms)	**17,345** (912 ms)
lcet10.txt (426,754)	89,345 (467 ms)	**89,014** (577 ms)	88,378 (579 ms)	**88,082** (683 ms)
plrabn12.txt (481,861)	125,047 (625 ms)	**124,867** (661 ms)	124,161 (598 ms)	**124,011** (767 ms)
ptt5 (513,216)	28,125 (406 ms)	**28,054** (425 ms)	27,511 (388 ms)	**27,480** (483 ms)
sum (38,240)	9,822 (69 ms)	**9,741** (90 ms)	9,726 (98 ms)	**9,656** (132 ms)
xargs.1 (4,227)	1,484 (16 ms)	1,484 (15 ms)	1,473 (12 ms)	1,473 (13 ms)
Total (2,810,784)	353,494 (2.80 s)	**351,302** (3.11 s)	350,434 (3.27 s)	**348,244** (3.70 s)

With HCM-Bin also available (Chapter 12), *hcmx* best-of reaches **350,874 bytes (fast)** and **347,696 bytes (max)**, 0.7% and 0.8% smaller than v3.3. That beats the lost original's ≈348,135 bytes. Without kennedy.xls the totals are fast 334,814 → 333,994 and max 331,573 → 330,899 bytes (−0.2%). Every file is equal to or better than v3.3. With trit2 alone, decode time rises 11–13%; the best-of selection, which uses HCM-Bin on kennedy.xls, rises 22% (fast 2.80 → 3.42 s; max 3.27 → 3.99 s).

Award: *BGS* does not apply; this is a size gain with a speed regression. Against v3.3 on Canterbury it is a **Baseline** improvement in size at an 11–22% decode cost.

11.6 Silesia (fast mode)

Summary. On Silesia (211,938,580 bytes), trit2 is 0.9% smaller than HCM v3.3 fast and the best-of 1.4% smaller (42,930,727 → 42,329,754 bytes), but decoding is 19% and 37% slower. The best-of is 0.7% smaller than ZPAQ -m4, but 71% slower to decode.

Configuration	Silesia total (bytes)	Total decode
HCM v3.3	42,930,727	202 s
hcm-trit v2 (rebuilt)	42,532,237	241 s
Best-of with HCM-Bin v1	42,329,754	276 s
ZPAQ -m4	*42,627,841*	*161 s*
HCM v3.3 max	*42,215,133*	*217 s*

Every Silesia file is smaller than with v3.3. HCM-Bin is selected for mozilla, mr, ooffice, osdb, sao and x-ray. Against ZPAQ -m4, the best-of is smaller but slower; HCM v3.3 max remains the better Tier 4 point. Per-file Silesia results are in Appendix A.

11.7 The pointer on a BWT back end

To test generality, the pointer was placed in a bzip2-style pipeline: run-length coding, 900 KB blocks, BWT and MTF, with the Huffman stage replaced by arithmetic coding. MTF rank 0 serves as the pointed-to value, with a model-informed flag and exclusion.

Summary. In a bzip2-style pipeline, the pointer made the Canterbury total 7.3% smaller than real bzip2 -9 (542,709 → 503,338 bytes), 1.2% of it beyond a control experiment with an extra context. Decoding took about 0.25 s against 0.14 s for bzip2, so this is a size result, not a No-Pain one.

Configuration (Canterbury, all 11)	Total bytes
Real bzip2 -9	542,709
Same front end + plain arithmetic coding	522,162
+ extra context (control experiment)	509,516
+ ternary pointer	**503,338**

The pointer beat real bzip2 on 10 of 11 files; cp.html was slightly worse. Text alone improved by about 1.1% over bzip2. Most of kennedy.xls's gain came from arithmetic coding itself. Decoding all 11 files took about 0.25 s against 0.14 s for real bzip2.

11.8 Independent control: where the pointer's gain comes from (29 October 2026)

Summary. The pointer's Canterbury gain comes from its refinement mixer, not from the yes/no flag. This was found by Andrew Goodwin (University of Sydney) in an independent review and re-run here with the same result.

Method. hcmx was forced to one variant per file in fast mode on the Canterbury corpus. The control build changes one line: the flag is coded with the model's own probability of the candidate (the product of its eight bit predictions) instead of the refined four-input mixer's estimate. Everything else,

including exclusion, is unchanged. Every file round-tripped exactly.

Variant	Canterbury, fast (bytes)
v3.3, no pointer	353,494
Pointer as published	351,302
Pointer, flag coded with the model’s own probability	353,575

Reading. On its own, the flag adds 81 bytes, which is rounding. All of the 2,192-byte gain comes from the refinement mixer re-estimating the candidate’s probability, a form of secondary estimation. This matches the theory: with a calibrated model, “is it the candidate?” followed by the byte with the candidate excluded costs exactly as much as coding the byte directly. The claim in Section 7.4 that a “yes” saves a byte’s full cost is therefore withdrawn. The useful part is the extra estimator, which could also be added as a mixer input or an SSE stage without the flag; that has not yet been tested. The BWT version of the pointer (Section 11.7) has its own control and keeps a 1.2% gain beyond it.

Chapter 12 — HCM-Bin

> **Executive summary.** HCM-Bin is HCM specialised for structured binary data — spreadsheets, scanned images and fixed-record files. It adds models that look at the *row above* in the detected record layout, much as a person reading a table looks up a column. On the Canterbury spreadsheet it slightly beats ZPAQ's strongest mode while decoding about eight times faster.

12.1 Design

HCM-Bin v1 is the ternary-pointer engine (Chapter 11) with four additional contexts, active once a record length has been detected:

Context	Content
R1	Byte in the row above + byte above-right
R2	Byte in the row above + last byte
R3	Bytes in rows 1 and 2 above
R4	Bytes in rows 1, 2 and 3 above

Per file, the encoder also tries a variant in which the pointer's candidate is the byte in the row above, with refinement keyed by that byte. It helps ptt5 and hurts kennedy.xls, so it is chosen per file. Mixer weights selected by column position were tried and rejected.

Naming. By the author's rule, the binary route is a separately named version, so the main HCM route stays undisturbed. v3.1's *record swap* (Chapter 10) is the lighter, zero-cost subset of this idea built into HCM itself.

12.2 Original results (25 October 2026)

Summary. HCM-Bin v1 was 2.9% smaller than ZPAQ -m5 on the three Canterbury binaries (55,710 → 54,112 bytes) and decoded 8–13 times faster (kennedy.xls 6.35 → 0.79 s; ptt5 3.29 → 0.26 s). It was 47% smaller than xz -9e, although xz kept sum (9,500 against 9,726 bytes).

File (bytes)	HCM-Bin v1	ZPAQ -m5	xz -9e	HCM v3 fast
kennedy.xls	**16,763**	16,822	51,868	53,899
ptt5	**27,623**	28,563	39,860	45,083
sum	9,726	10,325	**9,500**	10,930
Three binaries (1,581,200 bytes)	**54,112**	55,710	101,228	109,912

Decode CPU time	HCM-Bin v1	ZPAQ -m5
kennedy.xls	0.79 s	6.35 s
ptt5	0.26 s	3.29 s
sum	0.05 s	—

HCM-Bin is smaller than ZPAQ -m5 on all three binaries and 8–13 times faster to decode. **Award:** *No-Pain (Baseline, vs ZPAQ -m5) / Local / Canterbury binaries*: smaller and faster on every file timed. The combined speed-up on kennedy.xls and ptt5 is 9.2×, which also meets BGS-2's speed threshold. xz retains sum, a SPARC executable, where its branch-converter filter helps.

12.3 Rebuilt version in *hcmx* (27 October 2026)

Summary. The rebuilt Bin1 is the smallest result on kennedy.xls: 16,797 bytes in max mode, 10.9% smaller than v3.3 (18,861) and 0.1% smaller than ZPAQ -m5 (16,822), but it decodes 26% slower than v3.3. On ptt5 and sum the pointer alone (trit2) is now smaller than Bin1.

Compressed bytes, with median decode time. "noptr" is HCM-Bin without the ternary pointer.

File (bytes)	Mode	v3.3	trit2	Bin1	Bin1 noptr
kennedy.xls (1,029,744)	fast	18,680 (747 ms)	17,308 (838 ms)	**16,880** (1,155 ms)	18,219 (1,266 ms)
kennedy.xls (1,029,744)	max	18,861 (954 ms)	17,345 (912 ms)	**16,797** (1,202 ms)	18,211 (1,270 ms)
ptt5 (513,216)	fast	28,125 (406 ms)	**28,054** (425 ms)	28,535 (451 ms)	28,649 (437 ms)

File (bytes)	Mode	v3.3	trit2	Bin1	Bin1 noptr
ptt5 (513,216)	max	27,511 (388 ms)	**27,480** (483 ms)	27,837 (529 ms)	27,884 (528 ms)
sum (38,240)	fast	9,822 (69 ms)	**9,741** (90 ms)	9,839 (97 ms)	9,914 (130 ms)
sum (38,240)	max	9,726 (98 ms)	**9,656** (132 ms)	9,721 (152 ms)	9,782 (146 ms)

kennedy.xls with Bin1 max is **16,797** bytes, against ZPAQ -m5's 16,822. The rebuilt Bin1 is best only on kennedy.xls. On ptt5 and sum, the pointer alone (trit2) now wins, because v3.3's built-in record swap already captures most of the record structure. The original's ptt5 advantage (27,623 bytes) came from the row-above candidate variant, which the rebuild has not yet reproduced.

The pointer contributes substantially inside HCM-Bin: removing it costs 1,339 bytes (+7.9%) on kennedy.xls in fast mode (16,880 → 18,219 bytes).

12.4 Silesia

In *hcmx* best-of mode, Bin1 is selected for six Silesia files: mozilla, mr, ooffice, osdb, sao and x-ray. Per-file sizes are given in Appendix A. The best-of total of 42,329,754 bytes (fast) is 0.7% smaller than ZPAQ -m4's 42,627,841 but slower to decode: 276 s against 161 s.

12.5 Cost view

Using the author's per-customer model (decode energy + transfer + storage, per million customers) on the three Canterbury binaries:

Compressor	Cost
HCM-Bin	**$11.39**
ZPAQ -m5	$18.44
xz -9e	$19.76

HCM-Bin is cheapest despite being slower than xz, because its smaller output saves more in transfer and storage than its extra CPU time costs. The model is described in Chapter 19.

Chapter 13 — HCM Against the Field

> **Executive summary.** On small files, HCM is the strongest compressor tested outside the very slow Tier 5 tools (paq8px, cmix): smaller than ZPAQ's top mode on Canterbury and several times faster. On large files the picture changes. ZPAQ's mid-level mode, -m4, is both smaller and faster than HCM's fast mode on the Silesia corpus, and on 100 MB of Wikipedia text the block-sorting compressor BSC is smaller than HCM and about seventeen times faster. HCM is therefore not yet the compression king of its speed tier. This chapter shows exactly where it wins and where it loses.

13.1 Canterbury

Summary. On Canterbury (2,810,784 bytes), HCM v3.3 fast is 2.3% smaller than ZPAQ -m5 (361,793 → 353,494 bytes) and decodes about six times faster. The hcmx best-of build is 3.9% smaller than ZPAQ -m5. HCM is 28% smaller than xz -9e and 35% smaller than bzip2 -9. The Tier 5 tools paq8px and cmix are 14–17% smaller than HCM max, but decode two to three orders of magnitude slower.

Compressor	Canterbury total (bytes)	vs ZPAQ -m5	Tier
xz -9e	493,080	+36.3%	3
bzip2 -9	542,709	+50.0%	3
ZPAQ -m5	361,793	—	5
HCM v3.3 fast	**353,494**	**−2.3%**	4
HCM v3.3 max	**350,434**	**−3.1%**	4
hcmx max (best-of v3.3 / trit2 / Bin1)	**347,696**	**−3.9%**	4
paq8px v217 -8	302,019	−16.5%	5
paq8px "lite" (author's reduced model set)	302,913	−16.3%	5
cmix v21 (published figure)	292,617	−19.1%	5

ZPAQ, paq8px and cmix totals are derived from normalised figures (within 2 bytes).

HCM beats ZPAQ -m5 while decoding about six times faster. paq8px and cmix remain far ahead in size, at decode times two to three orders of magnitude longer.

13.2 Silesia — Tier 4 race (27 October 2026)

All decodes are single-threaded, in the same session.

Summary. On Silesia (211,938,580 bytes), ZPAQ -m4 is 0.7% smaller than HCM fast (42,627,841 against 42,926,135 bytes) and decodes 14% faster (160.8 s against 186.0 s). HCM max is 1.0% smaller than ZPAQ -m4 but 35% slower to decode. HCM fast is 8.5% smaller than bzip3 and 7.8% smaller than bsc-max, but those decode 8× and 15× faster. LZ4 is 2.3× larger and decodes in 0.3 s.

Silesia Tier-4 race — compressed bytes / decode s

File (bytes)	lz4	hcm-fast	hcm-max	bzip3	bsc	bsc-max	zpaq-m4	zpaq-m5
xml (5,345,280)	1,176,903 / 0.01	352,590 / 4.12	351,427 / 4.88	405,877 / 0.38	399,224 / 0.09	393,180 / 0.12	357,451 / 3.60	326,947 / 11.49
dickens (10,192,446)	6,405,331 / 0.02	2,224,460 / 9.77	2,181,812 / 10.74	2,233,934 / 1.39	2,274,312 / 0.37	2,259,134 / 0.58	2,237,879 / 9.04	2,094,748 / 26.05
mozilla (51,220,480)	26,380,856 / 0.05	13,410,668 / 50.31	13,166,396 / 58.79	15,881,570 / 6.15	16,183,088 / 2.71	15,907,194 / 4.15	13,496,587 / 47.06	12,041,060 / 136.65
mr (9,970,564)	5,426,956 / 0.02	2,230,498 / 7.36	2,206,541 / 8.88	2,119,990 / 1.35	2,231,072 / 0.38	2,206,012 / 0.56	2,153,628 / 2.64	2,181,310 / 19.41
nci (33,553,445)	5,434,120 / 0.03	1,428,413 / 18.86	1,382,358 / 21.73	1,425,956 / 1.79	1,283,634 / 0.41	1,210,336 / 0.53	1,248,417 / 9.13	1,251,109 / 79.02
ooffice (6,152,192)	4,338,985 / 0.01	2,288,203 / 7.04	2,257,148 / 8.39	2,526,882 / 0.87	2,587,452 / 0.43	2,559,108 / 0.64	1,921,973 / 5.49	1,766,555 / 16.70
osdb (10,085,684)	5,246,205 / 0.02	2,376,630 / 10.58	2,339,520 / 11.94	2,261,854 / 1.38	2,276,416 / 0.45	2,224,780 / 0.68	2,432,037 / 8.30	2,204,743 / 28.76
reymont (6,627,202)	3,158,255 / 0.01	1,093,130 / 5.35	1,063,753 / 6.41	980,459 / 0.77	993,980 / 0.20	982,364 / 0.28	1,051,244 / 4.88	956,504 / 17.65
samba (21,606,400)	7,637,089 / 0.03	3,405,599 / 17.85	3,362,096 / 20.70	3,976,068 / 1.86	3,968,420 / 0.72	3,909,928 / 1.09	3,228,824 / 17.87	3,053,823 / 62.75
sao (7,251,944)	6,790,389 / 0.02	3,921,719 / 9.22	3,898,248 / 10.97	4,673,307 / 1.12	4,724,796 / 0.73	4,679,764 / 1.11	4,542,517 / 7.61	3,899,259 / 28.13

File (bytes)	lz4	hcm-fast	hcm-max	bzip3	bsc	bsc-max	zpaq-m4	zpaq-m5
webster (41,458,703)	19,962,222 / 0.05	6,442,803 / 36.98	6,271,673 / 43.18	6,786,980 / 4.87	6,698,958 / 1.21	6,448,532 / 1.83	6,184,237 / 36.67	5,666,837 / 104.02
x-ray (8,474,240)	8,387,340 / 0.02	3,751,422 / 8.57	3,734,161 / 10.32	3,657,083 / 1.16	3,806,844 / 0.62	3,756,584 / 0.95	3,773,047 / 8.48	3,669,704 / 23.58
Total (211,938,580)	**100,344,651 / 0.3**	**42,926,135 / 186.0**	**42,215,133 / 216.9**	**46,929,960 / 23.1**	**47,428,196 / 8.3**	**46,536,916 / 12.5**	**42,627,841 / 160.8**	**39,112,599 / 554.2**

Two figures for HCM v3.3 fast on Silesia appear in this book: 42,926,135 bytes here (the published `hcm.cpp`, race session) and 42,930,727 in Section 13.4 and Chapter 11 (the same model inside the rebuilt hcmx*, a later session). The 0.01% difference comes from the two builds' container handling; each comparison uses figures from one session only.*

Decode multiple M (decode time relative to LZ4, Chapter 2): HCM fast about 2,690 (Tier 4); HCM max about 3,120 (Tier 5); ZPAQ -m4 about 2,320 (Tier 4).

Verdict. Overall, ZPAQ -m4 beats HCM Fast in both compressed size and decompression speed, so HCM is not the Tier 4 compression king. On individual files the picture is mixed: HCM fast wins outright on dickens, mozilla, osdb, sao, x-ray and xml among the Tier-4 entries. It loses on ooffice, mr, nci, reymont, samba and webster. ooffice is the largest single loss: executable code, where ZPAQ applies an x86 transform. Section 13.4 shows that adding that transform to HCM closes much of the gap.

13.3 enwik8

Summary. On enwik8 (100,000,000 bytes), bsc-max is 1.0% smaller than HCM max (21,480,530 against 21,701,018 bytes) and decodes 19× faster (6.0 s against 115.2 s). ZPAQ -m4 is 4.4% smaller than HCM max and decodes 11% faster.

enwik8 — compressed bytes / decode s

File (bytes)	lz4	hcm-fast	hcm-max	bzip3	bsc	bsc-max	zpaq-m4	zpaq-m5
enwik8 (100,000,000)	57,004,012 / 0.09	22,187,152 / 99.36	21,701,018 / 115.18	22,677,530 / 13.90	22,463,250 / 4.11	21,480,530 / 5.98	20,740,505 / 102.26	19,625,015 / 281.80
Total (100,000,000)	**57,004,012 / 0.1**	**22,187,152 / 99.4**	**21,701,018 / 115.2**	**22,677,530 / 13.9**	**22,463,250 / 4.1**	**21,480,530 / 6.0**	**20,740,505 / 102.3**	**19,625,015 / 281.8**

Verdict. HCM is beaten on large English text. BSC at its maximum setting is smaller (21,480,530 against 21,701,018 bytes for HCM max) and decodes about 17–19 times faster. ZPAQ -m4 is smaller still at similar speed. HCM's text models were tuned on small files; their hash tables saturate on 100 MB inputs. This is the clearest target for future work (Chapter 23).

13.4 Extensions tested on Silesia (fast mode)

Summary. The hcmx extensions make every Silesia file smaller than HCM v3.3. The best-of build is 1.4% smaller (42,930,727 → 42,329,754 bytes) and now beats ZPAQ -m4 in size by 0.7%, but it decodes 37% slower than v3.3 and 71% slower than ZPAQ -m4. The in-format x86 filter adds a further 0.7%. Values marked ≈ are derived from normalised figures (within ±100 bytes).

Configuration	Silesia total (bytes)	Total decode
HCM v3.3 (hcmx build, separate run)	42,930,727	202 s
hcm-trit v2	42,532,237	241 s
hcmx best-of (v3.3 / trit2 / Bin1)	42,329,754	276 s
+ x86 E8/E9 filter inside the format (*hcmx-s*)	≈42,053,700	—
trit2 "lite" — pointer gated to matches of 12 or more, or context guesses repeated 3 or more times	≈42,613,200	11% faster than trit2
ZPAQ -m4	*42,627,841*	*161 s*

The x86 filter shrinks ooffice from 2,269,256 to 1,990,101 bytes and decodes it faster: *No-Pain / Local / ooffice*. Without hcmx-s, the x86 filter on HCM v3.3 alone gives ooffice (6,152,192 bytes) fast ≈2,288,200 → ≈2,010,600 and max ≈2,257,100 → ≈1,976,500 bytes, and Silesia totals of fast ≈42,648,600 and max ≈41,934,600 bytes. Every Silesia file is smaller under hcmx than under v3.3.

Rejected speed experiments: removing order 8; skipping models on long matches; a fixed-size loop rewrite, which used fewer instructions but ran slower on the test CPU.

13.5 Laboratory work on paq8px and cmix

Change	Effect
paq8px profile-guided build (same output)	No speed-up
paq8px “lite” model set — text: drop sparse-match, sparse-bit, sparse, XML, nest, chart, record, char-group and indirect models; binary: drop sparse-bit, XML, nest, chart, DMC, linear-prediction and exe models	About 2× faster (kennedy.xls 414 → 181 s; lcet10.txt 114 → 51 s) for a Canterbury total of 302,913 bytes against 302,019
cmix v21 memory reduction — PPMd 14,000 → 500 MB, paq8 level 11 → 8, FXCM tables ÷ 4, shared map 2 GB → 128 MB, history 100 → 16 MB, smaller match and indirect tables	Runs in 2.5–4 GB instead of 32 GB; no size change on fields.c (2,087 bytes)
cmix time profile	paq8 sub-model 43%, mixers 22%, LSTM 21%, FXCM 10%
cmix without paq8 and LSTM	About 4.5× faster on fields.c, for +75 bytes
cmix capped at about 5 GB, with dictionary	alice29.txt 31,085 bytes (published about 31,257) in 444 s

These are *Modified* results on third-party code; they are not part of HCM. The paq8px “lite” set is a practical speed option for users of that tool.

13.6 Position of HCM

Strength	Weakness
Best result on small files (Canterbury) among Tier 1–4 tools tested	Loses to ZPAQ -m4 on large mixed data (Silesia)
Wins structured binaries and images within Tier 4 (sao, x-ray, osdb)	Loses large text to BSC and ZPAQ (enwik8)
Deterministic, small memory (42–68 MB)	Symmetric: decode is as slow as encode

Chapter 14 — Harrington Header Methods

> **Executive summary.** Many compression formats carry a table at the front of the file telling the decoder how to undo a remapping. The author's header methods shrink such tables by exploiting a simple fact: as entries are listed, fewer possibilities remain, so later entries need fewer bits, and the last few cost almost nothing. The saving is guaranteed and lossless, but small — tens of bytes per table — so it matters most for small files and for formats that store a full one-to-one map.

14.1 The shrinking header

Consider a swap table that is a permutation of N values, transmitted in order. When the i-th entry is written, only $N - i + 1$ values remain unused, so it can be coded in $\lceil \log_2(N - i + 1) \rceil$ bits, using an index into the list of remaining values. For $N = 16$:

Entries	Values remaining	Bits each
1–8	16 → 9	4
9–12	8 → 5	3
13–14	4 → 3	2
15	2	1
16	1	0

That is 49 bits in total, against 64 for raw storage. The decoder rebuilds the remaining-values list as it reads, so no other side information is needed.

14.2 The omitted-suffix (permutation) form

If the last k entries are omitted, the decoder knows exactly which k values are missing but not their order. One of the $k!$ orders is then sent as an index of $\lceil \log_2 k! \rceil$ bits. Two omitted entries need one bit, whatever the width of the values. Savings against raw storage of a 16-bit table:

Entries omitted k	Raw bits	Order index $\lceil \log_2 k! \rceil$	Saved
2	32	1	31
3	48	3	45
4	64	5	59
8	128	16	112
16	256	45	211
all 65,536	1,048,576	≈ 954,037	≈ 94,539 (≈ 11.8 KB)

14.3 Comparison with the theoretical bound

The information-theoretic minimum for an arbitrary permutation of N items is $\log_2 N!$ bits, which is reached by the Lehmer code (factorial number system).

Representation	Last 2	Last 8	Last 16	Full 256-value table
Raw (8 bits per entry)	16 bits	64 bits	128 bits	2,048 bits (256 bytes)
Omitted-suffix factorial index	1	16	45	—
Harrington shrinking header	1	17	49	1,793 bits (225 bytes)
Full Lehmer index (bound)	1	16	45	1,684 bits (≈ 211 bytes)

The shrinking header comes within 6.5% of the bound on a full 256-value table and matches it for short suffixes. Unlike a full Lehmer index it needs no big-number arithmetic — each entry is an independent fixed-width field — which suits streaming and simple decoders. For the author's own formats this trade-off is recommended. Where every byte matters, the full factorial index saves about 14 more bytes per 256-value table.

14.4 Measured use: swap tables before standard compressors

Partial swap tables (top x most frequent bytes mapped to $0 \ldots x-1$, $x \in \{4, 8, 16, 32, 64, 255\}$), stored with the shrinking header, were tested on Canterbury with and without a following delta step, in front of zstd -22, LZ4 -12 and gzip -9.

Outcome	Files
Gain	gzip on ptt5 only: 52,377 → 52,276 bytes (x = 64, 64-byte header); decode time not checked
No gain beyond the header cost (0 to +5 bytes)	Every other file, with every compressor

The table itself is small; the swap simply does not change what dictionary and entropy coders exploit (Chapter 15).

14.5 Measured use: Huffman code-length headers

An order-0 canonical Huffman code was built for each Canterbury file. Its code-length header was stored three ways: raw (4 bits × 256 symbols), as a Harrington shrinking list of used symbols plus 4-bit lengths, and not at all, using a static English-frequency table trained on 11.5 MB of text outside the corpus.

File	Body (own table)	+ raw header	+ Harrington header	Static English table, no header	gzip -9, for scale
alice29.txt	701,502	702,526	**702,390**	829,990	433,432
asyoulik.txt	606,448	607,472	**607,264**	666,826	390,528
cp.html	129,588	**130,612**	130,620	144,314	63,784
fields.c	56,206	**57,230**	57,286	63,565	25,016
grammar.lsp	17,356	18,380	**18,268**	20,684	9,872
kennedy.xls	3,700,256	**3,701,280**	3,703,073	21,399,776	1,677,768
lcet10.txt	2,004,513	2,005,537	**2,005,521**	2,261,890	1,155,344
plrabn12.txt	2,204,678	2,205,702	**2,205,650**	2,577,687	1,554,112
ptt5	852,407	**853,431**	854,284	11,657,919	419,016
sum	205,159	**206,183**	207,972	551,279	102,144
xargs.1	20,813	21,837	**21,701**	23,622	13,984

Units: total bits for the file.

The Harrington header wins on text, where fewer than about 100 symbols are used — for example 888 bits against 1,024 on alice29.txt. It loses on binaries that use all 256 values, where a list of used symbols is pure overhead. The static English table never wins: training text that differs from the corpus costs far more than any header. And because order-0 Huffman is far behind gzip on every file, none of these choices changes the standings.

14.6 Applicability to other people's formats

The method applies only to tables that are **true permutations**, in which every value appears exactly once, so that missing values can be inferred.

Format	What its header stores	Applicable?
Deflate (gzip, zip, PNG)	Code lengths per symbol	No
bzip2	Used-symbol bitmap, selectors, code lengths	No
zstd, LZ4, xz, Brotli	Normalised counts or code lengths	No
PNG palette	Colour list	No
ZPAQ	Model description program (about 300 bytes at -m5); repeated component types	No — not a permutation
JPEG custom Huffman tables	Up to 162 distinct symbols in code order	Partly: about 42 bytes per AC table if the symbol set is known; requires a new format

Inside the author's own formats — Quad Exchange, the Harrington Exchange and any HCM variant that stores a remap — the method applies directly and should be the default.

14.7 Assessment

Method	Class	Verdict
Shrinking header	Original (an independent variant of mixed-radix permutation coding)	Guaranteed, small saving; simple; recommended for the author's formats

Method	Class	Verdict
Omitted-suffix factorial index	Known (Lehmer code applied to a suffix)	Equivalent to the shrinking header for short suffixes
Static-frequency header elimination	Known	Did not help on Canterbury

Chapter 15 — From Quad Exchange to the Harrington Exchange

Executive summary. The Harrington Exchange is a two-step reordering of data. A *swap* renames each chunk value so that the most frequent values get the most desirable bit patterns. A *juggle* then splits every chunk into a leading part and a trailing part stored as separate streams. On spreadsheet-style data the combination finds real structure: on kennedy.xls it cut zstd's output by 34%, LZ4's by 57% and gzip's by 33%. Record delta (Chapter 4), developed later, does better still on the same file. On most other structured files the Exchange does not help (Chapter 22). Its strongest results come on the data it was designed for: biased, near-random data packed at any bit width (Sections 15.7–15.11). There, renaming by frequency adds about 12% beyond splitting alone. On NIST's biased random bits it beats xz and bitshuffle. On packed 12-bit noise it comes within about 1% of the theoretical limit, and decodes about 3.8× faster than xz, though still 5–12× slower than plain zstd.

15.1 History and naming

The method began as **Quad Exchange**, the author's first implementation, which worked on four-bit chunks (nibbles). When it was generalised to support chunk widths other than four bits — whole bytes, or 16-bit chunks such as the 16HE-10 design below — it was renamed the **Harrington Exchange**. The two names mark stages in the method's evolution:

Stage	Name	Chunk width	Status
First implementation	Quad Exchange	4 bits (nibbles)	Working code; full permutation, lossless
Generalisation	Harrington Exchange	Any width (8-bit tested; 16-bit table designed)	8-bit tested here; 16-bit table specified

15.2 Definitions

Swap (presort). The values of fixed-width chunks are ranked by frequency. The most frequent source values are mapped to the most *desirable* target patterns, and the least frequent to the least desirable. What counts as desirable depends on the optimisation goal. One goal is to put zeros in the leading bits so that the juggle's first stream is highly skewed; another is to group values so that a following delta or entropy stage works better. The mapping is a permutation, so it is exactly invertible, and the decoder needs only the swap table.

Juggle. After the swap, each chunk is split at bit position j. The first j bits of every chunk go to stream 1, and the remaining bits go to stream 2, which is stored reversed.

Quad Exchange mapping, as implemented. The author's Quad Exchange code ranks the 16 nibble values by frequency. For its chosen criterion it uses these targets:

- the five most frequent nibbles → 1111, 1110, 1101, 1011, 0111;
- the five least frequent → 0001, 0010, 0100, 1000, 0000;
- the remaining six → the remaining six patterns, in decimal order.

Other target orders suit other criteria, and the 8-bit tests in this chapter order targets by the zeros in the leading j bits.

Header. Only Swap1–Swap10 are stored, 40 bits in total, because the last six mappings follow from the rule. For a full 8-bit permutation, the Harrington shrinking header (Chapter 14) stores 256 entries in 225 bytes instead of 256.

16HE-10. A 16-bit table designed to protect a contiguous run of ten zeros for a juggle at $j = 10$:

- the first 64 entries are `0000000000xxxxxx`;
- then shifted tiers `1-0000000000-xxxxx`, `01-0000000000-xxxx`, … `000001-0000000000`, 63 entries in total;
- then every remaining pattern, by increasing number of ones, ending with sixteen ones.

15.3 Implementation corrections

1. **Partial table made lossless.** An early JavaScript version mapped only the top X chunk values (default 64) and passed the others through unchanged, so an unmapped value could collide with a mapped target. All results here use a full permutation.
2. **16HE-10 specification.** The tier structure is consistent: 127 protected entries, no overlaps, 65,536 entries ending in sixteen ones. Three refinements remain:

    - the final popcount order needs a tie-break, since up to 12,870 patterns tie;
    - the verification code should cover every tier, not just tiers 1–3;
    - a juggle cut at bit 10 places each tier's leading 1 in stream 1, so the cut position and the protected run should be chosen together.
3. **Quad Exchange source.** A stray first line, `Revised`, stops the file from running. The permutation logic itself is correct and lossless.

15.4 Where it works: kennedy.xls

kennedy.xls is 1,029,744 bytes of 13-byte spreadsheet records. Across the compressors, the Exchange cut zstd's output by 34%, LZ4's by 57% and gzip's by 33%. The best variant differs by compressor: splitting the nibbles alone was best for zstd (47% smaller), and splitting at the 13-byte record stride was best for LZ4 (83%) and gzip (77%). Record delta (Chapter 4) beats every Exchange variant on all three compressors.

Table 15.1 — kennedy.xls, compressed bytes (stream-input measurement; see note)

Variant	zstd -22	LZ4 -12	gzip -9
Plain	64,815	324,136	209,721
8-bit Harrington Exchange + juggle	**42,858** (−33.9%)	**139,394** (−57.0%)	**140,230** (−33.1%)
Nibble juggle only (high/low nibble streams, low reversed; no swap)	**34,368** (−47.0%)	—	—
Stride split at 13 (byte shuffle; known technique)	—	54,827 (−83.1%)	49,139 (−76.6%)
Record delta at 13 (Chapter 4)	26,398 (−59.3%)	43,654 (−86.5%)	42,972 (−79.5%)

Note: these runs compressed from standard input, so zstd frames omit the stored content size, which is 3 bytes smaller than in Chapter 4 (64,815 vs 64,818).

Why it works. The fields of kennedy.xls change slowly from row to row. Separating each byte's slowly varying high bits from its noisier low bits makes each stream more repetitive than the interleaved original. The nibble split alone delivers most of the gain, which shows that the *split* matters more than the *swap*. Record delta captures the row-to-row similarity directly, so it wins.

15.5 Assessment

Mechanism	Class	Verdict
Swap + juggle (Harrington Exchange)	Original	Large gains on fixed-record data (kennedy.xls); on other Canterbury files it loses, mostly to the table cost (Chapter 22)
Nibble juggle alone	Original variant; related to known bit- and byte-plane splitting	Most of the kennedy.xls gain, with no table
Swap alone (Quad Exchange, 8-bit)	Original	No effect on dictionary or entropy coders by itself; useful as a step before a juggle or delta stage (Chapter 22)
Superseded on kennedy.xls by	Record delta and the Combary activity map (Chapters 4 and 8)	Smaller with every compressor tested
Swap + juggle at any bit width on biased or odd-width data (Sections 15.7–15.10)	Original method; Mutated pipeline	**Best-supported original result:** frequency renaming adds about 12% beyond splitting alone on biased sources, and beats xz and bitshuffle; on real images the split alone is better

15.6 Variants for future work

- **Partial swap with a small header:** map only the *x* most and *y* least frequent values. The first test, with *x* from 4 to 64, gained only on gzip with ptt5 (52,377 → 52,276 bytes).
- **Self-generating maps:** the decoder rebuilds the ranking from counts as it decodes, so no header is needed. This is close to move-to-front, a known method.
- **16-bit Harrington Exchange:** the 16HE-10 table has been specified but not yet benchmarked.
- **Automatic (S, J) selection:** shortlist widths from bit-position statistics before compressing, instead of a 276-point search.
- **Real packed odd-width data:** camera RAW10/12/14, packed telemetry and CCSDS packets, with documented packing.
- **A selector:** ranking for near-memoryless skewed sources, split-only for data with numeric locality.

15.7 The Exchange at any bit width (28 October 2026)

The method was designed for **random data with a bias**, and works at any chunk width, not only nibbles and bytes. Here the swap width *S* is the leading part of each chunk that forms stream 1, and the juggle width *J* is the whole chunk. The author's sweep order was swap 1 / juggle 2 up to juggle

24, then swap 2 / juggle 3 up to 24, and so on: 276 combinations with $S < J \le 24$.

Swap step, as tested.

- The *K* most frequent *J*-bit values (K = the smaller of 256, 2^J and the number of distinct values) are mapped onto the patterns with the fewest 1s in their leading *S* bits, then the fewest 1s overall.
- The map is completed to a permutation by sending displaced targets back to the freed sources, so it is exactly reversible.
- The decoder regenerates the targets, so the mapping data is only K·J bits. Container metadata (widths, layout, lengths, leftover bits) adds about 25 bytes.

Structured files (zstd -19, all 276 combinations scored at zstd -3, best 3 rescored):

Summary. The Exchange loses on text and database data. It wins on the two 16-bit medical images only at juggle 16, which separates each pixel's high and low byte; that byte split is known work. No odd juggle width reached the top 30 on any structured file, because all four files store data in whole bytes.

File (bytes)	zstd -19	Best Exchange (swap/juggle)
alice29.txt (152,089)	49,212	49,425 (1/8): loses
osdb, first 524,288	176,498	188,462 (1/8): loses
x-ray, first 524,288	318,413	**295,511** (8/16)
mr, first 524,288	151,927	**144,234** (8/16)

15.8 Biased and odd-width data

Data.

- NIST SP 800-90B entropy-assessment sample files: biased random bits, biased random bytes, ring-oscillator hardware noise, true-random 4-bit. The 1- and 4-bit samples were packed tightly.
- A real 24-bit EEG recording (BioSemi BDF, MNE testing data, first 1,048,576 bytes).
- Two synthetic files generated for this test (seed 2360932695): packed 5-bit symbols with a geometric distribution, and packed 12-bit Gaussian "ADC-like" noise.
- Real x-ray pixels (12-bit values) packed tightly to 12 bits: 525,000 bytes.

15.9 Ablation with a real encoder and decoder

A complete container format was written: header, mapping, leftover bits and both streams. Every result below was decoded and compared byte for byte, at odd widths and with leftover bits included. Four variants share the same (S, J):

- **full:** rename + split + reversed stream 2;
- **split:** no rename;
- **rename:** no split;
- **norev:** rename + split, stream 2 not reversed.

Summary.

- On biased sources the renaming does work that splitting cannot: full is 11.5% smaller than split-only on the 12-bit source and 11.9% smaller on the NIST biased bits.
- Rename-only is worse than doing nothing, so the gain comes from the two steps together.
- On the biased bits the Exchange beats xz -9e and bitshuffle; on the 12-bit source it beats every known alternative tested.
- On the 5-bit source, simply unpacking to bytes ties it.
- On real data with numeric locality (x-ray, MRI, EEG), split-only or plain byte shuffle wins and the renaming hurts. A likely reason, not proven by these tests, is that frequency ranking disrupts the numeric closeness of neighbouring samples.
- Reversing stream 2 has only a small, inconsistent effect (at most 0.55%).

Data (bytes)	zstd -19	xz -9e	Best known transform	Full	Split only	Rename only	No reversal
Synthetic 12-bit (524,288), S4/J12	430,664	403,956	368,027 (unpack to 16-bit + byte shuffle)	**325,568**	368,065	399,859	325,558

Data (bytes)	zstd -19	xz -9e	Best known transform	Full	Split only	Rename only	No reversal
NIST biased bits (125,000), S13/J21	22,172	21,772	22,558 (unpack + shuffle); 23,177 (bitshuffle)	**19,722**	22,375	25,909	19,715
NIST biased bytes (1,000,000), S8/J16	327,568	334,836	323,054 (byte shuffle)	**321,128**	321,353	322,942	322,883
Synthetic 5-bit (524,289), S1/J5	401,571	398,376	**345,655** (unpack to bytes)	346,370	346,363	345,698	346,354
x-ray (524,288), S8/J16	318,413	**267,052**	269,649 (byte shuffle)	295,019	269,972	330,150	295,115
mr (524,288), S8/J16	151,927	**132,496**	139,643 (byte shuffle)	143,742	139,652	154,725	144,149
EEG 24-bit BDF (1,048,576), S16/J24	526,856	345,184	**323,516** (byte shuffle)	335,895	334,882	532,373	335,713
Real x-ray packed to 12 bits (525,000)	452,101	421,472	360,945 (unpack + shuffle)	358,593 (S6/J12)	**342,496** (S8/J12)	438,477	—

Full sweeps at zstd -19 (all 276 combinations, no shortcut):

- 12-bit source: best S8/J12 = 324,507 bytes. The earlier two-stage search had picked S4/J12.
- Biased bits: best S13/J21 = 19,722. The optimum is broad: 13/21 through 16/24 are all within 0.5%.
- Real 12-bit x-ray: best full Exchange S6/J12 = 358,593.

Theoretical limits (order-0 entropy):

- 12-bit source: about 322,000 bytes, so the Exchange comes within about 1%.
- Biased bits: 17,688 bytes.
- 5-bit source: about 340,300 bytes.

15.10 Decode speed

Summary. A compiled C decoder for the Exchange is 5–12× slower than plain zstd, because it must rebuild each symbol bit by bit. On the 12-bit source it is still smaller *and* about 3.8× faster to decode than xz -9e. This is a size win against zstd and a head-to-head win against xz, not a No-Pain win.

Data	Exchange decode (median of 15)	Plain zstd	xz -9e	LZ4 default
Synthetic 12-bit	5.1 ms	0.89 ms	19.2 ms	0.016 ms
NIST biased bits	1.49 ms	0.12 ms	1.13 ms	0.065 ms
NIST biased bytes	7.4 ms	1.0 ms	15.3 ms	0.56 ms
Synthetic 5-bit	11.3 ms	0.71 ms	—	0.016 ms
X-ray packed 12-bit (split only)	5.4 ms	1.09 ms	—	0.016 ms
EEG 24-bit (full)	43.9 ms	1.90 ms	—	0.11 ms

The 5-bit, x-ray, EEG and LZ4 times were added on 28 October 2026 (same in-process method).

15.11 Prior art and originality

Prior art	Relation	Difference
Bitshuffle (Masui et al. 2015); HDF5/Blosc byte shuffle	Splits typed data into bit or byte planes before an LZ coder	Fixed planes, no permutation of values; measured worse on both biased files
Move-to-front and symbol ranking	Renames symbols adaptively	Ranks by recency, with no split into a slot field
Slot + extra-bits codes (Deflate length codes, LZMA position slots, Rice and Elias-γ codes)	Factor a symbol into a class and an offset	Classes are numeric ranges; the Exchange's classes come from frequency rank at any bit width
CCSDS 121/123 preprocessing	Maps prediction residuals before entropy coding	A fixed mapping, not derived from the data's frequencies
Canonical Huffman	Orders the alphabet by frequency	The order stays inside the coder; the Exchange exposes it as a reversible transform that makes the split effective

Claim. The combination is the claim: a frequency-ranked permutation onto prefix-weight targets, completed to a cheap reversible map, plus a free (S, J) split at arbitrary widths. No exact match was found in the searches made, but a formal literature and patent search has not been done. The transform is reversible and removes no information; its gain comes from presenting the data in a form the stock back end codes more efficiently.

15.12 Comparison with outside baselines (28 October 2026)

Summary. Other AI reviewers proposed a list of baseline compressors, and it was run on the same six files. On all four biased files, the Exchange was the smallest of the practical codecs (zstd -22, Brotli 11, 7-Zip ultra, Blosc2, OpenZL). On the 5-bit file it was also smaller than paq8px, the strongest context-mixing compressor tested. On the 12-bit file it came within 1,354 bytes of paq8px, which took about 189 seconds to decode, against 5.1 ms for the Exchange. On real x-ray and EEG data, renaming hurts: Blosc2 with bytedelta is the better practical choice there, and context mixing is well ahead.

Method.

- Every result was decompressed and checked byte for byte; all passed.
- Settings: zstd –ultra -22 –long=27; Brotli -q 11 -w 24; 7-Zip -mx9 LZMA2 (64 MB dictionary); ZPAQ -m5; paq8px v217 -8.
- Blosc2 (zstd level 9) used the best of typesize 1–4 × shuffle, bitshuffle and shuffle+bytedelta. OpenZL 0.3.0 used the best of its generic profiles.
- Decode times for the outside codecs are single wall-clock runs that include program start-up (Blosc2 was timed in-process). Exchange times are from Section 15.10. Sizes are exact; speeds are approximate.
- Not run: cmix (setup failure) and Pcodec (its package server was unreachable).

Data (original bytes)	Exchange	zstd -22	Brotli 11	7-Zip ultra	Blosc2	OpenZL	ZPAQ -m5	paq8px -8
12-bit noise (524,288)	**324,507**	430,667	384,360	399,818	360,045	433,897	326,799	323,153
Biased bits (125,000)	**19,722**	22,191	22,480	21,799	21,640	25,596	18,986	18,064
Biased bytes (1,000,000)	**321,128**	327,396	324,538	334,450	321,626	354,931	292,664	290,460
5-bit skewed (524,289)	**346,370**	401,574	388,457	401,291	401,446	401,240	373,175	351,007
X-ray packed 12-bit (525,000)	358,593 (split only 342,496)	452,104	381,349	419,581	347,041	478,877	317,968	292,973
EEG 24-bit (1,048,576)	335,895 (split only 334,882)	526,866	354,014	344,966	302,941	543,605	290,957	268,628

Bold marks the smallest practical result. ZPAQ and paq8px are context-mixing compressors and set the size ceiling.

Data	Exchange decode	ZPAQ -m5 decode	paq8px -8 decode
12-bit noise	5.1 ms	3,284 ms	189,253 ms
Biased bits	1.49 ms	444 ms	27,381 ms
Biased bytes	7.4 ms	4,002 ms	269,710 ms

Readings.

- On biased, packed data the Exchange is the best practical choice tested. Pushing zstd to -22 does not close the gap (430,667 vs 324,507 bytes on the 12-bit file).
- On biased bytes, Blosc2 is close (498 bytes larger) and faster to decode, and context mixing is about 30,000 bytes smaller, so headroom remains there.
- On real images and signals, Blosc2 bytedelta and context mixing win. This supports the selector rule in Section 15.6: rank for near-memoryless skewed sources, split or delta for data with numeric locality.
- OpenZL's generic profiles assume byte-aligned values and did worse than plain zstd on the packed files; a custom SDDL description was not tried.
- Two of the four biased files are synthetic. Real packed odd-width sources (camera RAW, telemetry) remain the most useful next test.

15.13 Tier placement and the niche crown (28 October 2026)

Summary. Measured against LZ4 on the four biased files, the Exchange's decode multiple is M ≈ 38, which places it in **Tier 3** (Section 2.3). In that tier, and in the faster tiers below it, it produced the smallest output of every rival tested on all four biased files. It is therefore the **Tier 3 compression king for biased, packed data** (*Tier BGS / Class / biased packed sources*), with the coverage limits stated below.

Measurement. Sum of in-process median decode times over the four biased files (12-bit, biased bits, biased bytes, 5-bit): Exchange 25.0 ms, LZ4 default 0.66 ms, so M = 25.0 / 0.66 ≈ 38. LZ4 cannot compress three of these files and effectively stores them, which makes its decode unusually fast and M higher than it would be on Canterbury. Even so, M stays well inside Tier 3 ($10 < M \le 100$).

Standing against each class of rival.

Rival	Tier	Result on the four biased files
zstd -22, Blosc2	1	Exchange smaller on all four
Brotli 11, OpenZL	2–3 (not measured)	Exchange smaller on all four
7-Zip ultra (LZMA2), xz -9e	3	Exchange smaller on all four; also faster than xz on the 12-bit and biased-byte files (a No-Pain win over xz there)
ZPAQ -m5, paq8px	5	Exchange smaller *and* faster than ZPAQ on 12-bit and 5-bit, and than paq8px on 5-bit (No-Pain wins). Elsewhere it is 1–2 KB larger on the bit-level files but 300–37,000× faster (Baseline BGS-3)

Limits.

- The crown covers biased, near-memoryless sources packed at non-byte widths. On real x-ray and EEG data, Blosc2 with bytedelta and the context-mixing compressors do better.
- Two of the four biased files are synthetic.
- cmix and Pcodec were not tested.
- Decode multiples for Brotli and OpenZL were not measured; the conclusion does not depend on their tier, because the Exchange was smaller than both on every biased file.

15.14 Exchange → Blosc2 hybrid (28 October 2026)

Summary. Blosc2 was the closest practical rival on the biased files and is much faster to decode, so a hybrid was built. It keeps the Exchange's frequency renaming but hands the result to Blosc2 instead of the bit-level split and zstd. On the 12-bit file the hybrid is a **No-Pain win over Blosc2**: faster to decode (0.66–0.72 ms against 0.76 ms) and 35,280 bytes smaller. It keeps the Exchange's ratio at about seven times the Exchange's decode speed. On the other three biased files it is smaller than Blosc2 but still 1.3–2.5× slower to decode.

Design (Mutated: needs its own decoder).

1. Unpack the J-bit symbols into 8-, 16- or 32-bit integers.
2. Apply the Exchange's frequency-ranked renaming (Section 15.7).
3. **Fast mode:** one renamed stream. **Max mode:** split each renamed symbol at S into a high and a low stream.
4. Compress each stream with Blosc2. Codec, typesize, filter and block size are chosen per stream; zstd level 9 with 1 MB blocks was usually best.
5. The decoder runs Blosc2, applies the inverse renaming through a small lookup table, and repacks to J bits with width-specific loops for 5, 12 and 16 bits.

A Modified (stock-decodable) version is not possible: stock Blosc2 can only read untransformed data, and its options were already searched exhaustively for the baseline.

Method. Decode times are in-process medians of 21 runs, single-threaded, for both Blosc2 and the hybrid, repeated in separate sessions; ranges are shown where runs differed. Every result was decompressed and checked against the original (the last few bytes of each file, stored raw, were excluded from the check). Container size includes the renaming table plus an estimated 16-byte header.

File (original bytes)	Stock Blosc2	Exchange (zstd)	Hybrid, best mode	Result against Blosc2
12-bit noise (524,288)	360,045 B, 0.76 ms	324,507 B, 5.1 ms	**324,765 B, 0.66–0.72 ms** (max, S = 8)	**No-Pain: smaller and faster**
Biased bits (125,000)	21,640 B, 0.10 ms	19,722 B, 1.5 ms	20,777 B, 0.24–0.26 ms (fast)	863 B smaller, 2.5× slower
Biased bytes (1,000,000)	321,626 B, 0.93–1.19 ms	321,128 B, 7.2 ms	321,168 B, 1.26–1.29 ms (max, S = 8)	458 B smaller, about 1.3× slower
5-bit skewed (524,289)	401,048 B, 0.66 ms	346,370 B, 11.3 ms	345,725 B, 0.93 ms (fast)	55,323 B smaller, 1.4× slower

Readings.

- The Exchange's gain comes from the renaming, not from its bit-level split: once renamed, the data compresses about as well under Blosc2 as under the original split-and-zstd design, and decodes 6–12× faster.
- On the 5-bit file the hybrid (345,725 B) is also smaller than paq8px (351,007 B), at 0.93 ms against about 180 s.
- On the biased bits the file is small enough that fixed costs (table setup, a second stream) dominate. On the biased bytes the data is already near its entropy limit, so there is little left to gain against Blosc2.

- The remaining gap on the 5-bit file is mostly Blosc2's own work on 838,861 one-byte symbols; a narrower symbol container is the next thing to try.

15.15 Real-world sources: telemetry and radio (28 October 2026)

Summary. The Exchange was tested on two kinds of real data. **Flight telemetry was a clear loss:** on whole log files and on raw channels extracted from them, Blosc2 or xz won every time, because telemetry is smooth, correlated measurement and renaming destroys that structure. **Software-defined radio captures were mixed:** the Exchange family won the two noisiest captures and xz-hi won the other four. Each configuration was run once.

Data (all public, from GitHub).

- Telemetry logs: IRIG-106 Chapter 10 flight-test recordings (pychapter10 test files), a PX4 flight log (pyulog), a MAVLink drone log and an ArduPilot log (pymavlink). Each was capped at 1 MB.
- Raw telemetry channels: the PCM payload of the IRIG-106 file with packet headers removed, and PX4 topics (IMU, attitude, local position, actuator outputs) stored one field after another.
- Radio: 8-bit I/Q captures of 433 MHz sensor transmissions from the rtl_433 test set (five devices plus a 1 MB mix of many captures).

Raw telemetry channels (bytes).

Stream (original bytes)	zstd -19	xz -9e	Blosc2	ZPAQ -m5	Exchange	Exchange → Blosc2
IRIG-106 PCM sensor words (312,060)	78,862	64,464	23,577	15,275	45,654	33,806
PX4 IMU (1,092,480)	400,927	363,504	391,488	330,820	413,169	403,533
PX4 attitude (180,908)	127,431	116,948	103,458	102,275	121,853	112,537
PX4 local position (77,970)	16,124	12,760	10,890	14,256	16,174	13,770
PX4 actuator outputs (89,148)	36	160	75	1,101	79	95

Whole log files gave the same picture (Exchange worst or near worst on all six).

Radio captures (bytes).

SDR capture (original bytes)	zstd -19	Blosc2	xz -9e	xz-hi	Exchange (zstd)	Exchange → xz	ZPAQ -m5 (Tier 5)
Acurite 590TX (393,216)	152,695	147,665	138,676	**137,324**	149,830	139,465	103,586
Acurite 606TX (524,288)	381,155	380,829	354,460	353,428	372,472	**351,893**	340,041
Fine Offset WH1080 (393,216)	214,689	213,526	189,884	**189,304**	209,020	194,373	174,672
Fine Offset WH51 (1,000,000)	758,000	758,039	759,688	759,512	**753,163**	756,829	745,302
LaCrosse TX141 (393,216)	268,770	268,387	239,748	**239,080**	263,710	245,809	225,363
Mixed captures (1,048,576)	721,984	689,817	626,532	**620,948**	685,805	635,977	577,982

Bold marks the smallest result outside Tier 5.

Decode times on the radio captures (measured afterwards, in-process, median of 15 runs; every result round-trips exactly). Each cell gives size in bytes, then decode time.

SDR capture	Exchange (zstd)	Exchange → xz	xz -9e	xz-hi	zstd -19 (time)
Acurite 590TX	149,830 B, 2.7 ms	139,837 B, 9.2 ms	138,676 B, 8.8 ms	**137,324 B, 8.0 ms**	0.56 ms
Acurite 606TX	372,472 B, 6.5 ms	**351,893 B, 28.0 ms**	354,460 B, 28.0 ms	353,428 B, 25.2 ms	0.77 ms
Fine Offset WH1080	209,020 B, 2.8 ms	194,373 B, 16.9 ms	189,884 B, 14.9 ms	**189,304 B, 16.5 ms**	0.63 ms
Fine Offset WH51	**753,163 B, 11.8 ms**	756,829 B, 58.9 ms	759,688 B, 52.0 ms	759,512 B, 56.8 ms	1.03 ms
LaCrosse TX141	263,710 B, 4.9 ms	245,809 B, 17.4 ms	239,748 B, 18.8 ms	**239,080 B, 16.6 ms**	0.73 ms
Mixed captures	685,805 B, 11.6 ms	635,977 B, 49.7 ms	626,532 B, 46.1 ms	**620,948 B, 47.8 ms**	1.74 ms

- **WH51: No-Pain win over xz and xz-hi.** The full Harrington Exchange (8-bit symbols split 4/4, zstd back end) is smaller than both and decodes about 5× faster.
- **606TX: size win.** The Exchange's split with an xz back end is the smallest; it decodes as fast as xz -9e but 11% slower than xz-hi.
- On the other four captures xz-hi is smallest, while the Exchange decodes 3–6× faster than xz.
- In this timing pass Exchange → xz used the quick xz search on every file, so its 590TX size (139,837 B) differs slightly from the full-search figure above (139,465 B). Compression times were not recorded.

ZPAQ is in a different class. ZPAQ -m5 was smallest on every radio capture, but it is a Tier 5 context-mixing compressor. Its decode time was not measured on these files; it is **estimated at about 3–5 seconds per megabyte**, based on its measured times on the biased files (Section 15.12), since its speed varies little with content. The other compressors decode these captures in milliseconds (table above). ZPAQ's results are therefore size records in a far slower tier, not wins against the practical compressors.

Notes on method.

- Exchange → xz compresses the Exchange's two streams with xz instead of zstd. On the first capture both it and xz-hi used the full xz-hi search; on the other five, Exchange → xz used only the 36 basic xz settings (lc, lp, pb) while xz-hi used its full search, which slightly favours xz-hi.
- The best Exchange → xz setting on every capture was the split alone (J = 16, S = 8) without renaming.

Readings.

- The Exchange's area is confirmed from the other side: it wins only where the data behaves like skewed noise. The two radio captures it won are the noisiest, and on one of them (WH51) the win is No-Pain against xz; the captures with more signal structure, and all telemetry, went to xz, xz-hi or Blosc2.
- Data with an uneven value distribution is not enough on its own. The values must also be close to independent of their neighbours.

Chapter 16 — Images and Audio

Executive summary. Medical imaging is where lossless compression matters most: radiographs must be stored exactly, for years, in large volumes. Using only the standard JPEG XL decoder, per-image setting choices made several radiographs decode up to seven times faster at almost the same size, or smaller at the same speed. A tuned JPEG-LS setting gave the best size-and-speed point on the Silesia x-ray. For consumer formats — WebP and PNG — and for FLAC audio, the gains were small or came from methods that already exist, and are reported here for completeness.

16.1 JPEG XL (jxl-hi) on medical images

Images:

- Silesia **x-ray**: 1900 × 2230 pixels, 12-bit.
- Silesia **mr**: 19 slices of 512 × 512 pixels, 10-bit DICOM.
- Nine images from pydicom's test data: CT (693, eCT), MR (MR2, MR-SIEMENS), CR radiographs (RG1, RG3), ultrasound (OBXXXX1A, ALOKA) and nuclear medicine (JPEG2000_UNC).

Baseline rule. Effort 7, cjxl's default, is held fixed. Only other encoder parameters vary: predictor (-P), group size (-g) and MA-tree iterations (-I). Earlier wins that came from changing effort (e3–e5) were withdrawn under the baseline rule. Stock djxl decodes every output.

Summary. With the stock JPEG XL decoder, four radiographs became both smaller and faster to decode: x-ray 2.3% smaller and 1.35× faster, RG1 2.2% smaller and 1.04× faster, RG3 0.5% smaller and 1.30× faster, and mr 0.1% smaller and 1.06× faster. Accepting 3–5% more bytes gave much faster decoding: RG1 7.5×, x-ray 7.1×, MR2 4.8×. Most small images gained no candidate that survived re-timing.

Table 16.1 — JPEG XL candidates against stock cjxl effort 7. Complete re-timed candidates; MP/s means megapixels per second, so higher is faster. **Bold** marks Harrington wins: smaller and not slower, or a BGS award (values unchanged).

Image	Stock cjxl e7 (bytes / decode MP/s)	Candidate setting	Size	Decode MP/s	Size change	Speed-up
693_UNCR	81,194 / 12.4	`e7-P15-g2-I100`	80,368	10.5	-1.0%	0.85×
693_UNCR	81,194 / 12.4	`e7-P15-gd-Id`	81,084	12.1	-0.1%	0.98×
JPEG2000_UNC	16,643 / 15.2	`e7-P5-gd-I100`	15,684	14.9	-5.8%	0.98×
JPEG2000_UNC	16,643 / 15.2	`e7-P14-gd-I100`	16,479	14.4	-1.0%	0.95×
MR-SIEMENS-DICOM-WithOverlays	78,209 / 13.2	`e7-P15-gd-I100`	78,030	12.8	-0.2%	0.97×
MR-SIEMENS-DICOM-WithOverlays	78,209 / 13.2	`e7-P15-gd-Id`	78,220	13.0	+0.0%	0.99×
MR2_UNCR	586,979 / 9.6	`e7-P14-g2-I100`	560,973	9.2	-4.4%	0.96×
MR2_UNCR	586,979 / 9.6	**`e7-P5-gd-I0`**	**607,853**	**46.6**	**+3.6%**	**4.83×**
OBXXXX1A	14,223 / 20.2	`e7-P15-gd-I100`	13,802	18.0	-3.0%	0.89×
OBXXXX1A	14,223 / 20.2	**`e7-P5-g0-Id`**	**14,909**	**27.7**	**+4.8%**	**1.37×**
RG1_UNCR	4,244,086 / 7.3	**`e7-P15-g3-I100`**	**4,150,669**	**7.6**	**-2.2%**	**1.04×**
RG1_UNCR	4,244,086 / 7.3	**`e7-P1-g0-I0`**	**4,365,124**	**54.9**	**+2.9%**	**7.48×**
RG3_UNCR	809,540 / 15.0	**`e7-P15-g0-I100`**	**805,242**	**19.5**	**-0.5%**	**1.30×**
RG3_UNCR	809,540 / 15.0	`e7-P14-g0-I0`	838,344	25.6	+3.6%	1.71×
eCT_Supplemental	52,019 / 14.9	`e7-P14-g2-I100`	51,431	12.2	-1.1%	0.82×
gdcm-US-ALOKA-16	56,563 / 12.9	`e7-P15-g2-I100`	56,224	11.3	-0.6%	0.87×
gdcm-US-ALOKA-16	56,563 / 12.9	`e7-P15-g0-I100`	59,111	11.9	+4.5%	0.92×
xray	3,631,359 / 7.5	**`e7-P15-g1-I100`**	**3,547,061**	**10.1**	**-2.3%**	**1.35×**
xray	3,631,359 / 7.5	`e7-P2-g1-I0`	3,810,649	53.6	+4.9%	7.11×
xray	3,631,359 / 7.5	**`e7-P5-g0-I0`**	**3,641,608**	**40.0**	**+0.3%**	**5.31×**
mr	1,836,957 / 11.8	**`e7-P14-g1-I100`**	**1,834,723**	**12.4**	**-0.1%**	**1.06×**
mr	1,836,957 / 11.8	`e7-Pd-g0-Id`	1,883,768	13.6	+2.5%	1.16×

Awards (Baseline vs cjxl e7):

Image	Setting	Result	Award
x-ray	-P 5 -g 0 -I 0	+0.3% size, 5.3× faster	**BGS-2 (speed) / Local**
x-ray	-P 15 -g 1 -I 100	2.3% smaller, 1.35× faster	**BGS-1 (speed) / Local**, also smaller
RG1	-P 1 -g 0 -I 0	+2.9% size, 7.5× faster	**BGS-2 (speed) / Local**
MR2	-P 5 -I 0	+3.6% size, 4.8× faster	**BGS-2 (speed) / Local**
RG3	-P 15 -g 0 -I 100	0.5% smaller, 1.30× faster	**BGS-1 / Local**
OBXXXX1A	-P 5 -g 0	+4.8% size, 1.37× faster	**BGS-1 / Local**
RG1, mr	best size-and-speed candidate	smaller and faster	**No-Pain / Local**
JPEG2000_UNC	-P 5 -I 100	5.8% smaller; decode not faster on interleaved re-timing (21.7 → 23.8 ms, faster in 9 of 21)	**withdrawn** — size gain only (Appendix F)
693, MR-SIEMENS, ALOKA, eCT	—	no candidate passed re-timing	none

Images of about 0.5 megapixels show roughly 15% timing noise, so small-image awards are provisional.

16.2 JPEG-LS (tuned)

Tuned thresholds T1, T2, T3 = 12, 32, 64 and RESET = 255 on x-ray: 1,000,000 bits down to 423,801 → 421,829 (3,591,153 → 3,574,443 bytes), and decode 103.6 → 87.6 ms. Re-verified on 28 October with CharLS: the sizes match, and decoding was again faster (119.6 → 100.0 ms). An earlier version of this section labelled the normalised figures as bytes; that is corrected here. **No-Pain / Local.** Tuned JPEG-LS is the best size-and-speed point measured on x-ray.

16.3 Sample screening (the author's method)

Instead of searching settings on a whole image, the encoder first screens candidates on four sample tiles. It then checks only the top 3–10 candidates on the full image.

- **Search time:** about 100 times less.
- **Large radiographs** (x-ray, RG1): the screened choice matched the full-search choice.
- **MR2 and the 693 CT:** the tiles missed the background, and screening chose weaker settings.
- **Fix:** choose representative tiles that cover both background and anatomy.

16.4 WebP lossless — Kodak (24 images)

Baseline `cwebp -lossless -z 9`; the search covers -m 4–6 × -q 25–100; stock dwebp decodes. Compressed bytes (each image is 768 × 512 × 24-bit, 1,179,648 bytes raw), command-line decode.

Summary. The search improved 4 of 24 images, saving 2,006 bytes in total (11,204,040 → 11,202,034, −0.018%), with total decode 1.5% faster (262 → 258 ms, within command-line timing noise).

Image	-z 9	Decode	Best No-Pain	Setting	Decode
01	499,410	11.1 ms	499,282	-m 5 -q 75	9.3 ms
02–09	450,562 / 383,374 / 455,046 / 556,351 / 464,144 / 414,684 / 546,722 / 434,022	—	no gain	—	—
10	440,574	10.7 ms	439,685	-m 4 -q 50	10.6 ms
11	447,192	10.6 ms	no gain	—	—
12	403,224	11.6 ms	402,343	-m 4 -q 100	10.1 ms
13–15	597,831 / 512,410 / 427,336	—	no gain	—	—
16	414,292	10.5 ms	414,184	-m 5 -q 50	9.4 ms
17–24	443,168 / 557,214 / 481,758 / 359,198 / 488,062 / 515,702 / 422,106 / 489,657	—	no gain	—	—
Total	**11,204,040**	**262 ms**	**11,202,034**		**258 ms**

WebP's maximum preset is already essentially optimal for this data.

16.5 PNG — Kodak (known technique; not credited)

Baseline: Pillow `optimize=True`. The search covers a fixed filter type (0–4) or per-row adaptive filtering × zlib strategy (default, filtered, RLE). Decoding is by Pillow's zlib decoder. This is what optipng and similar tools already do, so it is reported but not claimed.

Image	Pillow	Decode	Best No-Pain	Decode		Image	Pillow	Decode	Best No-Pain	Decode
01	778,467	11.3	760,239	7.8		13	889,437	7.4	—	—
02	617,087	9.5	616,979	8.1		14	738,539	10.1	703,024	9.0
03	540,104	6.8	—	—		15	611,354	10.4	603,463	7.7
04	636,952	9.0	636,844	8.8		16	572,939	7.5	551,426	7.3
05	806,670	8.7	—	—		17	623,527	9.5	610,316	9.2
06	667,036	9.0	631,466	7.6		18	779,447	9.6	779,314	9.5
07	565,365	12.8	557,524	7.4		19	670,504	9.8	670,385	9.8
08	786,611	12.0	—	—		20	504,880	7.8	503,931	6.6
09	582,573	8.9	582,477	8.8		21	679,398	7.7	650,944	7.1
10	592,834	10.7	592,726	10.6		22	700,684	9.9	700,563	9.0
11	634,979	8.1	—	—		23	556,888	8.8	—	—
12	568,487	10.3	533,852	7.9		24	704,818	8.8	697,628	7.6

Compressed bytes; decode times in ms. Total: **≈15,809,700 → ≈15,597,900 bytes** (−1.3%), decode 225 → 202 ms (10% faster).

16.6 Canterbury ptt5 as an image

ptt5 is a 1728 × 2376 one-bit fax scan.

Summary. On the ptt5 fax image (513,216 bytes), lossless WebP is 13% smaller than 1-bit PNG but 3.5× slower to decode. The general-purpose zstd -22 is smaller than both image formats, and HCM v3.3 fast is 47% smaller than PNG, at a much longer decode time.

Method	Compressed bytes	Decode
PNG, 1-bit	53,274	2.5 ms
WebP lossless -z 9	46,266	8.7 ms (CLI)
WebP -m 6 -q 50	43,802	11.8 ms — slower, fails
zstd -22 (general-purpose)	43,548	—
HCM v3.3 fast	28,125	406 ms

The image formats lose to general-purpose compressors on this file.

16.7 FLAC — speech

Six speech files: the Free Spoken Digit Dataset, 8 kHz, 16-bit mono, concatenated per speaker. No lossless music source was reachable from the test environment. Baseline `flac -8`; stock flac decodes. Command-line timing.

Summary. Per-file settings made five of six speech files smaller and faster to decode: the total fell -2.1% (11,162,773 → 10,924,361 bytes) and decode time fell 10% (136 → 123 ms, command-line timing).

Speaker (WAV bytes)	flac -8 (bytes)	Decode	Best No-Pain (bytes)	Setting	Decode
george (3,533,784)	2,361,716	23.1 ms	2,322,527	-b 1152 -e -r 8	22.7 ms
jackson (4,131,724)	2,708,225	26.0 ms	2,656,161	-b 1152 -e	22.0 ms
lucas (4,593,732)	2,623,508	32.2 ms	2,524,164	-b 1152 -e -p	27.0 ms
nicolas (2,793,546)	530,243	18.0 ms	509,892	-b 1152 -e -p	16.9 ms
theo (3,110,942)	1,546,490	19.1 ms	1,519,026	-b 576 -e -p	16.3 ms (not confirmed: interleaved re-timing 36.1 → 37.0 ms, faster in 7 of 21)
yweweler (2,833,384)	1,392,591	17.9 ms	no gain (1,346,614 available but slower)	—	—
Total (20,997,112)	**11,162,773**	**136 ms**	**10,924,361**		**123 ms**

Most of the gain comes from a block size suited to 8 kHz audio. flac -8 uses 4,096 samples, which is about half a second at 8 kHz. The remaining switches (-e, -p, -r) are FLAC's own. **No-Pain / Local / speech** is credited only for the per-file block-size selection; music remains untested.

Re-verification note (28 October 2026). A random re-test (Appendix F) confirmed george and jackson as No-Pain. The theo setting is smaller but not faster to decode on interleaved re-timing, so FLAC is credited with four speech wins, not five.

16.8 Fax and bilevel pages

Fax pages are 1-bit images. Fax standards already code each line against the line above:

- Group 3 2-D mode (ITU-T T.4) and Group 4 (T.6) code changes relative to a reference line;
- JBIG and JBIG2 predict each pixel from a template of nearby pixels, including rows above, with an adaptive arithmetic coder.

The "row above" idea is therefore established prior art for this kind of data.

Test set. Canterbury ptt5 (a real fax page, 1728 × 2376), plus three randomly drawn pdf.js pages rendered at 200 dpi monochrome (seed 2369267215). Two of the rendered pages are almost blank. Single snapshot runs; all round trips exact.

Summary. No method of ours won on speed. HCM was second-smallest on the real fax and smallest on the busier rendered page, but decoded 5–60 times slower than the fax coders. The fastest decoding was plain, unmodified zstd, which is not our work. Record delta and the activity map did not help, and neither did fax-style tricks (XOR with the row above, skipping identical rows): zstd already finds repeated rows.

ptt5 (513,216 B raw)	Bytes	Decode
JBIG2 generic (jbig2enc)	**25,378**	67 ms
HCM max	27,480	422 ms
HCM fast	28,054	330 ms
TIFF Group 4	33,316	5.9 ms
zstd -19 + activity map	42,483	0.96 ms
zstd -19	43,640	0.30 ms
zstd -19 + row delta	47,792	0.33 ms
TIFF Group 3	80,850	5.8 ms

Busier rendered page (484,173 B raw)	Bytes	Decode
HCM max	**4,035**	339 ms
HCM fast	4,055	419 ms
zstd -19	5,236	0.11 ms
JBIG2 generic	7,678	62 ms
TIFF Group 4	23,090	4.6 ms
TIFF Group 3	43,992	3.8 ms

JBIG2's duplicate-line removal (TPGD) made two pages smaller and decode about twice as fast. It is a stock option of the format, so it earns no credit. On fax data the methods in this book are a slow size contender at best; a faster bilevel mode would have to adopt JBIG-style pixel templates, which are known work.

Chapter 17 — Columnar Formats (Exploratory)

Executive summary. Companies increasingly store data in columnar formats such as Parquet and ORC, which compress well because similar values sit next to each other. When an organisation imports *unknown* binary data — legacy exports or instrument dumps — it does not know the column layout. HCM's record detector can infer one automatically. On four of seven test files, reshaping the data into the detected columns made it both smaller and faster to read with stock Parquet and ORC readers. The baseline used here is deliberately simple, so these results show the mechanism rather than a production benchmark.

17.1 Method

- **Files.** The seven binary files of the two corpora: kennedy.xls, ptt5 and sum (Canterbury); mr, x-ray, sao and osdb (Silesia). Text files were excluded as not tabular.
- **Baseline.** The file stored as a one-column table of bytes (UTINYINT / TINYINT), one byte per row.
- **Candidate.** The file reshaped into *R* columns, where *R* is the detected record length (kennedy 13, sao 28, the others 2), one record per row. Any tail shorter than a record is stored separately and counted in the size.
- **Parquet.** DuckDB 1.3.2 writer and reader, single-threaded. Read time is a full `CREATE TABLE AS SELECT *`.
- **ORC.** Apache ORC C++ built from current source. In-process writer and reader; read time is a full scan into memory.
- **Verification.** Exact byte reconstruction (ORC) and row-count and column-sum checks (Parquet).
- **Class.** *Mutated*: stock readers, but the layout is chosen by the author's detector.

17.2 Parquet (DuckDB)

Summary. With Parquet, the detected-column layout is smaller on kennedy.xls, mr, x-ray and sao with both codecs: −58% (zstd) and −75% (Snappy) on kennedy.xls, −17% to −35% on x-ray, mr and sao with zstd. Read time fell on those three larger files by 21–32% (for example x-ray 137 → 102 ms with zstd). On ptt5, sum and osdb the plain layout is kept.

Compressed bytes / read time.

File (bytes; R)	zstd, 1 column	zstd, R columns	Snappy, 1 column	Snappy, R columns
kennedy.xls (1,029,744; R = 13)	163,150 / 11 ms	**68,737 / 11 ms**	449,841 / 11 ms	**113,673 / 11 ms**
ptt5 (513,216; R = 2)	**64,290 / 6 ms**	79,197 / 6 ms	**94,750 / 6 ms**	107,420 / 6 ms
sum (38,240; R = 2)	**15,203 / 1 ms**	18,391 / 1 ms	**21,074 / 1 ms**	25,562 / 1 ms
mr (9,970,564; R = 2)	4,579,869 / 155 ms	**3,820,062 / 123 ms**	6,499,551 / 151 ms	**4,464,799 / 119 ms**
x-ray (8,474,240; R = 2)	8,133,025 / 137 ms	**5,295,112 / 102 ms**	9,700,259 / 121 ms	**5,506,841 / 100 ms**
sao (7,251,944; R = 28)	6,520,071 / 119 ms	**5,366,265 / 81 ms**	7,801,337 / 115 ms	**6,241,748 / 77 ms**
osdb (10,085,684; R = 2)	**6,799,244 / 169 ms**	7,547,823 / 138 ms	**10,779,892 / 153 ms**	10,987,284 / 124 ms

DuckDB's timer has millisecond resolution, so the small files tie.

17.3 ORC

Summary. With ORC, the detected-column layout is smaller for kennedy.xls, mr, x-ray and sao with all three codecs. The clearest win is x-ray with zlib: 23.9% smaller (6,168,696 → 4,695,458 bytes) and read 1.8× faster (46.1 → 25.6 ms). kennedy.xls is 68–84% smaller; it reads 1.3–1.6× faster with zlib and Snappy and about the same with zstd (1.7 → 1.8 ms, at the timer's resolution). With Snappy on x-ray and sao, and zstd on sao, the smaller layout reads slower, so those are size trades.

Compressed bytes / read time (ms).

File (bytes; R)	zlib, 1 col	zlib, R cols	zstd, 1 col	zstd, R cols	Snappy, 1 col	Snappy, R cols
kennedy.xls (1,029,744; R = 13)	265,673 / 2.8 ms	**51,812 / 1.8**	155,173 / 1.7	**50,081 / 1.8**	442,287 / 1.8	**71,132 / 1.4**
ptt5 (513,216; R = 2)	**56,924 / 0.9**	62,456 / 1.1	**54,292 / 0.7**	58,547 / 0.8	**77,645 / 0.6**	86,072 / 0.7
sum (38,240; R = 2)	**14,658 / 0.2**	17,200 / 0.2	**15,209 / 0.1**	18,489 / 0.1	**19,557 / 0.1**	22,977 / 0.1

File (bytes; R)	zlib, 1 col	zlib, R cols	zstd, 1 col	zstd, R cols	Snappy, 1 col	Snappy, R cols
mr (9,970,564; R = 2)	3,825,865 / 34.8	**3,097,764 / 27.4**	3,895,699 / 14.2	**3,167,439 / 12.5**	5,352,837 / 15.3	**3,823,841 / 9.2**
x-ray (8,474,240; R = 2)	6,168,696 / 46.1	**4,695,458 / 25.6**	6,855,465 / 13.6	**4,703,390 / 10.2**	8,538,102 / 6.2	4,911,737 / 9.8
sao (7,251,944; R = 28)	5,611,170 / 37.3	**4,972,071 / 29.4**	6,397,107 / 11.6	4,863,785 / 18.0	6,550,036 / 11.4	5,879,441 / 16.1
osdb (10,085,684; R = 2)	**4,628,694 / 43.6**	5,516,163 / 51.5	**4,705,617 / 15.0**	5,939,066 / 18.2	**5,557,716 / 13.4**	7,155,662 / 16.1

Bold marks the better layout. Where the R-column layout is smaller but slower to read — x-ray and sao with Snappy, sao with zstd — the result is a size trade, not No-Pain.

17.4 Findings

1. **The same four files respond in every format and codec:** kennedy.xls, mr, x-ray and sao. They are also the files that respond to record delta (Chapter 4), which indicates the detector identifies real structure rather than a quirk of one codec.
2. **Wrong layouts are expensive.** On ptt5, sum and osdb the detected *R* does not correspond to real fields, and the reshaped table is 10–40% larger. A production importer must test both layouts and keep the better one, as done here.
3. **Read time often improves** as well as size, because the reader decompresses less data and decodes simpler columns.

17.5 Limitations

- **Artificial baseline.** Nobody stores a raw byte stream as a one-column Parquet table. The realistic comparison is against a hand-designed schema, which this experiment does not attempt.
- **Single-byte columns.** Fields wider than a byte, such as the 16-bit pixels of mr and x-ray, are split into byte columns. Typed multi-byte columns would likely do better again.
- **Small test set.** Seven files; real-world ingestion data remains untested.

Chapter 18 — Modern Log and Export Data

Executive summary. Twelve modern files were tested: server logs, JSON logs, CSV exports and the same data stored as Parquet and ORC. HCM was the smallest on the four log files, 22–41% smaller than the best standard compressor, but it decoded 24 to 359 times slower and lost badly on the large JSON and CSV exports, where its tables saturate. No-Pain xz tuning (xz-hi) made 6 of 12 files smaller and faster to open with stock xz, a modest 0.3% overall. Record delta and the activity map, so strong on fixed-record binaries, failed on this data: these are variable-length text records, and the detector's guesses (R = 2–11) do not match real fields. Every result was verified lossless.

18.1 Data

File	Bytes	Source	Detected R
hdfs.log	287,848	loghub HDFS 2k sample	5
openstack.log	595,119	loghub OpenStack 2k sample	5
apache_access.log	2,370,789	elastic/examples apache_logs	2
nginx.json	12,137,777	elastic/examples nginx_json_logs	3
nginxplus.json	2,784,760	elastic/examples nginx_json_plus_logs	10
counties2023.jsonl	27,059,879	NYT covid-19-data us-counties-2023, via DuckDB 1.3.2	2
counties2023.csv	11,541,836	NYT covid-19-data us-counties-2023	3
states.csv	2,217,680	NYT covid-19-data us-states	2
counties2023_uncompressed.parquet	3,622,377	DuckDB 1.3.2, no compression	11
counties2023_snappy.parquet	1,604,901	DuckDB 1.3.2, Snappy	11
counties2023_zstd.parquet	456,662	DuckDB 1.3.2, zstd	3
counties2023_zlib.orc	1,445,146	Apache ORC csv-import, zlib; schema `struct<date:string,county:string,state:string,fips:string,cases:bigint,deaths:bigint>`	11

Total: 66,124,774 bytes.

18.2 Method

- **Machine:** cloud Linux VM, Intel Xeon @ 2.10 GHz, 2 vCPUs, kernel 6.18; all runs single-threaded.
- **Versions:** gzip 1.12, bzip2 1.0.8, xz 5.4.5, zstd 1.6.0 (built from source), lz4 1.10.1, HCM via *hcmx* (v3.3 + trit2 + Bin1 best-of), built with g++ 13.3 `-O2 -march=x86-64-v2`.
- **Commands:** `gzip -9n`, `bzip2 -9`, `xz -9e -T1`, `zstd -19`, `zstd --ultra -22`, `lz4 -12`, `hcmx c` (fast) and `hcmx x` (max).
- **zstd-hi:** levels 19 and 22 × windowLog 27, searchLog 7 or 9, minMatch 3–6, targetLength 999, strategy 9; the smallest is kept.
- **xz-hi:** `--lzma2=preset=9e` × 8 lc/lp/pb settings × nice 64 or 273, plus `--delta=dist=R` variants.
- **Fallback:** a tuned result counts only if it is not larger and its decode is not slower than its base (zstd -22 or xz -9e); otherwise the base is kept.
- **Record delta (RD):** stock zstd -19 or xz -9e on the delta, decoded through the author's SIMD inverse (`undelta R`). The header is 2 bytes.
- **Activity map (AM):** flags and literals compressed separately with gzip -9 or zstd -19, rebuilt by `actbuild`. The header is 6 bytes.
- **Measurement:** each command ran through a fork/exec wrapper reading `wait4` rusage, giving wall time, CPU time and peak RSS. Decode time is the median of 5 runs. For pipelines, memory is the largest single process. Energy = CPU seconds × 15 W.
- **Verification:** every decode was compared byte for byte with the original. All 120 method–file results passed.
- **Not recorded:** activity-map encode time and memory (the map was built in Python, so its time is not representative).

18.3 Compressed sizes

Summary.

- **HCM max** was the smallest on the three plain logs and nginx.json: −26.3% (hdfs), −21.9% (openstack), −32.0% (apache) and −40.5% (nginx.json) against the best stock compressor.
- On the large exports, HCM lost: +70.9% on counties2023.jsonl, +40.1% on counties2023.csv and +14.7% on uncompressed Parquet against xz -9e. Over all 12 files, HCM max (4,340,977 bytes) was 7.1% larger than xz -9e (4,053,776).

- **xz -9e** was the best stock compressor on 10 of 12 files; bzip2 won hdfs.log and zstd -19 won the zstd-compressed Parquet.
- Already-compressed Parquet and ORC gained almost nothing from any method (−1.6% at best, by HCM on zstd Parquet).

Compressed bytes. Bold is the smallest per file; “(fb)” means the tuned result failed the No-Pain test and the base is kept.

File (bytes; R)	gzip -9	bzip2 -9	xz -9e	zstd -19	zstd -22	lz4 -12	HCM fast	HCM max	zstd-hi	xz-hi
hdfs.log (287,848; 5)	53,757	42,132	42,348	42,811	42,811	68,556	31,388	**31,068**	42,811	42,072
openstack.log (595,119; 5)	53,083	42,697	38,656	40,696	40,609	69,117	30,723	**30,192**	40,397 (fb)	38,308
apache_access.log (2,370,789; 2)	225,894	129,196	118,072	126,833	126,446	239,601	81,407	**80,254**	126,418 (fb)	117,124
nginx.json (12,137,777; 3)	384,543	234,581	228,132	264,718	264,444	479,794	137,526	**135,706**	263,841 (fb)	227,416
nginxplus.json (2,784,760; 10)	94,399	88,401	**50,932**	56,386	53,683	123,167	51,353	50,973	53,105 (fb)	50,864 (fb)
counties2023.jsonl (27,059,879; 2)	3,817,554	1,816,320	**355,052**	479,534	484,317	4,815,342	613,027	606,643	479,500	341,088 (fb)
counties2023.csv (11,541,836; 3)	3,264,038	1,329,744	332,984	424,017	424,313	3,989,134	522,869	466,414	424,017 (fb)	**324,760**
states.csv (2,217,680; 2)	436,312	374,167	218,628	257,634	257,621	596,552	219,895	**217,116**	257,621 (fb)	215,976 (fb)
counties2023_uncompressed.parquet (3,622,377; 11)	638,939	671,642	**307,676**	326,549	326,280	632,668	356,094	352,836	326,280	304,512 (fb)
counties2023_snappy.parquet (1,604,901; 11)	807,359	790,629	587,904	620,405	620,308	815,390	592,827	**583,793**	620,308 (fb)	581,732 (fb)
counties2023_zstd.parquet (456,662; 3)	433,297	438,703	421,612	420,384	420,384	435,086	**413,560**	413,734	420,384	419,232
counties2023_zlib.orc (1,445,146; 11)	1,390,126	1,399,094	**1,351,780**	1,354,529	1,354,528	1,397,258	1,375,519	1,372,248	1,354,528	1,350,816 (fb)
Total (66,124,774)	**11,599,301**	**7,357,306**	**4,053,776**	**4,414,496**	**4,415,744**	**13,661,665**	**4,426,188**	**4,340,977**	**4,409,210**	**4,013,900**

Compression ratio (original ÷ compressed; higher is better):

File	gzip -9	bzip2 -9	xz -9e	zstd -19	zstd -22	lz4 -12	HCM fast	HCM max	zstd-hi	xz-hi
hdfs.log	5.35	6.83	6.80	6.72	6.72	4.20	9.17	9.27	6.72	6.84
openstack.log	11.21	13.94	15.40	14.62	14.65	8.61	19.37	19.71	14.73	15.54
apache_access.log	10.50	18.35	20.08	18.69	18.75	9.89	29.12	29.54	18.75	20.24
nginx.json	31.56	51.74	53.21	45.85	45.90	25.30	88.26	89.44	46.00	53.37
nginxplus.json	29.50	31.50	54.68	49.39	51.87	22.61	54.23	54.63	52.44	54.75
counties2023.jsonl	7.09	14.90	76.21	56.43	55.87	5.62	44.14	44.61	56.43	79.33
counties2023.csv	3.54	8.68	34.66	27.22	27.20	2.89	22.07	24.75	27.22	35.54
states.csv	5.08	5.93	10.14	8.61	8.61	3.72	10.09	10.21	8.61	10.27
counties2023_uncompressed.parquet	5.67	5.39	11.77	11.09	11.10	5.73	10.17	10.27	11.10	11.90
counties2023_snappy.parquet	1.99	2.03	2.73	2.59	2.59	1.97	2.71	2.75	2.59	2.76
counties2023_zstd.parquet	1.05	1.04	1.08	1.09	1.09	1.05	1.10	1.10	1.09	1.09
counties2023_zlib.orc	1.04	1.03	1.07	1.07	1.07	1.03	1.05	1.05	1.07	1.07
Total	**5.70**	**8.99**	**16.31**	**14.98**	**14.97**	**4.84**	**14.94**	**15.23**	**15.00**	**16.47**

18.4 Decompression speed and memory

Summary.

- zstd and LZ4 decoded the whole set fastest: 0.11 s and 0.13 s of CPU in total. xz -9e needed 0.38 s and gzip 0.38 s.
- HCM fast needed 64.8 s and HCM max 75.2 s — about 170–200× slower than xz -9e. Throughput was about 0.3–1.5 MB/s, against hundreds of MB/s for the stock tools.
- Peak decoder memory: stock tools 2–54 MB; HCM 35–65 MB (fast) and 53–115 MB (max).

Each cell: median decode wall time · throughput (original MB per second) · peak RSS.

File	gzip -9	bzip2 -9	xz -9e	zstd -19	zstd -22	lz4 -12	HCM fast	HCM max	zstd-hi	xz-hi
hdfs.log	7.0 ms · 41 MB/s · 2 MB	13.8 ms · 21 MB/s · 3 MB	8.9 ms · 32 MB/s · 2 MB	4.7 ms · 62 MB/s · 3 MB	4.3 ms · 66 MB/s · 3 MB	3.9 ms · 74 MB/s · 3 MB	308.9 ms · 1 MB/s · 35 MB	331.1 ms · 1 MB/s · 55 MB	3.8 ms · 75 MB/s · 3 MB	6.6 ms · 44 MB/s · 2 MB
openstack.log	5.3 ms · 111 MB/s · 2 MB	15.1 ms · 39 MB/s · 4 MB	7.2 ms · 83 MB/s · 3 MB	4.7 ms · 126 MB/s · 4 MB	4.5 ms · 132 MB/s · 4 MB	4.0 ms · 147 MB/s · 3 MB	509.7 ms · 1 MB/s · 40 MB	579.7 ms · 1 MB/s · 67 MB	5.4 ms · 109 MB/s · 4 MB	6.9 ms · 86 MB/s · 3 MB

File	gzip -9	bzip2 -9	xz -9e	zstd -19	zstd -22	lz4 -12	HCM fast	HCM max	zstd-hi	xz-hi
apache_access.log	13.3 ms · 178 MB/s · 2 MB	87.6 ms · 27 MB/s · 5 MB	13.9 ms · 170 MB/s · 4 MB	6.8 ms · 351 MB/s · 5 MB	5.5 ms · 432 MB/s · 5 MB	6.6 ms · 358 MB/s · 5 MB	2,152.8 ms · 1 MB/s · 43 MB	2,451.2 ms · 1 MB/s · 74 MB	7.6 ms · 310 MB/s · 5 MB	13.2 ms · 179 MB/s · 4 MB
nginx.json	44.4 ms · 274 MB/s · 2 MB	219.7 ms · 55 MB/s · 5 MB	30.3 ms · 401 MB/s · 14 MB	11.4 ms · 1,069 MB/s · 11 MB	14.8 ms · 818 MB/s · 15 MB	17.8 ms · 680 MB/s · 14 MB	8,315.5 ms · 1 MB/s · 49 MB	10,855.2 ms · 1 MB/s · 71 MB	15.8 ms · 769 MB/s · 15 MB	30.0 ms · 405 MB/s · 14 MB
nginxplus.json	12.8 ms · 218 MB/s · 2 MB	70.1 ms · 40 MB/s · 5 MB	9.6 ms · 292 MB/s · 5 MB	5.5 ms · 502 MB/s · 6 MB	6.0 ms · 468 MB/s · 6 MB	6.4 ms · 433 MB/s · 5 MB	2,420.7 ms · 1 MB/s · 36 MB	2,399.5 ms · 1 MB/s · 53 MB	9.3 ms · 299 MB/s · 6 MB	9.7 ms · 288 MB/s · 5 MB
counties2023.jsonl	143.5 ms · 189 MB/s · 2 MB	660.9 ms · 41 MB/s · 5 MB	67.1 ms · 403 MB/s · 28 MB	21.6 ms · 1,252 MB/s · 12 MB	40.2 ms · 673 MB/s · 30 MB	31.2 ms · 866 MB/s · 19 MB	22,090.2 ms · 1 MB/s · 65 MB	25,730.3 ms · 1 MB/s · 92 MB	30.5 ms · 888 MB/s · 29 MB	70.7 ms · 383 MB/s · 28 MB
counties2023.csv	91.0 ms · 127 MB/s · 2 MB	454.5 ms · 25 MB/s · 5 MB	68.8 ms · 168 MB/s · 13 MB	17.6 ms · 657 MB/s · 12 MB	18.4 ms · 628 MB/s · 15 MB	22.8 ms · 507 MB/s · 17 MB	12,423.3 ms · 1 MB/s · 50 MB	13,835.5 ms · 1 MB/s · 76 MB	18.8 ms · 613 MB/s · 12 MB	48.3 ms · 239 MB/s · 13 MB
states.csv	21.8 ms · 102 MB/s · 2 MB	73.9 ms · 30 MB/s · 5 MB	25.7 ms · 86 MB/s · 4 MB	10.3 ms · 215 MB/s · 5 MB	7.5 ms · 296 MB/s · 5 MB	9.0 ms · 248 MB/s · 5 MB	2,357.3 ms · 1 MB/s · 35 MB	2,910.9 ms · 1 MB/s · 53 MB	8.8 ms · 251 MB/s · 5 MB	34.0 ms · 65 MB/s · 4 MB
counties2023_uncompressed.parquet	19.5 ms · 186 MB/s · 2 MB	109.4 ms · 33 MB/s · 5 MB	29.8 ms · 121 MB/s · 6 MB	7.0 ms · 515 MB/s · 7 MB	9.0 ms · 403 MB/s · 7 MB	7.4 ms · 487 MB/s · 6 MB	6,323.1 ms · 1 MB/s · 65 MB	7,084.9 ms · 1 MB/s · 115 MB	7.3 ms · 494 MB/s · 7 MB	30.4 ms · 119 MB/s · 6 MB
counties2023_snappy.parquet	16.0 ms · 100 MB/s · 2 MB	83.4 ms · 19 MB/s · 5 MB	44.7 ms · 36 MB/s · 4 MB	7.4 ms · 217 MB/s · 5 MB	6.8 ms · 235 MB/s · 5 MB	5.6 ms · 284 MB/s · 4 MB	3,382.3 ms · 0 MB/s · 63 MB	4,082.2 ms · 0 MB/s · 113 MB	7.4 ms · 216 MB/s · 5 MB	59.1 ms · 27 MB/s · 4 MB
counties2023_zstd.parquet	7.8 ms · 58 MB/s · 2 MB	40.2 ms · 11 MB/s · 3 MB	27.8 ms · 16 MB/s · 3 MB	4.8 ms · 95 MB/s · 4 MB	5.0 ms · 92 MB/s · 4 MB	4.9 ms · 93 MB/s · 3 MB	1,317.0 ms · 0 MB/s · 62 MB	1,493.0 ms · 0 MB/s · 112 MB	4.7 ms · 97 MB/s · 4 MB	23.3 ms · 20 MB/s · 3 MB
counties2023_zlib.orc	13.9 ms · 104 MB/s · 2 MB	150.8 ms · 10 MB/s · 5 MB	55.2 ms · 26 MB/s · 3 MB	8.9 ms · 163 MB/s · 6 MB	8.8 ms · 165 MB/s · 6 MB	8.7 ms · 166 MB/s · 5 MB	3,852.5 ms · 0 MB/s · 48 MB	4,408.2 ms · 0 MB/s · 82 MB	6.2 ms · 233 MB/s · 6 MB	62.1 ms · 23 MB/s · 4 MB

18.5 Compression speed and memory

Summary. HCM encoding took 236 s (fast) and 280 s (max) for the whole set, against 123 s for xz -9e and 128 s for zstd -22. gzip, bzip2 and LZ4 finished in under 8 s. Peak encoder memory ranged from about 2 MB (gzip) to about 414 MB (zstd -22 on the largest file).

Each cell: encode wall time · throughput · peak RSS.

File	gzip -9	bzip2 -9	xz -9e	zstd -19	zstd -22	lz4 -12	HCM fast	HCM max	zstd-hi	xz-hi
hdfs.log	0.02 s · 14.5 MB/s · 2 MB	0.03 s · 9.3 MB/s · 4 MB	0.14 s · 2.1 MB/s · 11 MB	0.20 s · 1.4 MB/s · 12 MB	0.15 s · 1.9 MB/s · 12 MB	0.02 s · 11.7 MB/s · 2 MB	0.99 s · 0.3 MB/s · 45 MB	1.44 s · 0.2 MB/s · 79 MB	0.14 s · 2.0 MB/s · 12 MB	0.10 s · 2.7 MB/s · 11 MB
openstack.log	0.03 s · 21.7 MB/s · 2 MB	0.08 s · 7.2 MB/s · 6 MB	0.44 s · 1.4 MB/s · 22 MB	0.36 s · 1.7 MB/s · 20 MB	0.58 s · 1.0 MB/s · 20 MB	0.02 s · 28.5 MB/s · 3 MB	1.80 s · 0.3 MB/s · 45 MB	2.07 s · 0.3 MB/s · 80 MB	0.59 s · 1.0 MB/s · 19 MB	0.45 s · 1.3 MB/s · 22 MB
apache_access.log	0.05 s · 43.7 MB/s · 2 MB	0.30 s · 7.9 MB/s · 8 MB	1.39 s · 1.7 MB/s · 68 MB	1.48 s · 1.6 MB/s · 54 MB	1.60 s · 1.5 MB/s · 70 MB	0.09 s · 26.2 MB/s · 5 MB	7.51 s · 0.3 MB/s · 49 MB	9.57 s · 0.2 MB/s · 83 MB	1.85 s · 1.3 MB/s · 54 MB	1.45 s · 1.6 MB/s · 68 MB
nginx.json	0.28 s · 42.8 MB/s · 2 MB	2.34 s · 5.2 MB/s · 8 MB	16.39 s · 0.7 MB/s · 114 MB	12.82 s · 0.9 MB/s · 95 MB	19.62 s · 0.6 MB/s · 271 MB	0.44 s · 27.5 MB/s · 19 MB	30.22 s · 0.4 MB/s · 68 MB	35.72 s · 0.3 MB/s · 104 MB	14.85 s · 0.8 MB/s · 271 MB	17.89 s · 0.7 MB/s · 114 MB
nginxplus.json	0.07 s · 39.7 MB/s · 2 MB	0.36 s · 7.8 MB/s · 8 MB	6.64 s · 0.4 MB/s · 31 MB	2.82 s · 1.0 MB/s · 54 MB	11.37 s · 0.2 MB/s · 70 MB	0.07 s · 42.5 MB/s · 5 MB	7.72 s · 0.4 MB/s · 48 MB	9.36 s · 0.3 MB/s · 83 MB	11.83 s · 0.2 MB/s · 70 MB	6.96 s · 0.4 MB/s · 31 MB
counties2023.jsonl	2.61 s · 10.4 MB/s · 2 MB	3.08 s · 8.8 MB/s · 8 MB	69.75 s · 0.4 MB/s · 250 MB	34.60 s · 0.8 MB/s · 110 MB	67.11 s · 0.4 MB/s · 414 MB	3.59 s · 7.5 MB/s · 22 MB	78.42 s · 0.3 MB/s · 97 MB	93.90 s · 0.3 MB/s · 134 MB	39.61 s · 0.7 MB/s · 110 MB	72.89 s · 0.4 MB/s · 250 MB
counties2023.csv	1.54 s · 7.5 MB/s · 2 MB	1.02 s · 11.3 MB/s · 8 MB	23.63 s · 0.5 MB/s · 122 MB	20.24 s · 0.6 MB/s · 95 MB	21.54 s · 0.5 MB/s · 271 MB	1.37 s · 8.4 MB/s · 24 MB	43.93 s · 0.3 MB/s · 68 MB	51.78 s · 0.2 MB/s · 103 MB	20.50 s · 0.6 MB/s · 95 MB	22.53 s · 0.5 MB/s · 122 MB
states.csv	0.28 s · 7.8 MB/s · 2 MB	0.20 s · 11.0 MB/s · 8 MB	3.15 s · 0.7 MB/s · 27 MB	1.87 s · 1.2 MB/s · 54 MB	3.33 s · 0.7 MB/s · 70 MB	0.51 s · 4.3 MB/s · 5 MB	8.49 s · 0.3 MB/s · 48 MB	10.27 s · 0.2 MB/s · 82 MB	2.78 s · 0.8 MB/s · 54 MB	3.04 s · 0.7 MB/s · 27 MB

File	gzip -9	bzip2 -9	xz -9e	zstd -19	zstd -22	lz4 -12	HCM fast	HCM max	zstd-hi	xz-hi
counties2023_uncompressed.parquet	0.82 s · 4.4 MB/s · 2 MB	0.22 s · 16.6 MB/s · 8 MB	1.25 s · 2.9 MB/s · 98 MB	0.77 s · 4.7 MB/s · 55 MB	2.31 s · 1.6 MB/s · 71 MB	0.72 s · 5.0 MB/s · 6 MB	22.79 s · 0.2 MB/s · 69 MB	28.97 s · 0.1 MB/s · 120 MB	1.75 s · 2.1 MB/s · 55 MB	0.82 s · 4.4 MB/s · 98 MB
counties2023_snappy.parquet	0.09 s · 18.0 MB/s · 2 MB	0.16 s · 10.0 MB/s · 8 MB	0.57 s · 2.8 MB/s · 80 MB	0.49 s · 3.3 MB/s · 38 MB	0.68 s · 2.3 MB/s · 38 MB	0.05 s · 30.2 MB/s · 5 MB	14.05 s · 0.1 MB/s · 67 MB	16.27 s · 0.1 MB/s · 118 MB	0.84 s · 1.9 MB/s · 38 MB	0.60 s · 2.7 MB/s · 80 MB
counties2023_zstd.parquet	0.03 s · 17.2 MB/s · 2 MB	0.08 s · 5.9 MB/s · 5 MB	0.22 s · 2.0 MB/s · 71 MB	0.09 s · 4.8 MB/s · 12 MB	0.06 s · 7.4 MB/s · 12 MB	0.02 s · 26.9 MB/s · 3 MB	4.76 s · 0.1 MB/s · 64 MB	5.52 s · 0.1 MB/s · 114 MB	0.06 s · 8.0 MB/s · 12 MB	0.21 s · 2.1 MB/s · 71 MB
counties2023_zlib.orc	0.05 s · 27.2 MB/s · 2 MB	0.20 s · 7.3 MB/s · 8 MB	0.65 s · 2.2 MB/s · 79 MB	0.42 s · 3.4 MB/s · 38 MB	0.43 s · 3.4 MB/s · 38 MB	0.08 s · 17.8 MB/s · 5 MB	17.22 s · 0.1 MB/s · 68 MB	18.58 s · 0.1 MB/s · 119 MB	0.35 s · 4.1 MB/s · 38 MB	0.50 s · 2.9 MB/s · 79 MB

18.6 CPU and energy

Summary. At 15 W per busy core, decoding the whole set cost about 1.7 J with zstd -19, 5.8 J with xz -9e and 1,129 J with HCM max. Encoding cost 88 J (gzip) to 4,207 J (HCM max).

Method	Total encode CPU (s)	Encode energy (J)	Total decode CPU (s)	Decode energy (J)
gzip -9	5.83	87.5	0.384	5.77
bzip2 -9	7.99	119.9	1.942	29.13
xz -9e	122.72	1,840.8	0.384	5.76
zstd -19	75.43	1,131.5	0.111	1.67
zstd -22	127.52	1,912.8	0.132	1.97
lz4 -12	6.90	103.5	0.125	1.87
HCM fast	235.56	3,533.4	64.798	971.97
HCM max	280.45	4,206.7	75.247	1,128.71
zstd-hi	94.06	1,411.0	0.122	1.84
xz-hi	126.20	1,893.0	0.390	5.85

18.7 No-Pain tuning (stock decoders)

Summary. xz-hi passed the strict test on 6 of 12 files. With the base kept for the other six, the set fell 0.32% (4,053,776 → 4,040,884 bytes) and total xz decode time fell 7.4% (389 → 360 ms). The largest wins were counties2023.csv (−2.5%, decode 29.7% faster) and the zstd Parquet (−0.6%, 16.1% faster). zstd-hi passed on 1 file (counties2023.jsonl, −1.0%); with fallback the set fell 0.11%. On six files xz-hi was smaller but decoded slower, so it was rejected.

File	xz-hi vs xz -9e: bytes	size	decode	verdict	zstd-hi vs zstd -22: bytes	size	decode	verdict
hdfs.log	42,348 → 42,072	-0.7%	8.9 → 6.6 ms	No-Pain	42,811 → 42,811	+0.0%	4.3 → 3.8 ms	no gain
openstack.log	38,656 → 38,308	-0.9%	7.2 → 6.9 ms	No-Pain	40,609 → 40,397	-0.5%	4.5 → 5.4 ms	fallback
apache_access.log	118,072 → 117,124	-0.8%	13.9 → 13.2 ms	No-Pain	126,446 → 126,418	-0.0%	5.5 → 7.6 ms	fallback
nginx.json	228,132 → 227,416	-0.3%	30.3 → 30.0 ms	No-Pain	264,444 → 263,841	-0.2%	14.8 → 15.8 ms	fallback
nginxplus.json	50,932 → 50,864	-0.1%	9.6 → 9.7 ms	fallback	53,683 → 53,105	-1.1%	6.0 → 9.3 ms	fallback
counties2023.jsonl	355,052 → 341,088	-3.9%	67.1 → 70.7 ms	fallback	484,317 → 479,500	-1.0%	40.2 → 30.5 ms	No-Pain
counties2023.csv	332,984 → 324,760	-2.5%	68.8 → 48.3 ms	No-Pain	424,313 → 424,017	-0.1%	18.4 → 18.8 ms	fallback
states.csv	218,628 → 215,976	-1.2%	25.7 → 34.0 ms	fallback	257,621 → 257,621	+0.0%	7.5 → 8.8 ms	no gain
counties2023_uncompressed.parquet	307,676 → 304,512	-1.0%	29.8 → 30.4 ms	fallback	326,280 → 326,280	+0.0%	9.0 → 7.3 ms	no gain
counties2023_snappy.parquet	587,904 → 581,732	-1.0%	44.7 → 59.1 ms	fallback	620,308 → 620,308	+0.0%	6.8 → 7.4 ms	no gain
counties2023_zstd.parquet	421,612 → 419,232	-0.6%	27.8 → 23.3 ms	No-Pain	420,384 → 420,384	+0.0%	5.0 → 4.7 ms	no gain

File	xz-hi vs xz -9e: bytes	size	decode	verdict	zstd-hi vs zstd -22: bytes	size	decode	verdict
counties2023_zlib.orc	1,351,780 → 1,350,816	-0.1%	55.2 → 62.1 ms	fallback	1,354,528 → 1,354,528	+0.0%	8.8 → 6.2 ms	no gain

18.8 Record delta and the activity map

Summary. Both mutated transforms failed on this data. Record delta made output 4–157% larger on 11 files. Its only win was the zstd Parquet with xz (−0.5%, decode 13% faster). The activity map was slightly smaller on two Parquet files (−0.1% to −1.0%) but always slower to decode. The detector found a “record length” in every file, but logs, JSON and CSV have variable-length rows, so the row above is not a matching field. As in Chapter 4, the size-and-time fallback correctly rejects these cases.

File (R)	Method	Bytes	vs base	Decode	Base decode	Verdict
hdfs.log (5)	RD + zstd -19	77,860	+81.9%	4.5 ms	4.7 ms	larger
hdfs.log (5)	RD + xz -9e	70,262	+65.9%	12.6 ms	8.9 ms	larger
hdfs.log (5)	AM + gzip -9	63,719	+18.5%	9.1 ms	7.0 ms	larger
hdfs.log (5)	AM + zstd -19	53,251	+24.4%	9.1 ms	4.7 ms	larger
openstack.log (5)	RD + zstd -19	68,020	+67.1%	5.5 ms	4.7 ms	larger
openstack.log (5)	RD + xz -9e	63,730	+64.9%	11.1 ms	7.2 ms	larger
openstack.log (5)	AM + gzip -9	64,936	+22.3%	13.3 ms	5.3 ms	larger
openstack.log (5)	AM + zstd -19	52,884	+29.9%	10.1 ms	4.7 ms	larger
apache_access.log (2)	RD + zstd -19	153,839	+21.3%	15.0 ms	6.8 ms	larger
apache_access.log (2)	RD + xz -9e	145,042	+22.8%	23.3 ms	13.9 ms	larger
apache_access.log (2)	AM + gzip -9	281,566	+24.6%	27.4 ms	13.3 ms	larger
apache_access.log (2)	AM + zstd -19	179,227	+41.3%	24.3 ms	6.8 ms	larger
nginx.json (3)	RD + zstd -19	332,201	+25.5%	59.0 ms	11.4 ms	larger
nginx.json (3)	RD + xz -9e	297,126	+30.2%	67.6 ms	30.3 ms	larger
nginx.json (3)	AM + gzip -9	542,970	+41.2%	102.3 ms	44.4 ms	larger
nginx.json (3)	AM + zstd -19	401,592	+51.7%	55.2 ms	11.4 ms	larger
nginxplus.json (10)	RD + zstd -19	125,875	+123.2%	12.7 ms	5.5 ms	larger
nginxplus.json (10)	RD + xz -9e	96,694	+89.8%	20.1 ms	9.6 ms	larger
nginxplus.json (10)	AM + gzip -9	105,132	+11.4%	25.5 ms	12.8 ms	larger
nginxplus.json (10)	AM + zstd -19	66,928	+18.7%	18.3 ms	5.5 ms	larger
counties2023.jsonl (2)	RD + zstd -19	812,299	+69.4%	86.1 ms	21.6 ms	larger
counties2023.jsonl (2)	RD + xz -9e	490,014	+38.0%	211.9 ms	67.1 ms	larger
counties2023.jsonl (2)	AM + gzip -9	4,440,596	+16.3%	275.5 ms	143.5 ms	larger
counties2023.jsonl (2)	AM + zstd -19	719,270	+50.0%	118.0 ms	21.6 ms	larger
counties2023.csv (3)	RD + zstd -19	1,090,188	+157.1%	45.2 ms	17.6 ms	larger
counties2023.csv (3)	RD + xz -9e	563,438	+69.2%	180.7 ms	68.8 ms	larger
counties2023.csv (3)	AM + gzip -9	3,696,583	+13.3%	187.1 ms	91.0 ms	larger
counties2023.csv (3)	AM + zstd -19	944,075	+122.7%	65.4 ms	17.6 ms	larger
states.csv (2)	RD + zstd -19	424,177	+64.6%	14.9 ms	10.3 ms	larger
states.csv (2)	RD + xz -9e	365,402	+67.1%	59.1 ms	25.7 ms	larger
states.csv (2)	AM + gzip -9	514,524	+17.9%	31.5 ms	21.8 ms	larger
states.csv (2)	AM + zstd -19	396,669	+54.0%	23.3 ms	10.3 ms	larger
counties2023_uncompressed.parquet (11)	RD + zstd -19	554,689	+69.9%	12.1 ms	7.0 ms	larger
counties2023_uncompressed.parquet (11)	RD + xz -9e	468,214	+52.2%	64.3 ms	29.8 ms	larger
counties2023_uncompressed.parquet (11)	AM + gzip -9	632,255	-1.0%	33.7 ms	19.5 ms	smaller, slower
counties2023_uncompressed.parquet (11)	AM + zstd -19	381,902	+17.0%	20.7 ms	7.0 ms	larger
counties2023_snappy.parquet (11)	RD + zstd -19	887,780	+43.1%	10.0 ms	7.4 ms	larger
counties2023_snappy.parquet (11)	RD + xz -9e	869,698	+47.9%	83.5 ms	44.7 ms	larger
counties2023_snappy.parquet (11)	AM + gzip -9	836,134	+3.6%	28.0 ms	16.0 ms	larger
counties2023_snappy.parquet (11)	AM + zstd -19	658,420	+6.1%	23.1 ms	7.4 ms	larger

File (R)	Method	Bytes	vs base	Decode	Base decode	Verdict
counties2023_zstd.parquet (3)	RD + zstd -19	416,329	-1.0%	7.6 ms	4.8 ms	smaller, slower
counties2023_zstd.parquet (3)	RD + xz -9e	419,462	-0.5%	24.3 ms	27.8 ms	No-Pain
counties2023_zstd.parquet (3)	AM + gzip -9	431,013	-0.5%	12.7 ms	7.8 ms	smaller, slower
counties2023_zstd.parquet (3)	AM + zstd -19	419,829	-0.1%	11.0 ms	4.8 ms	smaller, slower
counties2023_zlib.orc (11)	RD + zstd -19	1,412,322	+4.3%	7.5 ms	8.9 ms	larger
counties2023_zlib.orc (11)	RD + xz -9e	1,417,446	+4.9%	43.9 ms	55.2 ms	larger
counties2023_zlib.orc (11)	AM + gzip -9	1,396,087	+0.4%	26.8 ms	13.9 ms	larger
counties2023_zlib.orc (11)	AM + zstd -19	1,361,123	+0.5%	14.7 ms	8.9 ms	larger

18.9 Findings

1. **HCM's context models suit text logs.** On logs up to 12 MB, HCM was 22–41% smaller than xz -9e or bzip2. It is far too slow to decode for live log search, but it may suit cold log archives.
2. **HCM degrades on large repetitive exports.** On counties2023.jsonl (27 MB) it was 71% larger than xz -9e, the same saturation seen on enwik8 (Chapter 13). Larger hash tables or a long-range match model are the obvious fix.
3. **xz-hi is the only safe gain** here with stock decoders: small but reliable.
4. **Record methods need fixed-size records.** Text records do not qualify; the predictors of Section 23.3 (binary flag, strong spacing peak) would have screened these files out.
5. **Columnar files are already compressed.** Recompressing Parquet and ORC gains little; the opportunity is in the column layout before encoding (Chapter 17), not afterwards.

18.10 Failures and caveats

- No run failed verification.
- Timing is from one VM; small-file decode times (under 10 ms) include process start-up.
- The Parquet and ORC files were generated here, not taken from production systems.

Chapter 19 — Energy and Cost Model

> **Executive summary.** Compression is a trade: CPU time and electricity spent decoding are bought back by smaller storage and cheaper network transfer. This chapter prices that trade with public cloud rates. For data sent to many customers, a slower but stronger compressor can be the cheapest choice overall, because transfer costs dominate. For data processed on rented servers, decode speed matters more. Results that are both smaller and faster, the No-Pain results, are cheaper under every assumption.

19.1 Model

Parameter	Value
Network transfer	$0.09 per GB
SSD storage	$105 per TB
CPU power per core	15 W
Electricity	18.3 cents per kWh
Rented compute	AWS Fargate CPU and memory rates

Energy per decode:

- $E = P \times t$, with $P = 15$ W per core and t the measured single-thread decode time.
- 1 ms of decoding costs 15 mJ.
- 1 kWh = 3.6×10^6 J, so 1 J costs about 5.1×10^{-6} cents.

19.2 Energy per decode — kennedy.xls

Configuration	Size (bytes)	Decode	Energy
zstd -22 + record delta	26,401	520 µs	7.8 mJ
LZ4 -12 + record delta	43,653	725 µs	10.9 mJ
LZ4 -12	324,136	770 µs	11.6 mJ
gzip -9 + record delta	42,971	1,054 µs	15.8 mJ
zstd -22	64,818	1,359 µs	20.4 mJ
gzip -9 + Combary activity map	35,558	≈1,400 µs	≈21 mJ
xz-hi	23,656	≈2.0 ms	≈30 mJ
gzip -9	209,721	2,179 µs	32.7 mJ
xz -9e	51,868	6.9 ms	104 mJ
bzip2-hi	120,482	11.9 ms	179 mJ
bzip2 -9	130,280	15.8 ms	237 mJ
HCM v3.3 fast	18,680	≈700 ms	≈10.5 J
HCM-Bin v1	16,763	790 ms	11.9 J
ZPAQ -m5	16,822	6.35 s	95 J

Each record-delta pipeline uses *less* energy than its unmodified compressor (zstd 20.4 → 7.8 mJ, gzip 32.7 → 15.8 mJ). HCM-Bin uses one eighth of ZPAQ -m5's energy for a smaller file.

19.3 Cost per terabyte — 8 Canterbury text files (HCM v3 era)

Compressor	Transfer + decode electricity	Full cost, including rented hardware and disk
HCM v3 fast	**$23.35**	$64.88
bzip2 -9	$25.65	**$56.33**
ZPAQ -m5	$28.09	—
xz -9e	$28.71	$62.58
HCM v2	$26.92	$121.74

When the reader pays only for transfer and electricity — the typical *distribution* case — HCM is cheapest, because its smaller output saves more in transfer than its decode energy costs. When the reader also rents the CPU time, as in the *server-side processing* case, bzip2's speed wins. The model makes this crossover explicit rather than assuming one compressor is "best".

19.4 Cost per million customers — Canterbury binaries

Decode energy, transfer and storage for kennedy.xls, ptt5 and sum, delivered to one million customers:

Compressor	Cost
HCM-Bin v1	**$11.39**
ZPAQ -m5	$18.44
xz -9e	$19.76

19.5 Reading the model

1. **No-Pain results are cheaper on the reading side.** They reduce both bytes and decode time, so transfer, storage and decode costs fall under every price assumption. The encoder-side search costs extra time, energy and memory once per file; it pays off when files are read many times, but not necessarily for data written once and rarely read.
2. **Distribution favours ratio.** When a file is decoded once by each of many recipients, transfer dominates, and strong compressors win even when slow.
3. **Processing favours speed.** When a file is decoded repeatedly on rented machines, CPU time dominates, and Tier 1–3 tools win.
4. **Decoder memory** carries cost in shared environments. It has been recorded since 27 October 2026 and will be priced in the next revision of the model.

Chapter 20 — Economic Potential and Commercial Applications

> **Executive summary.** This chapter estimates what the methods in this book could be worth if they were used at scale. It keeps three kinds of figure strictly apart. **Demonstrated** figures are measurements from this book. **Projected** figures apply a measured percentage to an assumed data volume at published cloud prices. **Speculative** figures also assume that a company's data resembles the test files, which has not been checked. No company has adopted any of this work, and nothing here should be read as saying one has. The main lesson is that savings scale with volume. A 1–3% gain on a stock format is worth little to one user and a great deal to a provider holding exabytes, and it needs no change on the reading side.

20.1 How to read the numbers

Label	Meaning	Example
Demonstrated	Measured in this book, on the stated files, with exact round trips	7z-hi: Silesia 48,757,480 → ≈47,333,100 bytes (−2.9%), decode 5.3% faster
Projected	A demonstrated percentage multiplied by an assumed volume and a published price	2.9% of 1 PB in object storage at $0.023 per GB-month
Speculative	A projection that also assumes the customer's data behaves like the test data	Assuming a legacy spreadsheet archive responds like kennedy.xls

Prices used (US list prices, accessed October 2026; real contracts are usually lower):

- object storage (Amazon S3 Standard, first 50 TB tier): **$0.023 per GB-month**;
- internet data transfer out (AWS, first 10 TB tier): **$0.09 per GB**;
- Snowflake storage, billed on compressed size: **about $23–40 per TB-month** (capacity vs on-demand);
- electricity: 18.3 cents per kWh, and 15 W per busy core (Chapter 19).

1 PB is taken as 1,000,000 GB. Every projection is linear in its assumptions, so readers can rescale it to their own volumes and prices.

20.2 Demonstrated gains that are cheapest to adopt

Summary. The cheapest gains to adopt are those whose output opens with the standard, unmodified reader, because nothing changes on the reading side. They range from 0.02% to 6% on whole corpora, with decoding not slower.

Method	Format	Decoder	Demonstrated result	Decode	Chapter
7z-hi	.7z	stock 7-Zip	Silesia −2.9%; Canterbury −6.1%	5–8% faster	5
lzma-hi	LZMA	stock	Silesia −1.2%	3.9% faster	5
xz-hi	.xz	stock xz	Canterbury −6.1% (mostly kennedy.xls)	faster in first session; unconfirmed	5
bzip2-hi	.bz2	stock bzip2	Canterbury −2.0%	7% faster	5
zstd-hi	.zst	stock zstd	Canterbury −0.04% to −1.1% by level	not slower	5
Brotli-hi	Brotli	stock	Canterbury −0.9% at quality 11	not slower	5
Detector-chosen PDF row filters	PDF	standard readers (poppler tested)	−0.6% vs qpdf on 139 PDFs	not slower to render	6
FLAC block-size choice	FLAC	stock flac	speech −2.1%	10% faster	16
WebP lossless search	WebP	stock dwebp	Kodak −0.018%	not slower	16

20.3 Demonstrated gains that need a small decoder change

Summary. On data built from fixed-size records, record delta and the activity map cut compressed size by more than half and make decoding faster. They need only a short inverse step after the standard decoder. The gains are large but confined to that kind of data.

Method	Example	Demonstrated result	Decode
Record delta + zstd	kennedy.xls	64,818 → 26,401 bytes (−59.3%)	2.6× faster
Record delta + gzip	kennedy.xls	209,721 → 42,971 bytes (−79.5%)	2.1× faster
Activity map + gzip	kennedy.xls	209,721 → 35,558 bytes (−83.0%)	1.5× faster
Record delta + zstd -1	Silesia x-ray (medical image)	6,772,118 → 5,025,007 bytes (−25.8%)	17% faster
Detected columns in ORC (zlib)	Silesia x-ray	6,168,696 → 4,695,458 bytes (−23.9%)	1.8× faster to read

20.4 Application areas

Legacy spreadsheets and fixed-record exports

Old binary spreadsheets (.xls), instrument dumps, mainframe exports and fixed-width record files are what record detection was designed for. kennedy.xls is the strongest result in the book. It is still one file, and other spreadsheets were not tested.

- **Demonstrated:** −59% with zstd at 2.6× faster decoding; −83% with gzip plus the activity map.
- **Speculative:** suppose an organisation keeps a 100 TB archive of zstd-compressed legacy exports, and half of it behaves like kennedy.xls. Record delta would remove about 30 TB (100 TB × 50% × 59%). In object storage that is about $690 per month, or $8,300 per year. A sample of the real archive is needed to test the assumption; the predictors of Section 23.3 are a cheap way to screen it.

Cloud object storage (AWS, Google Cloud, Azure)

Providers and their customers store data already compressed with gzip, zstd, xz or 7-Zip. Encoder-side tuning changes only how files are written.

- **Projected:** 1 PB of 7-Zip archives costs about $23,000 per month in S3 Standard. A 2.9% reduction saves about $670 per month, or $8,000 per year. At 1 EB (1,000 PB) the same percentage is about $8 million per year. The extra encoding search is a one-time CPU cost per file, which these figures do not subtract.
- **Projected (transfer):** delivering 1 PB per month to the internet at the first-tier rate of $0.09 per GB costs about $90,000. A 1% smaller file saves about $900 per month. Large customers pay lower volume rates, which reduces this proportionally.

Data warehouses (Snowflake, Databricks) and columnar formats (Parquet, ORC)

Snowflake bills storage on compressed size, so any reduction in compressed bytes is a direct saving.

- **Demonstrated:** the detected-column layout helped four of seven unknown binary files imported into Parquet and ORC, by 17–84%. The baseline was a one-column byte table, which no one would use in practice (Chapter 17).
- **Not yet demonstrated:** whether a detected layout beats a hand-designed schema on real warehouse data. Chapter 18 tests real CSV, JSON and Parquet exports with general-purpose methods. A warehouse saving should not be claimed until such a comparison exists.
- **Projected illustration:** a 500 TB warehouse at $23 per TB-month costs $11,500 per month. Each 1% of compressed size saved is about $115 per month, or $1,380 per year.

Content delivery and browsers (Cloudflare and others)

Web content is sent with gzip, Brotli or zstd. Browsers already decode these, so encoder tuning needs no browser change.

- **Demonstrated:** zstd-hi and Brotli-hi gains on Canterbury are 0.04–1.1%, with decoding not slower.
- **Projected:** for static assets compressed once and served many times, a 0.5% smaller response saves 0.5% of the transfer for those assets. On 1 PB per month at $0.09 per GB, that is about $450 per month. The gain on real web assets has not been measured.

Archiving tools and backup

Archivers (7-Zip, xz, bzip2, RAR) write data once and may read it rarely. The encoder search costs more CPU once, then saves storage for the life of the archive. For data written once and almost never read, decode speed matters less and storage dominates.

Documents (PDF)

- **Demonstrated:** −0.6% against qpdf on 139 PDFs, render time not slower. The test set is a reader’s bug collection, not an office archive, and only one reader was tested (Chapter 6).

- **Projected:** for a document repository already optimised with qpdf, 0.6% of 1 PB is 6 TB, about $140 per month in object storage. The larger saving, 15% against unoptimised originals, is mostly qpdf's known work and is not claimed.

Audio (FLAC)

- **Demonstrated:** −2.1% on six speech recordings, decoding 10% faster. Music was not tested.
- **Projected:** a 1 PB speech archive would shrink by about 21 TB, about $480 per month in object storage, if real recordings behaved like these.

Medical imaging

- **Demonstrated:** with the stock JPEG XL decoder, some radiographs decode up to 7.5× faster for 3–5% more bytes, and four images became both smaller and faster (Chapter 16).
- **Projected:** where images are read many times, as in clinical viewing, faster decoding can matter more than size. The book has no data on reading patterns in real hospital systems.

20.5 Energy

Summary. The energy saved by faster decoding is real but small per file. It becomes significant only at very large read volumes.

- **Demonstrated:** record delta cut zstd's decode of kennedy.xls from 1,359 to 520 µs, saving about 12.6 mJ per decode at 15 W.
- **Projected:** one billion such decodes save about 12.6 MJ, which is 3.5 kWh, or about 64 cents of electricity. Even at that scale, the money is in bytes stored and transferred, not in decode electricity. Decode speed matters most where it sets latency or the number of servers needed, which this book does not model.

20.6 Licensing and commercial use

The software in this book is released under the **MIT License** (Chapter 24). In plain terms:

- anyone may use, copy, modify, merge, publish, distribute, sublicense and sell the software, including commercially, without paying a fee;
- the copyright notice and the licence text must be kept in all copies or substantial portions of the software;
- the licence does **not** require prominent attribution in a product's interface or marketing. The author would value recognition, such as a credit naming Michael Harrington or the Harrington methods, but this is a request, not a licence condition;
- the software is provided "as is", without warranty.

The MIT License covers the **software**. It is separate from the author's **patents**. The author was granted five U.S. patents, four of which apply to Combary in data storage, transmission and encryption; all but one (U.S. 11,636,878) have expired (Section 24.4). No patents are pending on the compression methods in this book. Anyone planning a commercial product that uses Combary for storage, transmission or encryption should check U.S. 11,636,878 itself, and should not assume that the MIT software licence grants patent rights beyond what the licence text says. This is not legal advice.

20.7 What would turn projections into demonstrations

1. **Real customer data.** Run the predictors (Section 23.3) and the No-Pain search on samples of real archives, warehouses and web assets.
2. **Production timing.** Re-time on several CPUs and with multi-threaded decoders.
3. **Reader compatibility.** Test PDF output in more readers, and record-delta pipelines in real ingestion systems.
4. **Encoding cost.** Price the extra encoder search, which this chapter leaves out.
5. **Independent replication.** Have a third party reproduce the demonstrated results.

Until then, the demonstrated figures in Sections 20.2 and 20.3 are the only ones this book stands behind.

Sources for prices: Upstash, "AWS S3 Pricing Explained (2026)", https://upstash.com/blog/aws-s3-pricing-explained-2026; Espresso AI, "Snowflake Storage Pricing: On-Demand vs. Capacity", https://espresso.ai/guides/snowflake-storage-pricing. Prices change; check current list prices before relying on any figure.

Chapter 21 — HaBiTS and Binary–Ternary Conversion

> **Executive summary.** HaBiTS is the author's lossless method for converting binary data into ternary digits and back, built on Combary and designed to work on live streams without a header. As a front end to compression it failed, and the reasons it failed shaped everything that later succeeded. Compressors find patterns at byte boundaries; HaBiTS redraws those boundaries, so the patterns disappear. This chapter records the complete negative evidence, because it is the clearest demonstration in the research of what a transform must *not* do.

21.1 The method

HaBiTS ("Harrington Binary-Ternary System") maps a binary stream to a stream of trits and back with no loss. Its properties as designed by the author:

- **Variable-length, Huffman-style code.** Groups of input bits are assigned trit codewords of varying length, chosen so that the mapping is prefix-free and exactly invertible.
- **Headerless streaming.** The code table is fixed by the method, not by the data, so a stream can be converted as it arrives without first transmitting a table.
- **Combary-compatible.** The 2 symbol is available to carry positional meaning, which is the basis of later Combary mechanisms (Chapter 7).

Conventional ternary conversion, by contrast, writes each byte as six base-3 digits ($3^6 = 729 \geq 256$). It is fixed-length and aligned to bytes, but it is not compact: six trits carry $\log_2 729 \approx 9.51$ bits of capacity for 8 bits of input, so about 15.9% of the representable capacity goes unused.

21.2 The original goal

The *Harrington Compression and Reordering Suite* aimed to compress data in ternary form: convert with HaBiTS, then model and code the trits. Early prototypes used HaBiTS as the final packing stage after Huffman coding. The table below lists those routes alongside binary routes measured on the same 200 KB English text file (Python documentation).

Route (early HCM, October 2026)	1,000,000 bits down to (normalised)	Encode time
BWT > MTF > zero runs > Huffman > **HaBiTS 5-trit**	217,000	0.8 s
BWT > MTF > zero runs > Huffman (no HaBiTS)	223,900	0.5 s
per-context Huffman (4-byte) > **HaBiTS 5-trit**	237,400	~3 s
per-context Huffman (3-byte) > **HaBiTS 5-trit**	243,800	~2 s
per-context Huffman (2-byte) > **HaBiTS 5-trit**	277,900	~1 s
BWT > MTF > zero runs > context arithmetic coder	176,900	0.9 s
Context-balance coder, binary (precursor of HCM v2)	148,600	27 s
Reference: ZPAQ -m5 / bzip2 / xz	140,500 / 168,900 / 185,200	—

As a final packing stage, HaBiTS was competitive: 217,000 against 223,900 for the same Huffman route without it. It could not, however, compete with arithmetic coding, which already reaches the entropy that ternary packing tries to approach.

21.3 HaBiTS as a modelling alphabet

The decisive experiment fed the *same* adaptive context-mixing engine with four representations of the 11 Canterbury files and compared the totals.

Summary. Fed to the same model, bytes gave the smallest Canterbury total (437,409 bytes). Conventional ternary digits were 0.5% larger, a plain bit stream 14.4% larger, and HaBiTS trits 26.2% larger.

Representation fed to the model	Canterbury total (bytes)
Bytes (model predicts 8 bits per byte along a byte tree)	**437,409**
Conventional ternary (6 base-3 digits per byte)	439,688
Plain bit stream (no byte alignment)	500,474
HaBiTS trits	552,136

Byte-aligned conventional ternary is almost as good as bytes: about 1–1.5% worse on text and about 0.5% *better* on kennedy.xls and ptt5. HaBiTS, whose variable-length codewords drift across byte boundaries, is 26% worse. Bit-stream modelling without alignment falls in between.

21.4 HaBiTS as a transform in front of HCM

Summary. Converting the data to HaBiTS before HCM made the Canterbury total 54% to 67% larger than compressing it directly (414,602 bytes).

Pipeline into HCM v3 fast (Canterbury, 11 files)	Total bytes
Direct (no transform)	**414,602**
HaBiTS trits, packed base-3	638,290
Conventional trits re-encoded as HaBiTS bits	693,021

Both conversions are exactly lossless, and both are catastrophic for compression.

21.5 Variants explored inside the ternary model

Variant (conventional 6-trit coding inside HCM)	Effect
Swapping trit values (which digit is “2”)	Up to about 3% change, file-dependent
XOR 255 before base-3 conversion	Helped binary files
Frequency-sorted or Huffman-shaped trit trees	Much worse: about 161,000 against 148,227
Hybrid tree: top 4 bits binary, low nibble ternary	Under 0.1% change on text

The swap and XOR observations were carried forward into hcm-trit v1 (Chapter 11), which tries five swap patterns per file and keeps the smallest.

21.6 HaBiTS and ternary forms in front of standard compressors

Ternary forms were also tried in front of zstd -22, LZ4 -12 and gzip -9 on Canterbury: trits as bytes, trit planes packed five per byte, and trit planes unpacked, each with and without XOR 255. **No variant improved any file with any of the three compressors.**

21.7 Why it failed — the lesson

Every practical compressor learns from **aligned symbols**. LZ matchers look for repeated byte strings, context models condition on previous bytes, and BWT sorts byte contexts. A transform keeps those structures only if it maps aligned units to aligned units.

- **Conventional ternary** maps each byte to exactly six trits, so repetition in the input remains repetition in the output, and the model still sees whole symbols. It lost almost nothing.
- **HaBiTS** maps variable-length bit groups to variable-length trit groups. The same byte produces different codewords depending on what precedes it, so repeated strings no longer look repeated, and a model’s contexts dissolve. The author’s earlier observation — that HaBiTS “ruined standard compressors’ ability to detect and predict patterns” — is confirmed quantitatively here.

This lesson set the design rule for every later transform: **preserve alignment, and change only *what* is stored, never *where* the symbol boundaries fall.** Record delta, the activity map and the ternary pointer all obey it, and all succeeded. The Harrington Exchange’s nibble juggle (Chapter 15) obeys it partially, and succeeded partially.

21.8 Where HaBiTS remains relevant

HaBiTS keeps its value in the setting it was designed for: lossless, headerless conversion of a binary stream into a ternary medium. That covers ternary signalling, Combary-style storage in which the trits are physical, and encryption built on the Combary method. In those uses its job is faithful conversion, not compression, and the findings of this chapter do not count against it.

Chapter 22 — Negative Results

> **Executive summary.** Most ideas tested in this research did not work. They are listed here in full for two reasons. Other researchers can avoid repeating them, and the pattern of failures is itself evidence: nearly every failure either broke the alignment of the data or added a decoding pass slower than the compressor it was meant to help.

22.1 Transforms that broke alignment

Idea	Result	Chapter
HaBiTS trits as the modelling alphabet	552,136 vs 437,409 bytes for the Canterbury total (26% worse)	21
HaBiTS or conventional trits in front of HCM	638,290 / 693,021 vs 414,602 bytes (54–67% worse)	21
Trit bytes and trit planes (with and without XOR 255) in front of zstd, LZ4 and gzip	Worse on every file	21
Bit-plane splitting in front of zstd, LZ4 and gzip	No gain on any file	15
Frequency-sorted or Huffman-shaped trit trees in HCM	≈452,500 vs 416,635 bytes (≈9% worse)	21

22.2 Swaps and headers

Idea	Result	Chapter
Quad Exchange nibble permutation alone	Never helped; costs its 5-byte header	15
8-bit Harrington Exchange + juggle, outside record data	Lost on 9 of 11 Canterbury files, mostly to table cost	15
Partial swap tables (top *x*) before compressors	Only gzip on ptt5 gained (−101 bytes)	14
Static English-frequency Huffman table (no header)	Never beat a per-file table	14
Harrington header on binaries using all 256 values	Larger than raw code lengths	14
Compressing a ZPAQ model header separately	322 → ≈230 bytes, but stock ZPAQ cannot read the result	14

22.3 Harrington Exchange and Quad Exchange outside fixed-record data

Applied to every Canterbury file with no per-file fallback, the 8-bit Harrington Exchange made 9 of 11 files larger; the corpus total still fell, because of the large gains on kennedy.xls (and a small one on ptt5 with LZ4). The Quad Exchange nibble permutation on its own made every file very slightly larger, by its 5-byte header plus a few bytes. A permutation preserves every repeated string, so dictionary coders find the same matches, and order-0 entropy coders are indifferent to symbol labels. The success case is in Chapter 15.

Table 22.A — 8-bit Harrington Exchange + juggle on Canterbury (compressed bytes; best of j = 1–4 and stream layouts; 256-byte table included; no per-file fallback)

File	Original	zstd -22	+ transform	LZ4 -12	+ transform	gzip -9	+ transform
alice29.txt	152,089	49,212	49,452	62,973	63,229	54,179	54,427
asyoulik.txt	125,179	45,138	45,343	58,329	58,585	48,816	49,061
cp.html	24,603	7,716	7,948	10,311	10,567	7,973	8,214
fields.c	11,150	3,017	3,270	4,221	4,477	3,127	3,386
grammar.lsp	3,721	1,214	1,468	1,737	1,993	1,234	1,483
kennedy.xls	1,029,744	64,815	42,858	324,136	139,394	209,721	140,230
lcet10.txt	426,754	121,337	121,584	164,145	164,401	144,418	144,438
plrabn12.txt	481,861	167,439	167,683	224,675	224,931	194,264	194,401
ptt5	513,216	43,548	43,759	66,160	63,711	52,377	52,387
sum	38,240	10,975	11,166	16,309	16,565	12,768	13,021
xargs.1	4,227	1,729	1,982	2,423	2,679	1,748	1,996
Total	**2,810,784**	**516,140**	**496,513**	**935,419**	**750,532**	**730,625**	**663,044**

Totals change: zstd -3.8%, LZ4 -19.8%, gzip -9.2%.

Table 22.B — Quad Exchange (nibble permutation) on Canterbury (compressed bytes; 5-byte header included)

File	Original	zstd -22	+ transform	LZ4 -12	+ transform	gzip -9	+ transform
alice29.txt	152,089	49,212	49,248	62,973	62,978	54,179	54,187
asyoulik.txt	125,179	45,138	45,157	58,329	58,334	48,816	48,825
cp.html	24,603	7,716	7,721	10,311	10,316	7,973	7,984
fields.c	11,150	3,017	3,040	4,221	4,226	3,127	3,137
grammar.lsp	3,721	1,214	1,233	1,737	1,742	1,234	1,244
kennedy.xls	1,029,744	64,815	64,832	324,136	324,141	209,721	209,786
lcet10.txt	426,754	121,337	121,417	164,145	164,150	144,418	144,432
plrabn12.txt	481,861	167,439	167,506	224,675	224,680	194,264	194,285
ptt5	513,216	43,548	43,594	66,160	66,165	52,377	52,378
sum	38,240	10,975	10,978	16,309	16,314	12,768	12,778
xargs.1	4,227	1,729	1,746	2,423	2,428	1,748	1,757
Total	**2,810,784**	**516,140**	**516,472**	**935,419**	**935,474**	**730,625**	**730,793**

Totals change: zstd +0.1%, LZ4 +0.0%, gzip +0.0%.

22.4 Combary roles that failed

Idea	Result	Chapter
2 as segment cut marker (per-block transform choice)	Worse than one choice per file for all three compressors at block sizes of 4, 16 and 64 KB	7
13-bit per-record activity masks	Worse than packed flags	8
Frequency-ranked activity-mask codes	Worse than packed flags	8
Activity map in front of HCM	5–35% larger (faster decode only)	8

22.5 Decode-time failures

Idea	Size	Decode	Chapter
Activity map + LZ4 / Snappy / LZFSE	Much smaller	Slower than the plain decoder	8
Record delta + LZ4 with a scalar inverse loop	Smaller	Slower; fixed by the SIMD inverse	4
Record delta + LZ4 at fast levels (Silesia)	Some smaller	Never passed	4
Simple ternary pointer (fixed escape table, matches ≥ 16)	418,135 vs 414,602 bytes	—	11
Ternary pointer without exclusion	449,073 vs 414,602 bytes	—	11
Lazy second-nibble trick	427,846 vs 414,602 bytes	—	11
HCM loop rewritten to fixed size	Fewer instructions	Slower on the test CPU	13
Column-selected mixer weights (HCM-Bin)	Worse	—	12

22.6 Tools where little or nothing was gained

Tool	Outcome	Reason
RAR 7.30	Only RAR's own switches helped, except sao (filter disabled) and mr	Closed source; it already detects tables internally
Snappy, LZFSE	Only kennedy.xls passed	Their speed leaves no room for a reconstruction pass
WebP lossless	11,204,040 → 11,202,034 bytes (−0.018%)	Its maximum preset is already near optimal
PNG	Gains reproduce optipng	Known work
gzip (encoder tuning)	Gains only through zopfli	Known work; excluded
zstd, LZ4 (level search)	Nothing beyond small zstd-hi gains	Encoders already near their format limits
FLAC	Small; speech only	Block size the main lever
JPEG XL on 693, MR-SIEMENS, ALOKA, eCT	No candidate passed re-timing	Small images, noisy timing
Fax / bilevel pages	No speed win of ours; HCM smallest or second but 5–60× slower than fax coders	Fax coders already use the row above and pixel templates (Section 16.8)
LZ4 (Tier 1) with record delta or the activity map	Never passes	LZ4 decodes near memory speed; a separate reverse pass always costs more

Tool	Outcome	Reason
Harrington Exchange on text, database files and real images	Loses, or split-only / byte shuffle wins	Byte-aligned or numerically local data; renaming disrupts it (Section 15.9)
WinZip comparison	Not attempted	Closed source, Windows only
Harrington Exchange on flight telemetry (IRIG-106, PX4, MAVLink, ArduPilot), whole logs and extracted raw channels	Lost on all 11 files; Blosc2 or xz won in the practical tiers	Telemetry is smooth and correlated; renaming destroys sample-to-sample similarity (Section 15.15)

22.7 Results withdrawn after scrutiny

Claim	Why withdrawn
Ternary pointer: a “yes” saves a byte’s full cost (§7.4)	Independent control (Goodwin; re-run in §11.8): the flag alone saves nothing; the gain is the refinement mixer (secondary estimation)
HCM v2 beating ZPAQ on 6 of 11	The ZPAQ archive stored a long path (≈100 bytes); a fair run gave 5 of 11
zstd-hi and Brotli-hi decode speed-ups	Artefacts of parallel timing
jxl-hi wins at effort 3–5	Changing effort is a stock level, not an improvement under the baseline rule
zopfli gzip gains	Known third-party work
x-ray with RAR	Gain within command-line timing noise on re-run
First PDF run (filled-background.pdf 143,065 → 923 bytes)	JSON round-trip rounded a 100,001-digit number; fixed and re-run (Chapter 6)
JPEG XL No-Pain award for JPEG2000_UNC	Decode not faster on interleaved re-timing (21.7 → 23.8 ms, faster in 9 of 21); size gain kept, award withdrawn (Appendix F)
FLAC speed win on theo	Decode not faster on re-timing (36.1 → 37.0 ms, faster in 7 of 21); four FLAC wins, not five
x-ray record delta + zstd -19 as No-Pain with the original inverse	Decode 4–15% slower on re-test; restored only with the prefix-sum inverse (Section 4.3a)
JPEG-LS “bytes” figures	Were 1,000,000-bit-normalised values; corrected in Section 16.2 (result unchanged)

22.8 The common thread

The successful methods — record delta, the activity map with gzip, the ternary pointer, the HCM record contexts and the No-Pain encoder searches — share two properties:

1. They keep symbol boundaries intact.
2. Their decode-side cost is a single cheap pass, or nothing at all.

Every failure above violates at least one of these.

Chapter 23 — What Generalizes, and Open Questions

Executive summary. Three findings held broadly in these experiments and are likely, though not proven, to hold on other data. First, structured binary data — tables, instrument output, medical images — benefits from automatic record detection with most compressors tested; RAR, which already detects tables itself, is the exception. Second, encoder-side searches that keep the standard format can make widely used tools smaller and faster without any change to reader software, provided decode time is checked per file. Third, HCM's ternary pointer improved every file in every HCM configuration tested; whether it helps other context-mixing compressors has not yet been tested. Other results are specific to particular files, and are labelled as such.

23.1 General findings

Finding	Evidence	Coverage
Record detection + delta (or the equivalent built-in filter) improves fixed-record data with the LZ-family compressors tested, except RAR (which applies its own delta)	kennedy.xls, mr and x-ray with gzip, zstd, LZ4, Snappy, LZFSE, xz and 7-Zip; the same files in Parquet and ORC	Class (fixed-record binary)
Detector-chosen PDF row filters beat qpdf with no reader-side change	139 PDFs: qpdf 81,681,005 → 81,173,302 bytes (−0.6%), render time not slower (Chapter 6)	General (pdf.js test set)
No-Pain encoder search improves stock formats with no reader-software change	lzma-hi and 7z-hi: every file smaller on Canterbury and Silesia, none meaningfully slower; bzip2-hi: every Canterbury file smaller and faster; xz-hi: General on Canterbury only with kennedy.xls included (without it, 10.5% slower)	General (per tool as stated)
The ternary pointer improves HCM on every file	Canterbury, all 11 files; Silesia, all 12; also on a BWT back end	General within HCM; untested in other context-mixing compressors
Integer-only context mixing with cache-line nibble blocks gives large speed-ups at no size cost	HCM v3: about 4× faster than v2	General (implementation)
Transforms must preserve symbol alignment	Every HaBiTS, trit and bit-plane failure (Chapter 22)	General (design rule)

23.2 File-specific results

Result	Why it may not generalize
kennedy.xls gains of 2–8×	An extreme, highly regular 13-byte record file; reported with and without it throughout
sao: disabling RAR's own filter	One file on which RAR's detector misfires
ooffice: x86 filters	Applies to x86 executables only
Medical-image jxl-hi settings	Chosen per image; sample screening needs representative tiles

23.3 Predictors of success

Before compressing an unknown file, the following signals predict whether the record methods of this book will help:

1. A detected record length R with a strong peak in the byte-spacing histogram.
2. A low fraction of "active" bytes in the activity map, meaning most bytes repeat the row above.
3. A binary file flag: more than 10% control bytes.
4. Absence of x86 or other machine-code signatures, for which branch filters work better instead.

A fast pre-pass that computes these four signals, followed by the size-and-time fallback, is the recommended deployment pattern.

23.4 Open questions and future work

There is considerable work still to be done, and many of the open questions presented here offer promising opportunities for further research. We intend to pursue these investigations as time and resources permit. At present, Michael Harrington is also focused on launching a business involving an innovative cold-pack technology, which necessarily limits the time available for compression research. These questions are presented not only as directions for our own future work, but also as opportunities for other researchers to investigate, expand upon, and potentially improve these methods.

1. **Branch-free activity-map decoding.** Expanding flags with SIMD masks could make the activity map No-Pain for LZ4, Snappy and LZFSE, where it already gives the largest size reductions.
2. **Large-text HCM.** Hash-table scaling and dictionary preprocessing with an MIT-licensed dictionary, to compete with BSC and ZPAQ on enwik8.
3. **Beating ZPAQ -m4 at Tier 4.** hcmx-s with the x86 filter is now smaller on Silesia (198,424 vs 201,133) but slower; the speed gap is the remaining obstacle.
4. **Typed columnar inference.** Multi-byte column types for Parquet and ORC, compared against hand-designed schemas.
5. **Combary marker theory.** A predictive test for when a location marker's positional cost is lower than the information it isolates (Chapter 7).
6. **Broader corpora.** Modern data — JSON logs, container images, genomic files and larger medical-image sets — to establish *Class* coverage beyond the classic corpora.
7. **Independent replication** of all results using the published scripts.
8. **Open items from the research log:**
    - Recheck bzip2-hi at a fixed block size.
    - Measure the decode multiple M for each record-delta and activity-map pipeline on full corpora.
    - Rebuild hcm-trit v1.
    - Broaden the JPEG XL image results with representative sample tiles.

Chapter 24 — Licensing and Use

> **Executive summary.** Michael Harrington's work — the author's additions in HCM and its variants (whose core design follows lpaq1; see Section 1.8), the record-detection pipeline, the Combary activity map, the Harrington Exchange, the header methods, and the test scripts — is released under the MIT License, which allows anyone to use it for any purpose, including commercially, provided the copyright notice is kept. Many results in this book are improvements to other people's software. Those projects keep their own licenses, and where a license requires it, modified versions stay under that license. This chapter lists which is which.

24.1 The author's work — MIT License

Copyright © 2026 Michael Harrington. Released under the MIT License.

The following are covered:

- HCM v3–v3.3, hcm-trit, HCM-Bin, hcmx and hcmx-s, together with their source code and Windows builds;
- the record-length detector, the record-delta pipeline and its SIMD inverse;
- the Combary activity map and its encoder and decoder;
- the Quad Exchange, the Harrington Exchange and the Harrington header methods;
- the No-Pain search scripts, the benchmark harnesses and the result files;
- **the author's changes** to third-party tools (patches, search drivers, configuration tables) — to the extent that the license of the underlying project permits MIT licensing of those changes.

Published repository: https://github.com/CombaryMaker/Harringtons_Insights

24.2 Third-party software used or modified

Licenses are given as published by each project at the time of testing. Readers should confirm them against each project's own license file before redistribution.

Software	License	Used as	Status of the author's changes
zstd	BSD-3-Clause or GPL-2.0 (dual)	zstd-hi; back end	Encoder-side settings and driver: MIT. Any patched library copy follows zstd's license.
LZ4	Library BSD-2-Clause; CLI GPL-2.0	Back end	Driver only: MIT
zlib / gzip	zlib license / GPL-3.0+	Back end	Driver only: MIT
xz / liblzma (v5.4.5)	Public domain (liblzma); parts of the tools GPL	xz-hi	Search driver: MIT; output is standard .xz
LZMA SDK	Public domain	lzma-hi	MIT
7-Zip	LGPL-2.1+ (with the unRAR restriction); some BSD parts	7z-hi (command-line settings only)	Driver: MIT; 7-Zip itself unmodified
bzip2	bzip2 license (BSD-style)	bzip2-hi (patched `compress.c`)	The patch must keep bzip2's notices; the author's additions are MIT where compatible
Brotli	MIT	Brotli-hi	MIT
Snappy	BSD-3-Clause	Back end	Driver: MIT
LZFSE	BSD-3-Clause	Back end	Driver: MIT
RAR / unrar	Proprietary (RAR); unrar freeware license	Switch search only, unmodified	No changes to RAR; scripts MIT
FLAC	libFLAC BSD-3-Clause; tools GPL-2.0	Encoder settings	Driver: MIT
libwebp	BSD-3-Clause	Encoder settings	Driver: MIT
libjxl (JPEG XL)	BSD-3-Clause	jxl-hi settings	Driver: MIT
JPEG-LS implementation	Per the implementation used	Tuned parameters	Settings: MIT
ZPAQ	Public domain (libzpaq)	Comparison only	—
paq8px	GPL	Laboratory "lite" build	**Modified versions remain GPL**
cmix	GPL-3.0	Laboratory memory-reduced build; dictionary pre-step	**Modified versions and the dictionary experiment remain GPL-3.0**; excluded from the MIT release
bsc, bzip3	Apache-2.0, LGPL-3.0	Comparison only	—

Software	License	Used as	Status of the author's changes
DuckDB	MIT	Parquet experiments	—
Apache ORC, Apache Parquet	Apache-2.0	Columnar experiments	Benchmark program: MIT
qpdf	Apache-2.0	PDF baseline and file writer	Rewrite script: MIT; qpdf unmodified
poppler (pdftoppm)	GPL-2.0 / GPL-3.0	Render verification only	—
pdf.js test PDFs	Apache-2.0 (repository); individual test files carry their own terms	Test data only	—
Arm Optimized Routines	MIT or Apache-2.0 with LLVM exception	exp/log routines ported into HCM v2	Retain the original notice in any distribution of v2

24.3 Rules applied

1. **The author's own code** carries the MIT License and the author's copyright.
2. **Changes to permissively licensed projects** — BSD, MIT, zlib or public domain — are released under MIT, with the original project's notices retained as its license requires.
3. **Changes to copyleft projects** — GPL or LGPL — stay under that project's license whenever they are distributed. In this research that applies to the paq8px and cmix laboratory builds and to the cmix dictionary experiment. None of these is part of HCM.
4. **Tools used without modification** (RAR, ZPAQ, bsc, bzip3, DuckDB) are named for reproducibility only. Their licenses govern their own use.
5. **Output files** produced by the Modified tools — standard .xz, .7z, .bz2, .zst, .br, .rar, .flac, .webp and .jxl files — are ordinary files in those formats. No license from this work attaches to them.

24.4 Patents and trademarks

Combary patents. The author was granted five U.S. patents, four of which apply to Combary (Chapter 7) in data storage, data transmission and data encryption. Four have expired because their maintenance fees were not paid; one remains in force.

U.S. patent	Title	Status (October 2026)
9,437,236	—	Expired (maintenance fee not paid; lapsed October 2020)
9,786,318	—	Expired (maintenance fee not paid)
10,121,510	—	Expired (maintenance fee not paid)
10,930,314	Encoding Data (issued 23 February 2021)	Expired (maintenance fee not paid)
11,636,878	Encoding Data (issued 25 April 2023; continuation)	In force

The exact claims are in the patents themselves, which are public at the U.S. Patent and Trademark Office. Expired patents grant no exclusive rights to anyone.

Compression methods. No patents are pending on any of the compression methods, transforms or tool modifications described in this book.

Relationship between the two. The MIT License (Section 24.1) covers the compression software and methods in this book. It does not grant rights under the author's one Combary patent still in force (U.S. 11,636,878). Applying Combary ideas in compression, as in Chapters 7, 8 and 11, falls under the MIT-licensed work. Using Combary in an application claimed by U.S. 11,636,878 does not.

Trademarks. Product names in this book are trademarks of their respective owners and are used for identification only.

24.5 What the MIT License requires, and the author's wish for recognition

The author's intention is to release the compression modifications under the MIT License. Commercial and non-commercial use is permitted without royalties. The only legal condition is that the copyright notice and the permission notice be kept in all copies or substantial portions of the software. The license does not require prominent attribution in a product's interface or documentation. The author nonetheless asks that users who adopt these methods credit Michael Harrington as their creator.

Appendix A — Full Results Tables

Sizes are compressed bytes (≈ where derived from normalised figures, within ±100 bytes); times are median decode times. This appendix collects the per-file tables not reproduced in full in the chapters. Every other per-file table appears in its chapter: HCM Canterbury (Chapters 10–12), Silesia Tier-4 race and enwik8 (Chapter 13), record delta (Chapter 4), activity map (Chapter 8), encoder tuning (Chapter 5), images and audio (Chapter 16), and columnar formats (Chapter 17).

A.1 hcmx on Silesia, fast mode

File (bytes)	v3.3	trit2	bin1	Chosen
dickens (10,192,446)	2,224,460 / 9.8 s	2,214,295 / 11.8 s	—	trit2
mozilla (51,220,480)	13,410,668 / 56.7 s	13,280,379 / 70.4 s	13,140,954 / 89.3 s	bin1
mr (9,970,564)	2,230,498 / 8.2 s	2,221,194 / 9.7 s	2,209,638 / 11.8 s	bin1
nci (33,553,445)	1,428,413 / 19.6 s	1,388,676 / 20.7 s	—	trit2
ooffice (6,152,192)	2,288,203 / 7.6 s	2,269,256 / 9.3 s	2,254,137 / 11.9 s	bin1
osdb (10,085,684)	2,376,630 / 11.0 s	2,355,941 / 13.1 s	2,333,287 / 16.6 s	bin1
reymont (6,627,202)	1,097,722 / 5.3 s	1,089,243 / 6.0 s	—	trit2
samba (21,606,400)	3,405,599 / 20.2 s	3,352,638 / 22.0 s	—	trit2
sao (7,251,944)	3,921,719 / 10.9 s	3,919,781 / 14.4 s	3,906,120 / 17.0 s	bin1
webster (41,458,703)	6,442,803 / 39.1 s	6,378,584 / 45.4 s	—	trit2
x-ray (8,474,240)	3,751,422 / 9.6 s	3,718,404 / 13.5 s	3,718,336 / 18.6 s	bin1
xml (5,345,280)	352,590 / 4.4 s	343,846 / 4.5 s	—	trit2
Total (211,938,580)	**42,930,727 / 202 s**	**42,532,237 / 241 s**	—	**best-of 42,329,754**

A.2 hcmx-s on Silesia, fast mode (x86 filter and “lite” pointer)

File (bytes)	v3.3	trit2	lite	bin1	bin1-lite	trit2-x86	bin1-x86
dickens (10,192,446)	≈2,224,500 / 10.9 s	≈2,214,300 / 13.3 s	≈2,217,000 / 12.8 s	—	—	—	—
mozilla (51,220,480)	≈13,410,600 / 53.5 s	≈13,280,400 / 79.2 s	≈13,312,300 / 64.8 s	≈13,141,000 / 84.0 s	≈13,171,700 / 69.8 s	—	—
mr (9,970,564)	≈2,230,500 / 7.1 s	≈2,221,200 / 9.1 s	≈2,224,600 / 8.2 s	≈2,209,600 / 10.9 s	≈2,212,800 / 9.7 s	—	—
nci (33,553,445)	≈1,428,400 / 18.3 s	≈1,388,700 / 22.4 s	≈1,390,400 / 23.0 s	—	—	—	—
ooffice (6,152,192)	≈2,288,200 / 7.6 s	≈2,269,300 / 9.8 s	≈2,279,000 / 9.1 s	≈2,254,100 / 11.1 s	≈2,263,600 / 10.5 s	≈1,990,100 / 9.3 s	≈1,978,100 / 11.6 s
osdb (10,085,684)	≈2,376,600 / 11.9 s	≈2,355,900 / 14.5 s	≈2,360,700 / 13.7 s	≈2,333,300 / 18.2 s	≈2,336,200 / 16.4 s	—	—
reymont (6,627,202)	≈1,097,700 / 5.5 s	≈1,089,200 / 7.1 s	≈1,089,700 / 6.9 s	—	—	—	—
samba (21,606,400)	≈3,405,600 / 17.8 s	≈3,352,600 / 23.2 s	≈3,352,400 / 23.0 s	—	—	—	—
sao (7,251,944)	≈3,921,700 / 11.0 s	≈3,919,800 / 16.0 s	≈3,919,900 / 11.3 s	≈3,906,100 / 16.2 s	≈3,907,300 / 12.7 s	—	—
webster (41,458,703)	≈6,442,800 / 36.9 s	≈6,378,600 / 46.6 s	≈6,384,000 / 43.6 s	—	—	—	—
x-ray (8,474,240)	≈3,751,400 / 9.1 s	≈3,718,400 / 14.5 s	≈3,738,600 / 11.9 s	≈3,718,300 / 16.2 s	≈3,738,000 / 13.1 s	—	—
xml (5,345,280)	≈352,600 / 3.9 s	≈343,800 / 4.4 s	≈344,500 / 4.2 s	—	—	—	—

A.3 zip-hi — per-file choices for Silesia (results from the 27 October session)

Each file was compressed into a standard .zip using the method and settings shown, chosen by search. Decode CPU time and decoder memory are as recorded. The files were verified with 7-Zip and WinRAR. Files using PPMd or LZMA need a zip reader that supports those methods; the zip folder built into Windows supports only Deflate. A comparison against WinZip’s own output was not attempted: WinZip is closed-source and runs only on Windows, so it could be neither modified nor tested in the research environment.

File	Size (bytes)	1,000,000 bits down to	Method and settings	Decode	Decoder memory
dickens	2,268,508	222,568	`-mm=PPMd -mo=8 -mmem=64m`	1.36s	71MB
mozilla	13,309,321	259,844	`-mm=LZMA -mx9 -md=26 -mfb=273 -mlc=3 -mlp=0 -mpb=2`	1.02s	54MB

File	Size (bytes)	1,000,000 bits down to	Method and settings	Decode	Decoder memory
mr	2,298,976	230,576	`-mm=PPMd -mo=6 -mmem=64m`	0.82s	45MB
nci	1,446,220	43,102	`-mm=LZMA -mx9 -md=25 -mfb=273 -mlc=4 -mlp=0 -mpb=0`	0.15s	38MB
ooffice	2,419,567	393,285	`-mm=LZMA -mx9 -md=23 -mfb=273 -mlc=4 -mlp=0 -mpb=0`	0.19s	12MB
osdb	2,369,785	234,965	`-mm=PPMd -mo=10 -mmem=64m`	1.54s	69MB
reymont	1,022,713	154,320	`-mm=PPMd -mo=12 -mmem=64m`	0.77s	71MB
samba	3,726,508	172,472	`-mm=LZMA -mx9 -md=25 -mfb=273 -mlc=4 -mlp=0 -mpb=0`	0.26s	27MB
sao	4,415,023	608,805	`-mm=LZMA -mx9 -md=23 -mfb=64 -mlc=0 -mlp=2 -mpb=2`	0.29s	13MB
webster	6,392,189	154,182	`-mm=PPMd -mo=8 -mmem=256m`	4.93s	157MB
x-ray	3,848,954	454,195	`-mm=PPMd -mo=4 -mmem=64m`	1.94s	57MB
xml	378,791	70,865	`-mm=PPMd -mo=16 -mmem=64m`	0.33s	56MB

Appendix B — Specifications

B.1 HCM v3 container

Field	Size	Content
Magic	4 bytes	`HCM3` (`HCMT` for hcm-trit v1)
Mode	1 byte	Bit 0: max mode. Binary flag (v3.1+). PTR = 64 (ternary pointer). XREC = 128 (HCM-Bin record contexts).
Length	4 bytes	Original size, little-endian
CRC-32	4 bytes	Of the original data
Payload	rest	32-bit binary arithmetic-coded bits

hcm-trit v1 inserts after the mode byte: version (1 byte), swap pattern (2 bytes, base 3) and flip (1 byte).

B.2 Record-length detector

```
input: first min(n, 200000) bytes
last[v] = -1 for all byte values v
count[d] = 0
for i in 0..len-1:
    v = x[i]
    if last[v] >= 0: count[i - last[v]] += 1
    last[v] = i
take the 20 most frequent distances; R = first d with 2 <= d <= 512; else R = 1
```

The HCM in-model detector (v3.1) additionally requires a byte value to repeat at equal spacing three or more times.

B.3 Record delta

- **Encode:** `y[i] = x[i] - x[i-R]` (mod 256) for i ≥ R, otherwise `y[i] = x[i]`.
- **Decode:** with step = min(R, 16), the decoder adds 16-byte vectors `t[i..i+15] + o[i-R..i-R+15]` into the output buffer `o`, advancing by `step`. A scalar tail finishes the remainder. Lanes at or beyond R in each step are recomputed in the next step.
- **Header:** 1 flag byte + R.

B.4 Combary activity map

- **flags:** bit i (most significant bit first) = 1 if i < R or `x[i] != x[i-R]`.
- **literals:** `x[i]` for each set flag, in order.
- **Streams:** flags and literals are compressed independently.
- **Header (≈6 bytes):** R and the stream lengths.
- **Decode:** `out[i] = flag ? lit[j++] : out[i-R]`.

B.5 Harrington shrinking header

For a permutation of N values transmitted in order, entry i (1-based) is written as its index among the still-unused values, in $\lceil \log_2(N - i + 1) \rceil$ bits. The final entry is implicit.

B.6 Quad Exchange

Element	Content
Top-5 targets	15, 14, 13, 11, 7
Bottom-5 targets	1, 2, 4, 8, 0
Remaining 6	Mapped in ascending order

Element	Content
Header	Swap1–Swap10 as nibbles (40 bits)
Tie-breaks	Most common: count descending, value ascending. Least common: count ascending, value ascending.

Appendix C — Commands and Versions

Tool	Version / source	Baseline command
gzip	1.12	`gzip -9n`
zstd	GitHub, 1.6.0-dev	`zstd --ultra -22` (and -19, -9, -3, -1)
LZ4	GitHub, latest at test date	`lz4 -12`
xz	5.4.5	`xz -9e`
LZMA SDK	Igor Pavlov	level 9
7-Zip	Linux build	`7z a -mx9`
bzip2	GitHub, latest	`bzip2 -9`
Brotli	1.2.0	quality 2, 5, 8, 11
RAR	7.30 beta 2 (Linux x64 trial)	`rar a -m5 -ma5 -mt1`
Snappy	GitHub, latest	library default
LZFSE	GitHub, latest	library default
FLAC	xiph/flac, latest	`flac -8`
libwebp	GitHub, latest	`cwebp -lossless -z 9`
libjxl	latest	`cjxl -d 0 -e 7`
ZPAQ	public release	`-m4`, `-m5`
paq8px	v217	`-8`
cmix	v21	default
DuckDB	1.3.2	`COPY … (FORMAT parquet, COMPRESSION zstd | snappy)`
Apache ORC	C++ from GitHub, current	writer defaults with zlib, zstd or snappy

Test machine: Linux cloud container, Intel Xeon at 2.30 GHz (2 virtual CPUs), 7 GB RAM, single-threaded decoding.

Reproducibility gap. Most third-party tools were built from their latest source on the test date, and exact commit identifiers were not recorded for every build. They are not yet recorded; until they are, rebuilding from source may give small differences.

Per-chapter search spaces are listed in each chapter. Exact per-file chosen settings are stored in the result files listed in Appendix D.

Appendix D — Scripts, Tools and Repository

Public repository: https://github.com/CombaryMaker/Harringtons_Insights (HCM v3.3, xz-hi, bzip2-hi, Windows executables).

Author's working archive: `Documents\data compression\`

Path	Contents
`hcm\`	HCM v3.3 source and Windows executables
`tools\hcmx\`	hcmx (v3.3 + ternary pointer + HCM-Bin): source, Canterbury and Silesia results (`cant.json`, `sil.json`)
`tools\hcmx-s\`	hcmx-s (x86 filter, lite pointer), Silesia results
`tools\tier4_race\`	Tier 4 race, enwik8 and activity-map-before-HCM results (`t4.json`, `t4e.json`, `t5.json`)
`tools\silesia_record_delta\`	Record-delta harness (`rb.c`) and fast-level results
`tools\lzma_hi_xz_hi\`	lzma-hi / xz-hi searches and re-timings
`tools\7z_hi\`	7z-hi search and timings
`tools\zstd_brotli_hi\`	zstd-hi / Brotli-hi configurations and timings
`tools\image_jxl_jls\`	JPEG XL and JPEG-LS searches and re-timings
`tools\zip-hi\`	zip-hi search and per-file results
`bzip2-hi\`, `xz-hi\`, `windows\`	Published tool sources and builds

Scripts for the later experiments — Snappy, LZFSE, RAR, FLAC, WebP, PNG, Parquet, ORC, the Combary activity map and the header tests — are in `tools\oct27_experiments\`; the PDF scripts and logs are in `tools\pdf_hi\`. The bzip2-hi per-file run and the early HCM laboratory sources (Chapters 10–11) are not in the archive: the earlier workspace was lost. Their published totals are reproducible only through the rebuilt tools (hcmx), not byte-for-byte.

Verification procedure: every script decompresses with the stock decoder (or stock decoder + inverse step) and compares the result byte for byte with the original before recording a size.

Appendix E — References

Corpora and benchmarks

1. Arnold, R. and Bell, T. “A corpus for the evaluation of lossless compression algorithms.” *Proceedings of the Data Compression Conference (DCC ’97)*, Snowbird, UT, IEEE, 1997, pp. 201–210. (Canterbury Corpus.)
2. Deorowicz, S. *Universal lossless data compression algorithms.* PhD thesis, Silesian University of Technology, Gliwice, 2003. (Silesia Corpus.)
3. Mahoney, M. *Large Text Compression Benchmark.* https://www.mattmahoney.net/dc/text.html (enwik8, enwik9).
4. Mozilla. *pdf.js* test suite, `test/pdfs`. https://github.com/mozilla/pdf.js
5. *Free Spoken Digit Dataset.* https://github.com/Jakobovski/free-spoken-digit-dataset
6. Eastman Kodak. *Kodak Lossless True Color Image Suite* (24 images, PhotoCD PCD0992).

Modelling and coding

7. Witten, I. H., Neal, R. M. and Cleary, J. G. “Arithmetic coding for data compression.” *Communications of the ACM* 30(6), 1987, pp. 520–540.
8. Cleary, J. G. and Witten, I. H. “Data compression using adaptive coding and partial string matching.” *IEEE Transactions on Communications* 32(4), 1984, pp. 396–402. (PPM; escape and exclusion.)
9. Mahoney, M. V. *Adaptive weighing of context models for lossless data compression.* Technical Report CS-2005-16, Florida Institute of Technology, 2005. (PAQ context mixing.)
10. Mahoney, M. *Data Compression Explained.* 2010–2013. https://www.mattmahoney.net/dc/dce.html
11. Mahoney, M. *The ZPAQ Open Standard Format for Highly Compressed Data, Level 2.* 2014. https://www.mattmahoney.net/dc/zpaq206.pdf
12. Knoll, B. *cmix.* https://github.com/byronknoll/cmix
13. paq8px contributors. *paq8px.* https://github.com/hxim/paq8px
14. Burrows, M. and Wheeler, D. J. *A block-sorting lossless data compression algorithm.* SRC Research Report 124, Digital Equipment Corporation, Palo Alto, 1994.
15. Huffman, D. A. “A method for the construction of minimum-redundancy codes.” *Proceedings of the IRE* 40(9), 1952, pp. 1098–1101.
16. Lehmer, D. H. “Teaching combinatorial tricks to a computer.” *Proceedings of Symposia in Applied Mathematics* 10, American Mathematical Society, 1960, pp. 179–193. (Lehmer code.)

Formats and tools

17. Deutsch, P. *DEFLATE Compressed Data Format Specification version 1.3.* RFC 1951, IETF, 1996.
18. Deutsch, P. and Gailly, J.-L. *ZLIB Compressed Data Format Specification version 3.3.* RFC 1950, IETF, 1996.
19. Collet, Y. and Kucherawy, M. *Zstandard Compression and the ‘application/zstd’ Media Type.* RFC 8878, IETF, 2021.
20. Alakuijala, J. and Szabadka, Z. *Brotli Compressed Data Format.* RFC 7932, IETF, 2016.
21. Alakuijala, J. and Vandevenne, L. *Data compression using Zopfli.* Google, 2013.
22. Collet, Y. *LZ4.* https://github.com/lz4/lz4
23. Pavlov, I. *LZMA SDK* and *7-Zip.* https://www.7-zip.org/sdk.html
24. Collin, L. and Pavlov, I. *The .xz File Format.* https://tukaani.org/xz/xz-file-format.txt; *XZ Utils.* https://tukaani.org/xz/
25. Seward, J. *bzip2 and libbzip2.* https://sourceware.org/bzip2/
26. Roshal, A. *RAR archiver.* https://www.rarlab.com/
27. Google. *Snappy.* https://github.com/google/snappy
28. Apple. *LZFSE.* https://github.com/lzfse/lzfse
29. Grebnov, I. *libbsc.* https://github.com/IlyaGrebnov/libbsc
30. Kosinski, K. *bzip3.* https://github.com/kspalaiologos/bzip3
31. van Beurden, M. and Weaver, A. *Free Lossless Audio Codec.* RFC 9639, IETF, 2024.
32. Google. *WebP Lossless Bitstream Specification.* https://developers.google.com/speed/webp/docs/webp_lossless_bitstream_specification
33. W3C. *Portable Network Graphics (PNG) Specification,* Second Edition, 2003; ISO/IEC 15948:2004.
34. ISO/IEC 18181-1. *Information technology — JPEG XL image coding system — Part 1: Core coding system.*
35. ITU-T Recommendation T.87 / ISO/IEC 14495-1. *Lossless and near-lossless compression of continuous-tone still images (JPEG-LS).*
36. ISO 32000-1:2008. *Document management — Portable document format — Part 1: PDF 1.7.* Section 14.4.4.4, “LZW and Flate Predictor Functions”.
37. Berkenbilt, J. *qpdf.* https://github.com/qpdf/qpdf

38. Apache Software Foundation. *Apache Parquet* (https://parquet.apache.org) and *Apache ORC* (https://orc.apache.org) format specifications.
39. DuckDB Foundation. *DuckDB*. https://duckdb.org
40. The poppler developers. *poppler* (`pdftoppm`). https://poppler.freedesktop.org

Appendix F — Random Re-verification of Wins (28 October 2026)

Purpose. To check recorded wins without choosing which ones to re-test, and to record compression time alongside size and decode time.

Method.

- Each method's recorded wins were listed. A sample was drawn with `random.Random(seed).sample()` from the sorted list, seed 2360932695 (drawn with Python's `secrets`).
- Sample size per method: 1 if it had 1–2 wins, 2 if it had 3, otherwise 3. RAR was limited to its single win (sao).
- Measured: compressed bytes, compression time (median of 3), decompression time (median of 5–21), CPU time and peak memory for command-line tools, and an exact round trip.
- Close decode results were re-timed in interleaved pairs, 15–31 pairs.
- All 54 measurements round-tripped exactly.
- The draw, the full list of wins and the raw results are saved with the scripts in the author's tools archive.

F.1 Results

Method	Samples	Sizes	Win holds?	Compression time, base → ours
Record delta	kennedy zstd -22; kennedy Snappy; x-ray zstd -19	Reproduced	Yes; Yes; x-ray only with the new prefix-sum inverse (Section 4.3a)	1.35 → 0.84 s; equal; 5.1 → 7.8 s
Record delta, fast level	mr xz -0	Reproduced	Tie within noise	0.76 → 0.70 s
Activity map + gzip	kennedy	Close (zlib, not GNU gzip)	Yes, 2.48 → 1.60 ms	0.54 → 0.02 s
xz-hi	hdfs.log; openstack.log	Exact	Yes; Yes (interleaved)	0.16 → 0.13 s; 0.63 → 0.47 s
lzma-hi	sao; x-ray; xml	liblzma, not the 7-Zip SDK	sao not confirmed; x-ray and xml yes	about equal
7z-hi	asyoulik; fields.c; kennedy	Exact	Yes, all three	kennedy 0.40 → 0.75 s
zstd-hi	counties.jsonl L22; plrabn12 L9; ptt5 L3	Reproduced	Yes, all three	72.9 → 36.3 s; 0.03 → 0.19 s; equal
Brotli-hi	asyoulik q5; sum q8; xargs q11	Reproduced	Ties on asyoulik and xargs; sum yes	equal
bzip2-hi	plrabn12; sum; xargs	Reproduced	Yes; yes; tie	0.07 → 22.9 s; 0.01 → 3.5 s; 0.005 → 0.49 s
RAR -mcd-	sao	Exact	Yes (interleaved 108.0 → 97.3 ms)	1.9 → 3.8 s
PDF row filters	3 PDFs	Reproduced; pages identical	Yes, all three	0.24 → 2.0 s
HCM v3.3, hcm-trit v2, HCM-Bin	7 samples	Exact	Size wins hold; decode costs as documented	slower, as documented
Harrington Exchange	kennedy LZ4, zstd	Exact (decoder written and verified)	Size yes; prototype decoding slower	1.43 → 0.19 s; 1.73 → 2.33 s
Shrinking header	256-entry tables	1,793 bits = 225 bytes	Yes	under 0.4 ms
Column layout	mr ORC zlib, zstd; sao Parquet zstd	Exact	Yes, all three	faster
JPEG XL	RG3; x-ray; MR2 (speed setting)	Exact	Yes, all three	RG3 0.65 → 1.95 s; x-ray 2.0 → 6.6 s; MR2 0.44 → 0.09 s
JPEG XL, disputed award	JPEG2000_UNC	Exact	**Withdrawn**: not faster	equal
JPEG-LS	x-ray	Reproduced (units corrected)	Yes, 119.6 → 100.0 ms	0.12 → 0.09 s
FLAC	george; jackson; theo	Reproduced	Yes; yes; **theo not faster**	0.09 → 0.40 s; 0.09 → 0.38 s; 0.05 → 2.5 s
WebP	01; 10; 12	Reproduced	Ties on 01 and 10; 12 yes	13.4 → 0.5 s; 11.3 → 0.3 s; 10.9 → 1.7 s

F.2 Why some results changed

1. **Timing noise on a shared cloud machine.** Identical decodes varied by 5–30% from run to run. Original single-session timings could not separate a 2–5% win from noise.
2. **Process start-up.** It dominates command-line timings under about 20 ms.
3. **Substitute tools.** liblzma replaced the 7-Zip SDK, and zlib replaced GNU gzip.
4. **A real implementation weakness.** The original inverse was slow at R = 2 (fixed: Section 4.3a).
5. **A units error** in the JPEG-LS section, now corrected.

Wins with large margins reproduced every time.

F.3 Compression time

Tuned encoders are often slower to compress, sometimes greatly so:

- bzip2-hi: 0.07 → 22.9 s on plrabn12.txt;
- JPEG XL at -I 100: about 3×;
- the PDF rewrite: about 8×;
- FLAC per-file search: 4–50×.

Others compress faster:

- zstd-hi on the 27 MB JSONL: 72.9 → 36.3 s;
- record delta with zstd -22: 1.35 → 0.84 s;
- the WebP search settings: 6–40× faster than -z 9;
- the activity map with gzip;
- the detected column layouts.

No-Pain judges decoding, so these times do not change any verdict. The trade-off is discussed in the Introduction.

Appendix G — Best Results Charts

This appendix gathers the best recorded result of every tool built or modified in this book, the author's own builds (HCM and hcmx, built on the lpaq1 design, and the original Harrington Exchange), modified encoders read by stock decoders (xz-hi, lzma-hi, 7z-hi, zip-hi, bzip2-hi, zstd-hi, Brotli-hi, flac-hi, the PDF row filter, RAR with detector delta) and mutants (record-delta front ends, lpaq1-h, the Exchange→Blosc2 hybrid), and sets them beside every standard tool measured on the same benchmark. Sizes are given as 1,000,000 bits down to X bits. *Harrington best-of* is the smallest of the author's own tools chosen file by file, as a production system would choose; *Best standard per file* does the same for the stock tools, so the comparison is fair in both directions.

No new measurements were made for these charts. They are drawn from the tables in Chapters 4, 5, 6, 13 and 15–18, Appendix A, and the same-machine run of Andrew Goodwin's three benchmark sets on 29 October 2026. Decode times from different chapters come from different runs; each stock-versus-ours pair was measured in the same run. The scripts that draw them (best_data.py, best_plots.py, plots.py) are in the repository under benchmarks/charts-2026-10-29.

For each benchmark there is a full ranking (shortest bar is best), a size-against-decode-speed chart (the dashed line joins the results that nothing else beats on both counts) and a chart showing what each of the author's changes did to the stock tool it modifies.

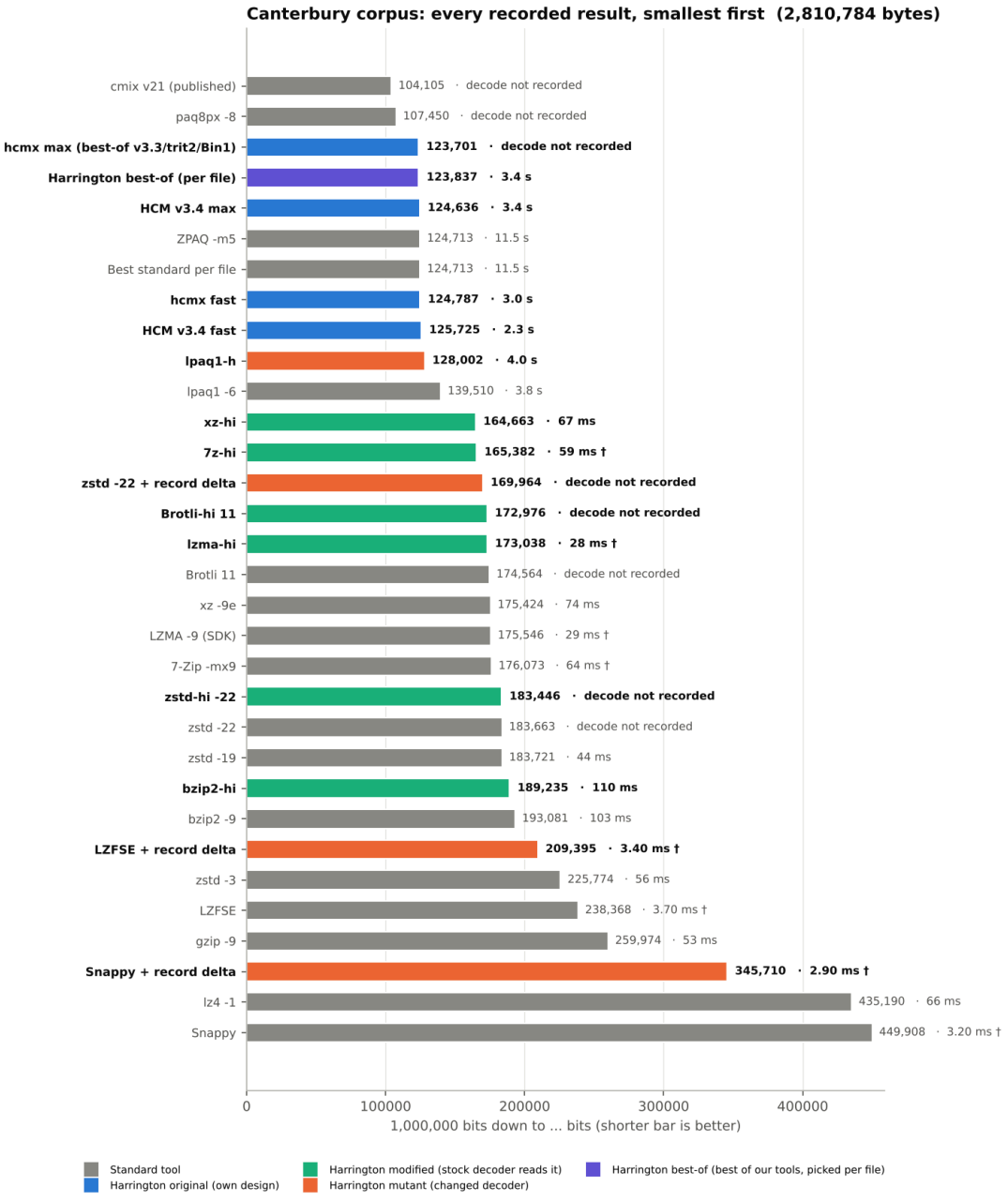
G.1 Scoreboard

Scoreboard: our best against the best standard tool, every benchmark

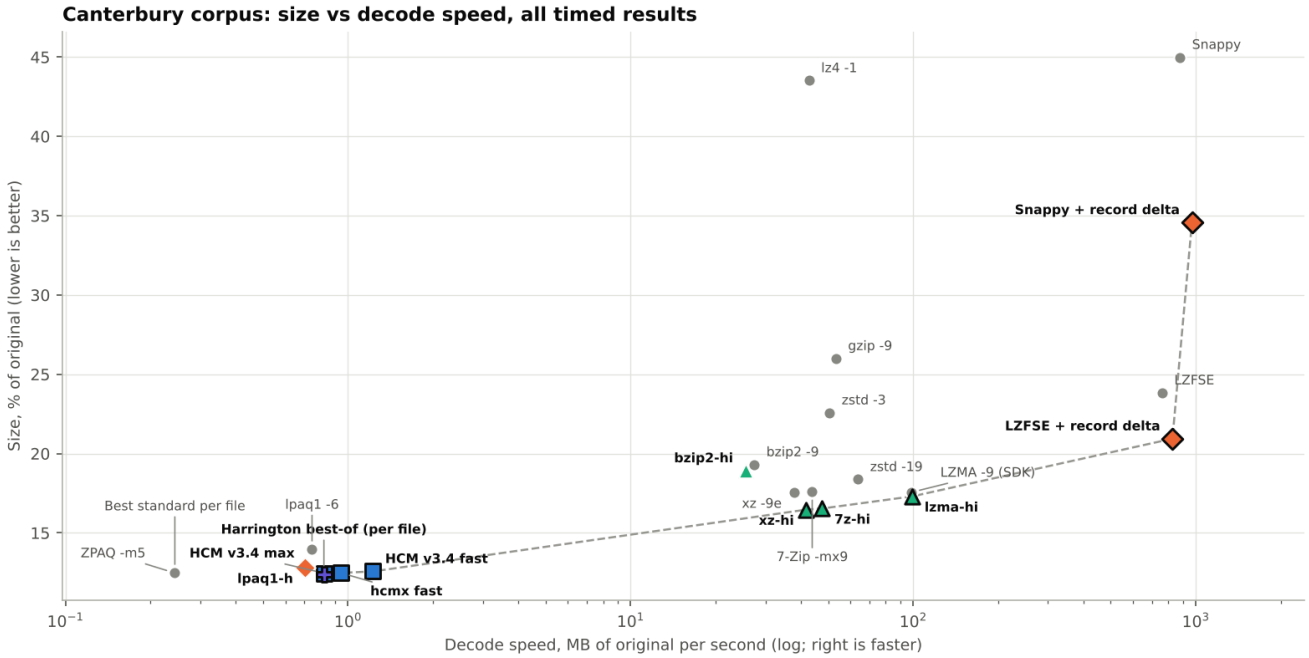
Sizes are 1,000,000 bits down to X bits. "Smaller AND faster" is our smallest entry that beats a standard tool on both size and decode time.

Benchmark	Our best (size, decode)	Best standard tool (size, decode)	Size verdict	Smaller AND faster
Canterbury corpus	hcmx max (best-of v3.3/trit2/Bin1) 123,701 (decode not recorded)	cmix v21 (published) 104,105 (decode not recorded)	18.8% behind cmix v21 (published)	Harrington best-of (per file) 123,837 in 3.4 s beats ZPAQ -m5 124,713 in 11.5 s
Silesia corpus (full)	Harrington best-of (per file) 197,371 (258.9 s)	ZPAQ -m5 184,547 (554.2 s)	6.9% behind ZPAQ -m5	zip-hi (7-Zip zip/PPMd, per-file) 207,119 in 13.6 s beats bzip3 221,432 in 23.1 s
Silesia 1 MB sample (Goodwin)	Harrington best-of (per file) 279,108 (14.6 s)	ZPAQ -m5 271,711 (54.7 s)	2.7% behind ZPAQ -m5	Harrington best-of (per file) 279,108 in 14.6 s beats lpaq1 -6 289,591 in 15.8 s
Synthetic set (Goodwin)	Harrington best-of (per file) 445,817 (22.2 s)	ZPAQ -m5 455,660 (83.1 s)	**OURS SMALLEST OF ALL**	Harrington best-of (per file) 445,817 in 22.2 s beats ZPAQ -m5 455,660 in 83.1 s
Biased packed data	Harrington best-of (per file) 465,170 (15 ms)	order-0 rANS 447,217 (28 ms)	4.0% behind order-0 rANS	Harrington best-of (per file) 465,170 in 15 ms beats ZPAQ -m5 465,419 in 10.6 s
Radio SDR captures	xz-hi 612,815 (171 ms)	ZPAQ -m5 (decode estimated) 577,465 (15.0 s)	6.1% behind ZPAQ -m5 (decode estimate	-
Modern logs and exports	xz-hi (smallest) 60,702 (390 ms)	xz -9e 61,305 (384 ms)	**OURS SMALLEST OF ALL**	xz-hi (smallest) 60,702 in 390 ms beats bzip2 -9 111,264 in 1.9 s
PDF documents (139 files)	Ours, smallest always 837,340 (11.6 s)	qpdf optimisation 851,978 (11.3 s)	**OURS SMALLEST OF ALL**	Ours, No-Pain pick 846,683 in 11.2 s beats qpdf optimisation 851,978 in 11.3 s
Speech audio (FLAC, 6 speakers)	flac-hi (No-Pain) 520,279 (123 ms)	flac -8 531,634 (136 ms)	**OURS SMALLEST OF ALL**	flac-hi (No-Pain) 520,279 in 123 ms beats flac -8 531,634 in 136 ms

G.2 Canterbury corpus

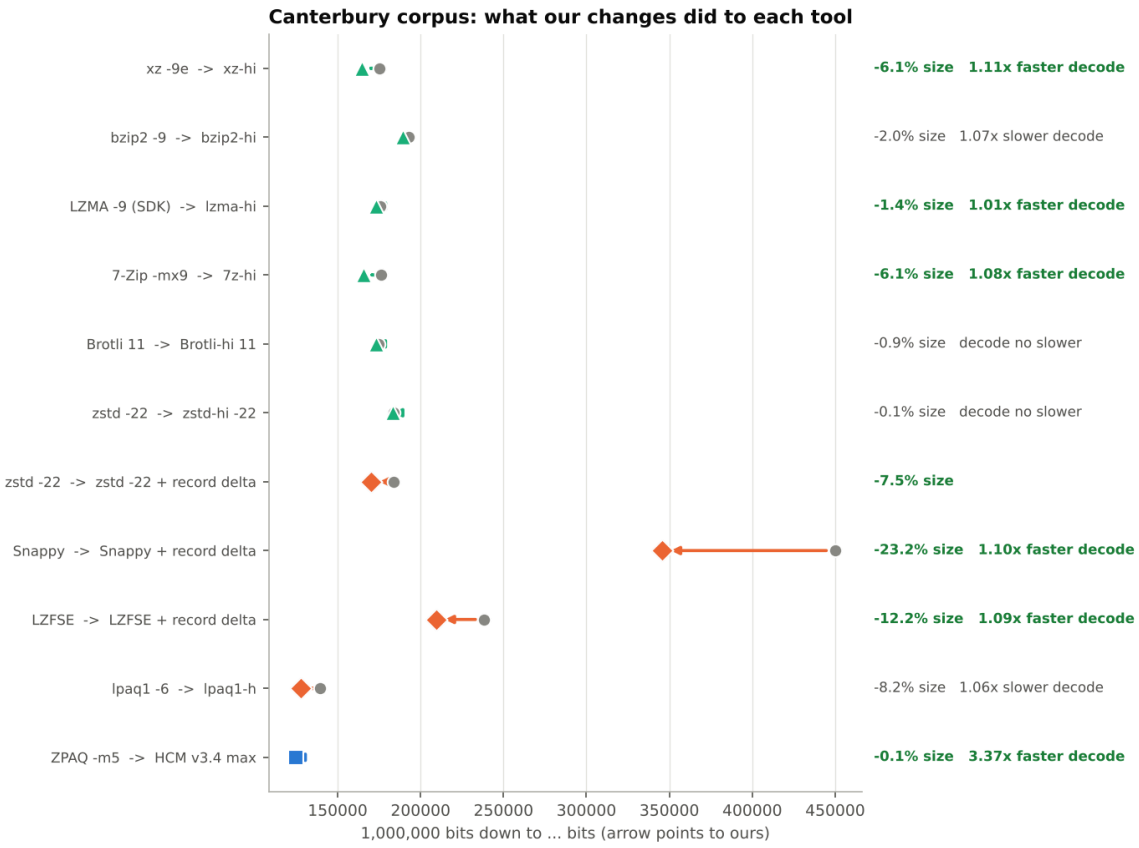


Decode times: † = recorded in a book chapter run (in-process); unmarked = 29 Oct 2026 same-machine run (wall clock).

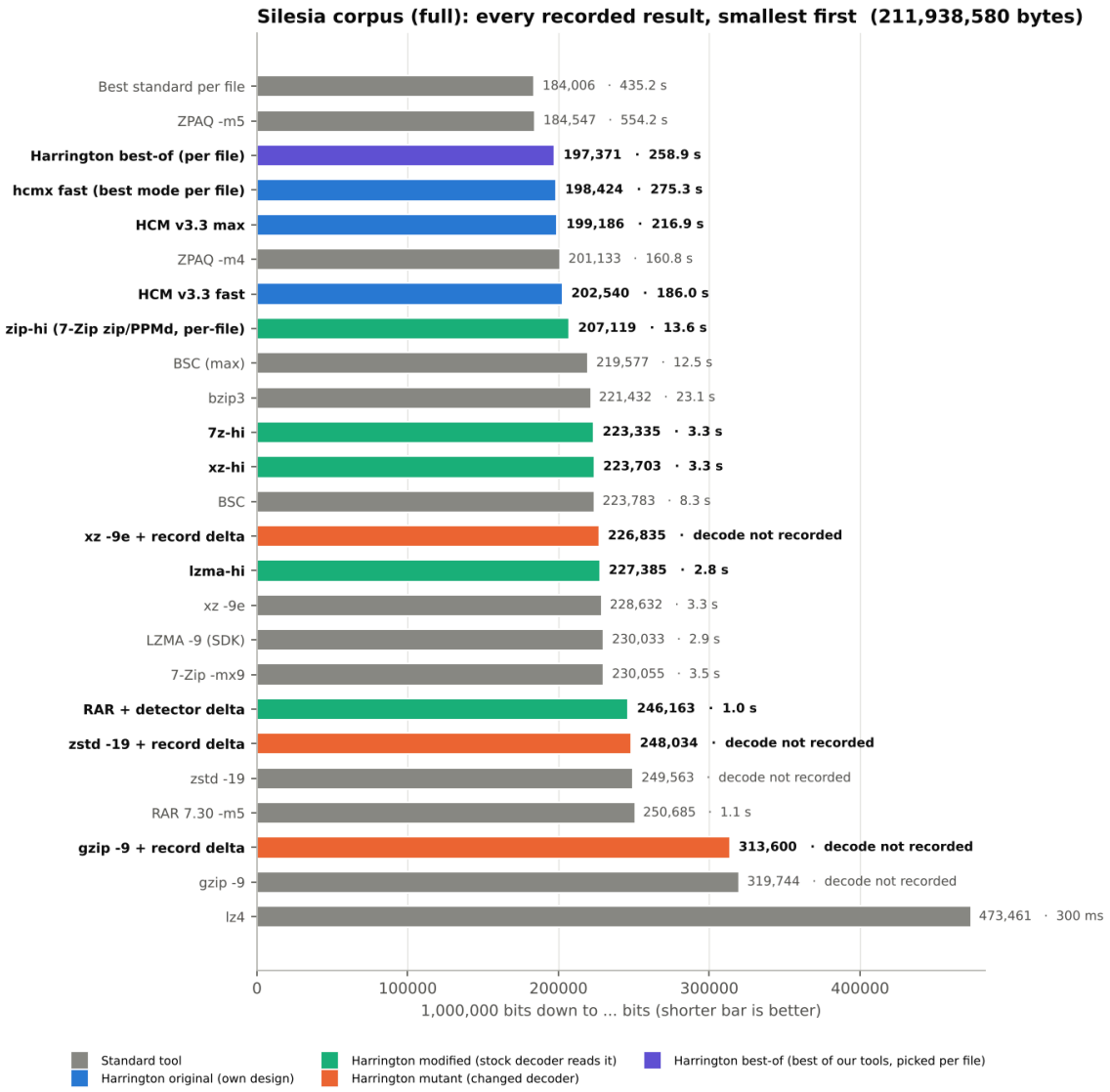


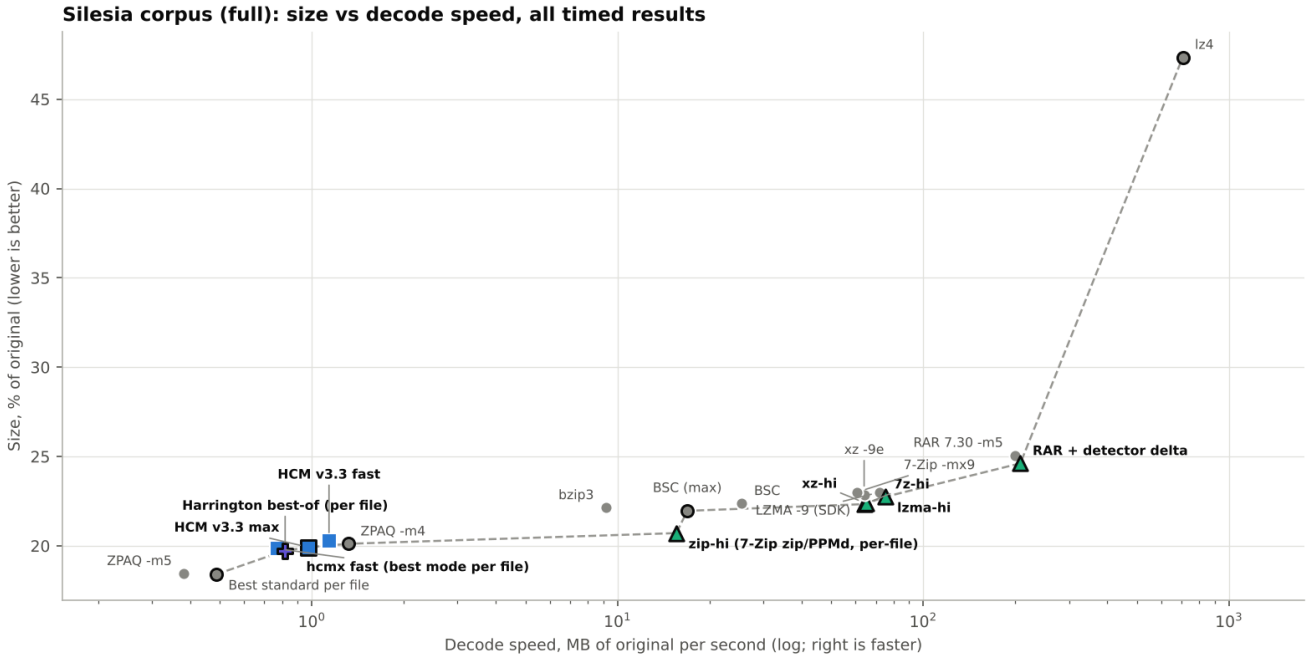
Timing sources: ch4, ch5, run. Times from different book chapters were measured in different runs; each stock-vs-ours pair comes from the same run.

- Standard tool
- Harrington original (own design)
- Harrington modified (stock decoder reads it)
- Harrington mutant (changed decoder)
- Harrington best-of (best of our tools, picked per file)
- Best-trade-off line (nothing beats these on both)
- On the line


Canterbury corpus: what our changes did to each tool
xz -9e -> xz-hi
bzip2 -9 -> bzip2-hi
LZMA -9 (SDK) -> lzma-hi
7-Zip -mx9 -> 7z-hi
Brotli 11 -> Brotli-hi 11
zstd -22 -> zstd-hi -22
zstd -22 -> zstd -22 + record delta
Snappy -> Snappy + record delta
LZFSE -> LZFSE + record delta
lpaq1 -6 -> lpaq1-h
ZPAQ -m5 -> HCM v3.4 max
-6.1% size 1.11x faster decode
-2.0% size 1.07x slower decode
-1.4% size 1.01x faster decode
-6.1% size 1.08x faster decode
-0.9% size decode no slower
-0.1% size decode no slower
-7.5% size
-23.2% size 1.10x faster decode
-12.2% size 1.09x faster decode
-8.2% size 1.06x slower decode
-0.1% size 3.37x faster decode
150000
200000
250000
300000
350000
400000
450000
1,000,000 bits down to ... bits (arrow points to ours)

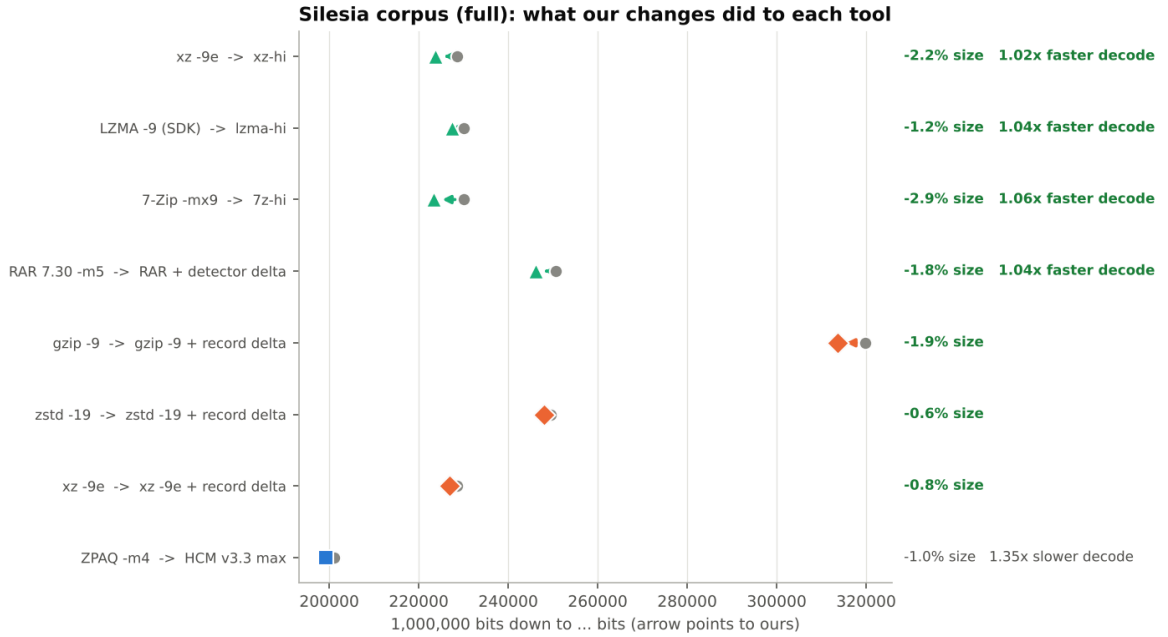
G.3 Silesia corpus (full, 211,938,580 bytes)



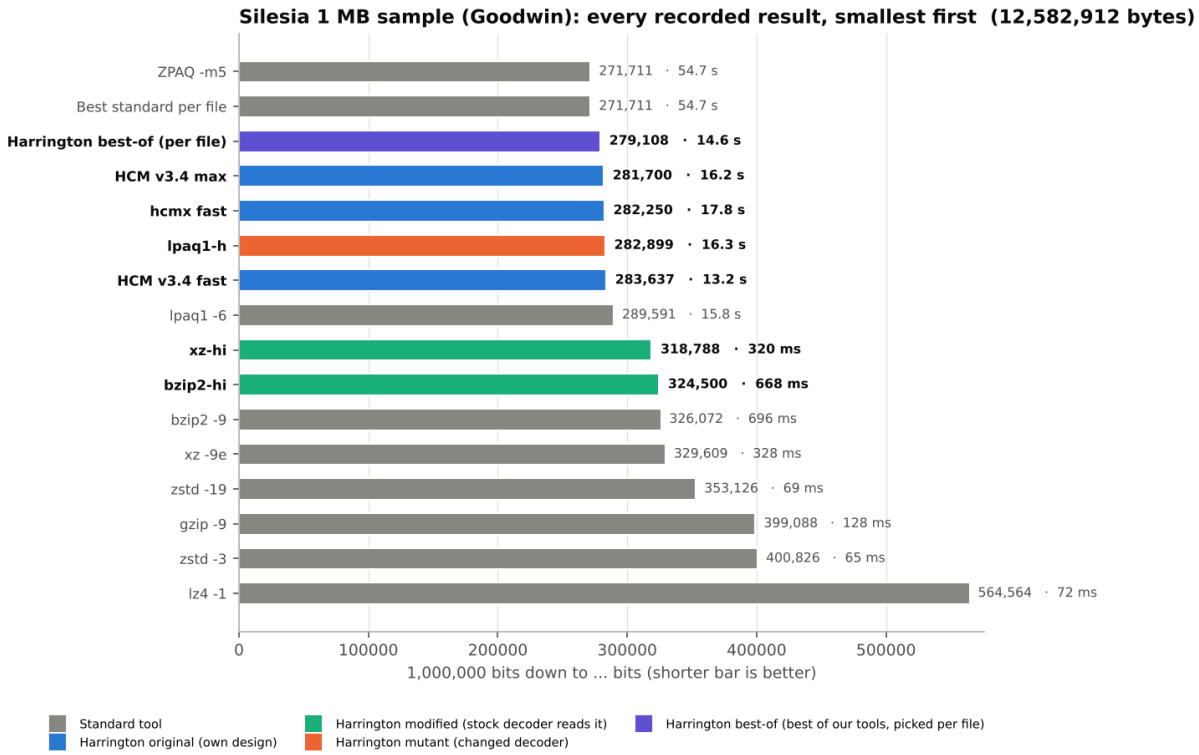


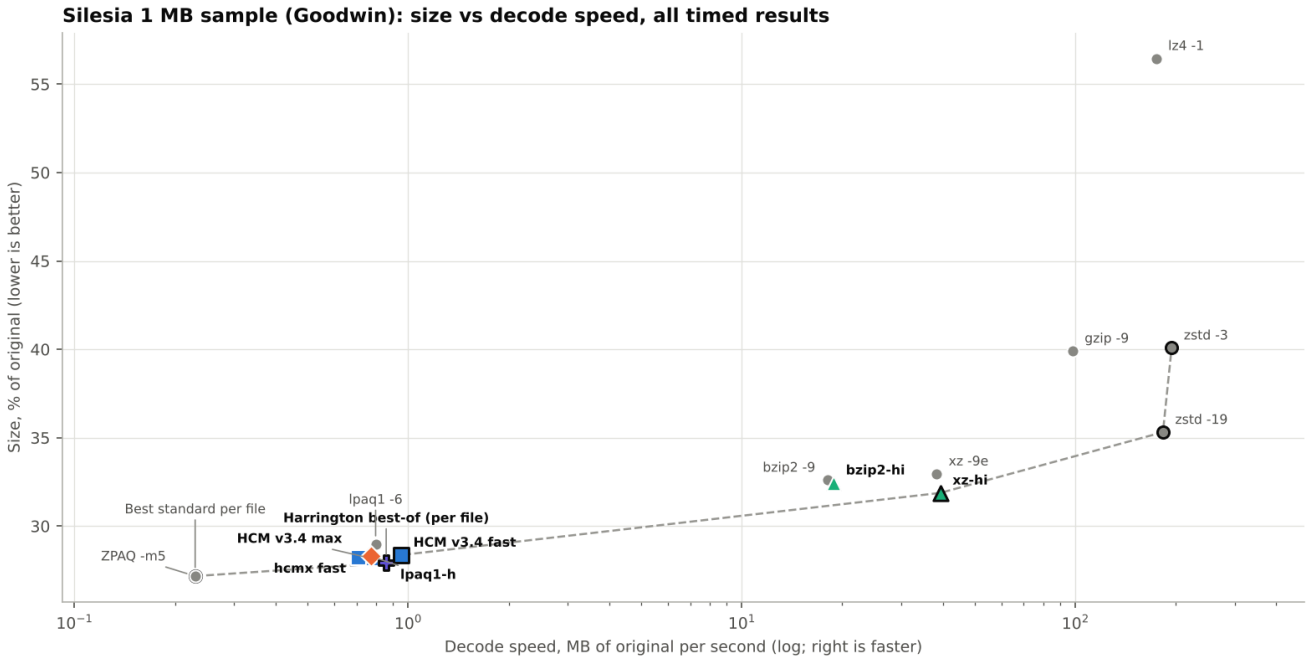
Timing sources: A.1/A.2, ch13, ch5, ch5/ch13/A, composite. Times from different book chapters were measured in different runs; each stock-vs-ours pair comes from the same run.

- Standard tool
- Harrington original (own design)
- Harrington modified (stock decoder reads it)
- Harrington mutant (changed decoder)
- Harrington best-of (best of our tools, picked per file)
- Best-trade-off line (nothing beats these on both)
- On the line



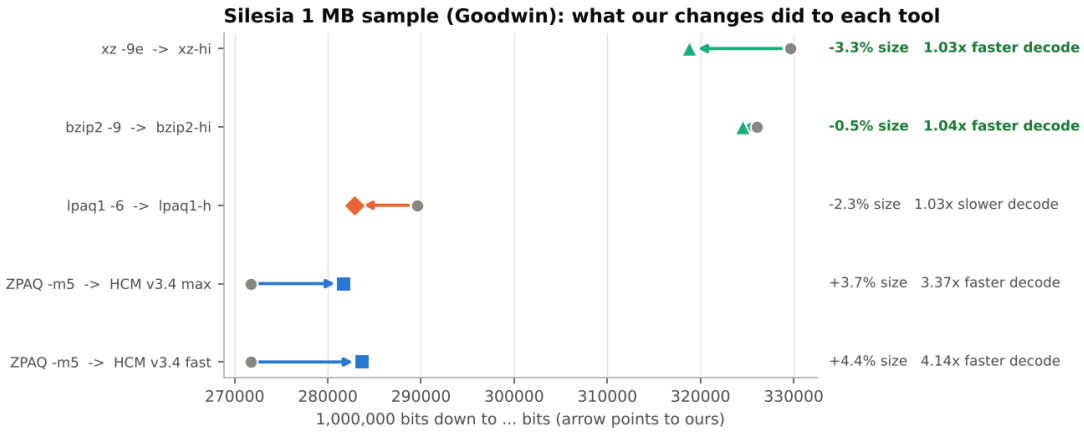
G.4 Silesia 1 MB sample (Goodwin)



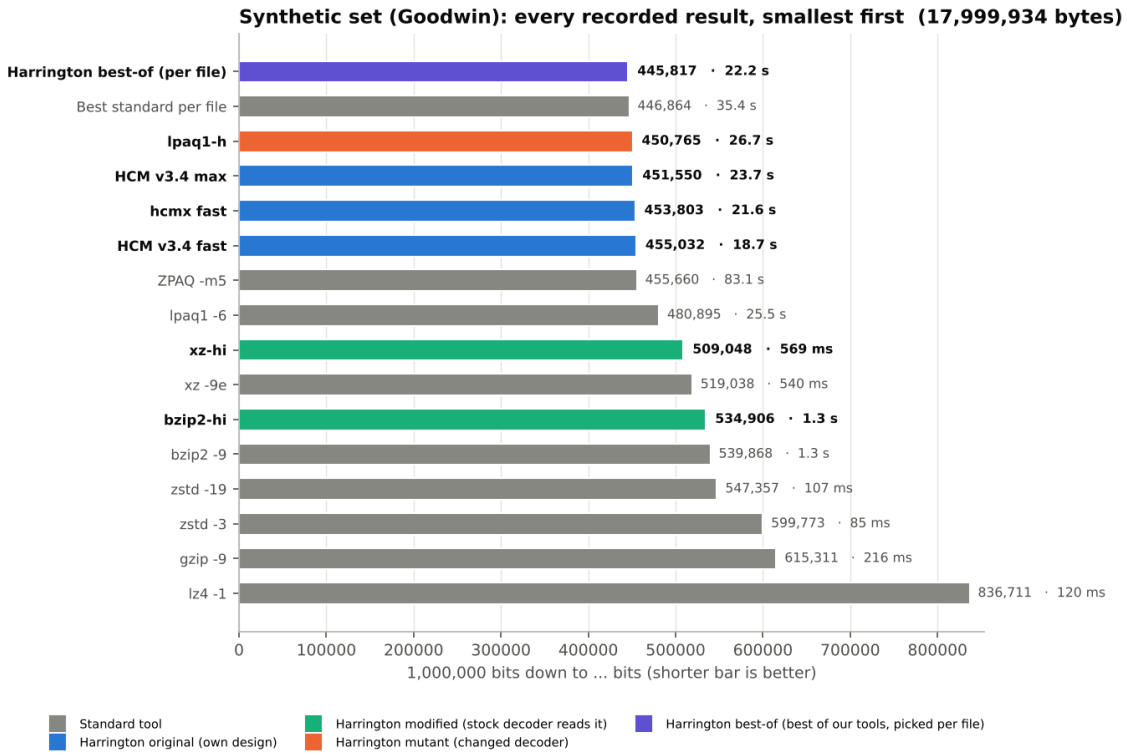


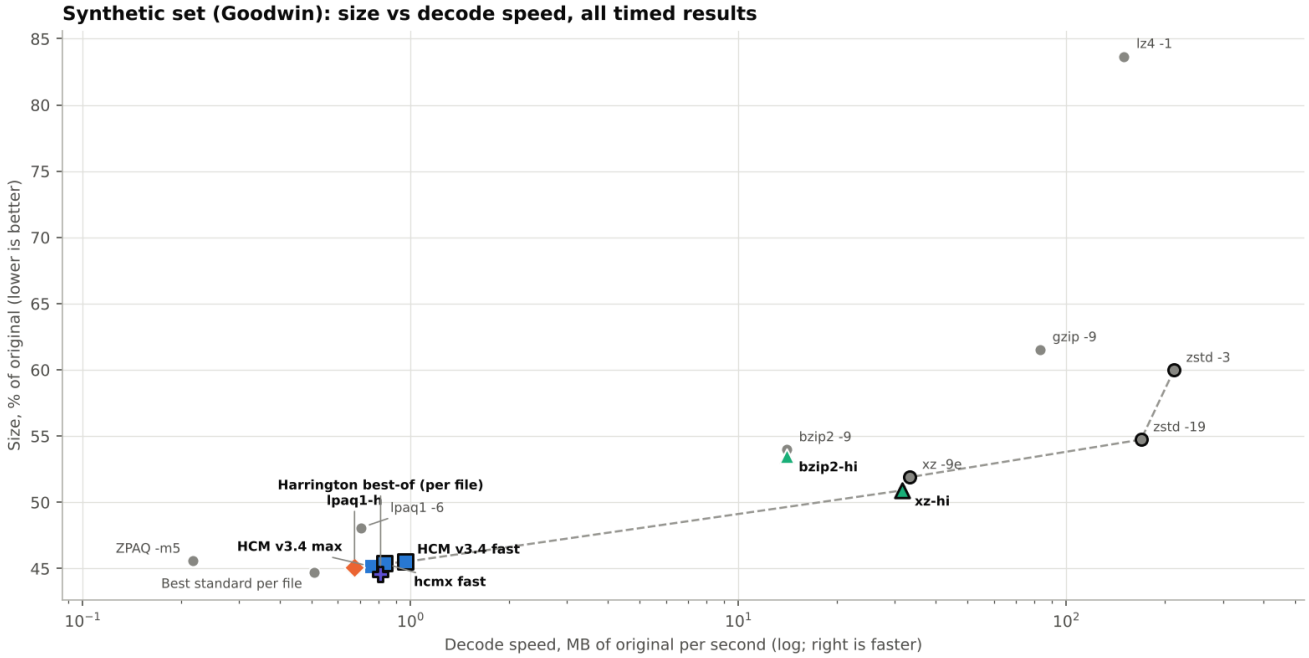
Timing sources: run.

Standard tool
Harrington original (own design)
Harrington modified (stock decoder reads it)
Harrington mutant (changed decoder)
Harrington best-of (best of our tools, picked per file)
Best-trade-off line (nothing beats these on both)
On the line



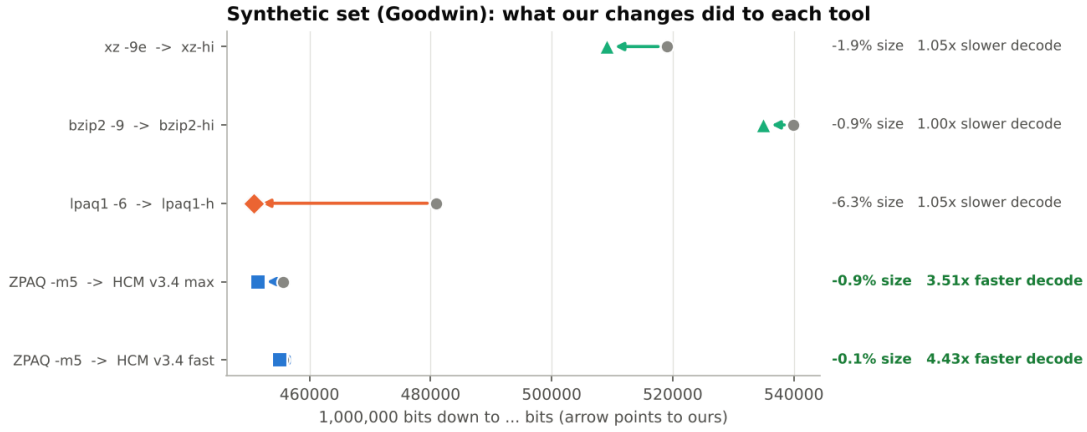
G.5 Synthetic set (Goodwin)



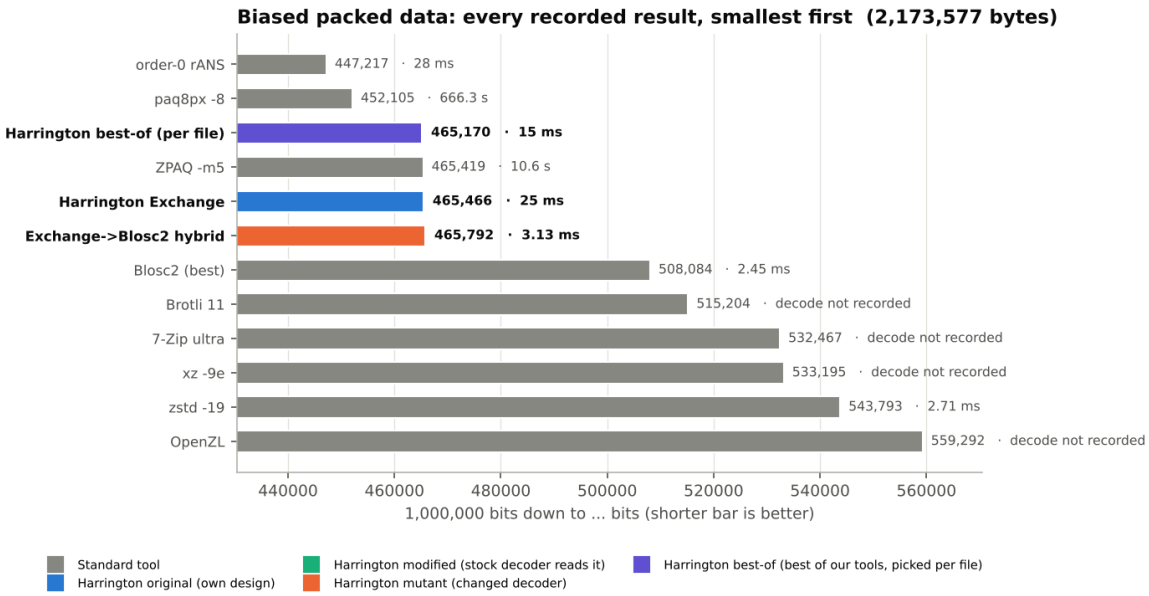


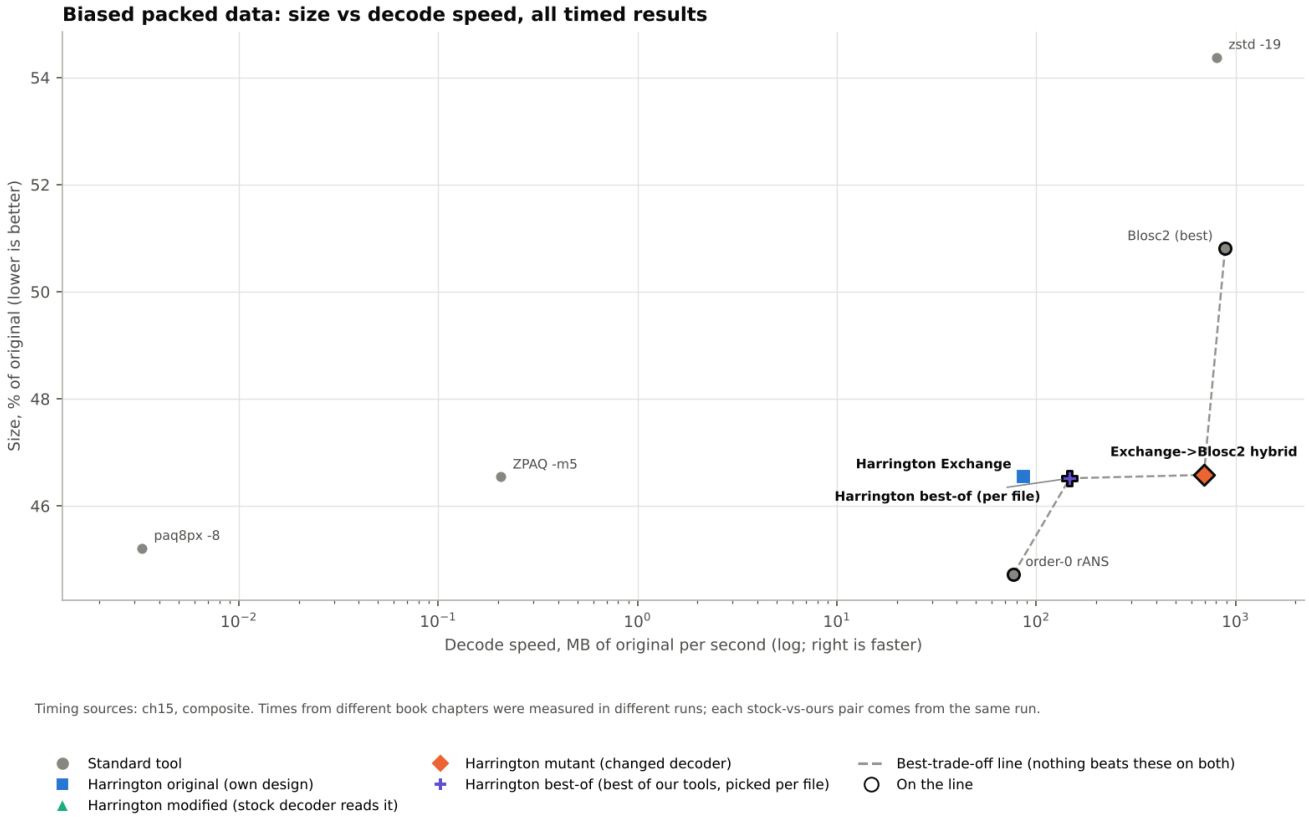
Timing sources: run.

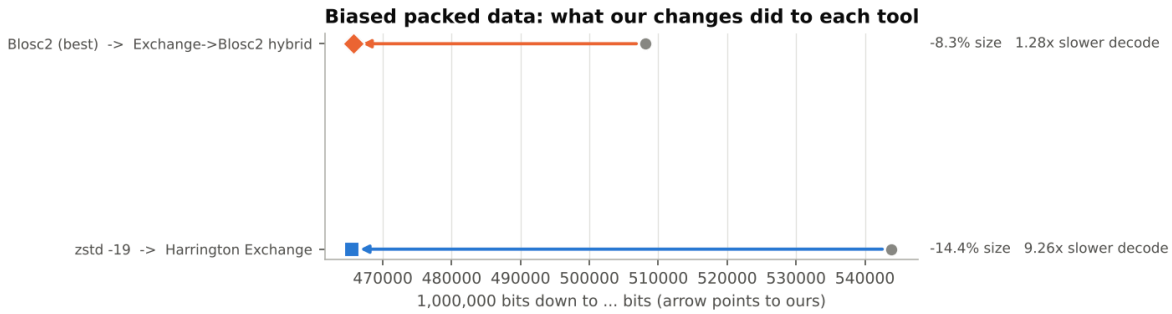
- Standard tool
- Harrington original (own design)
- Harrington modified (stock decoder reads it)
- Harrington mutant (changed decoder)
- Harrington best-of (best of our tools, picked per file)
- Best-trade-off line (nothing beats these on both)
- On the line



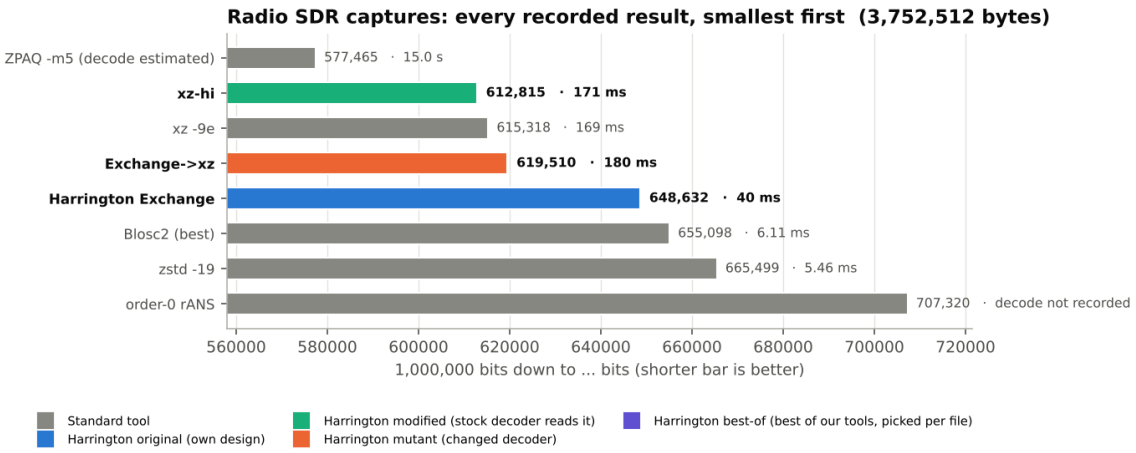
G.6 Biased packed data

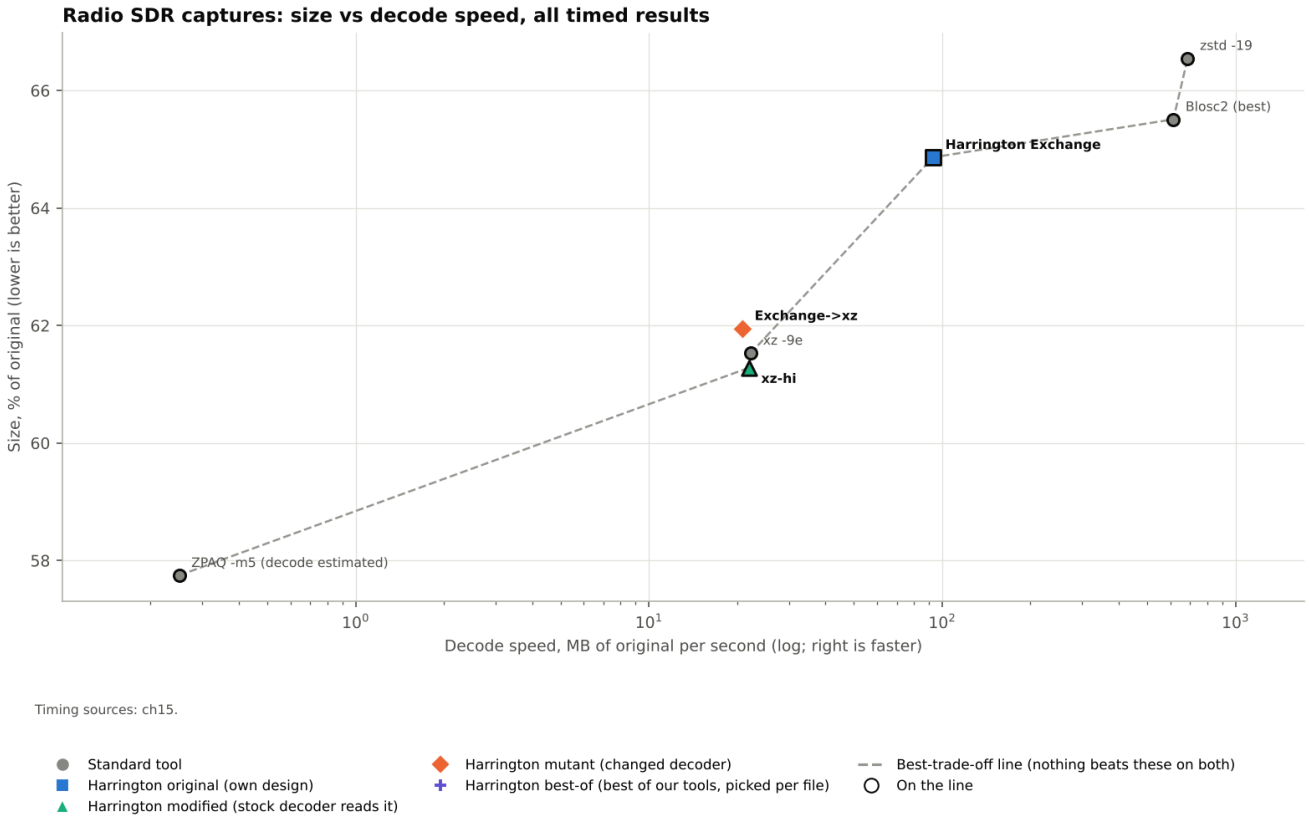


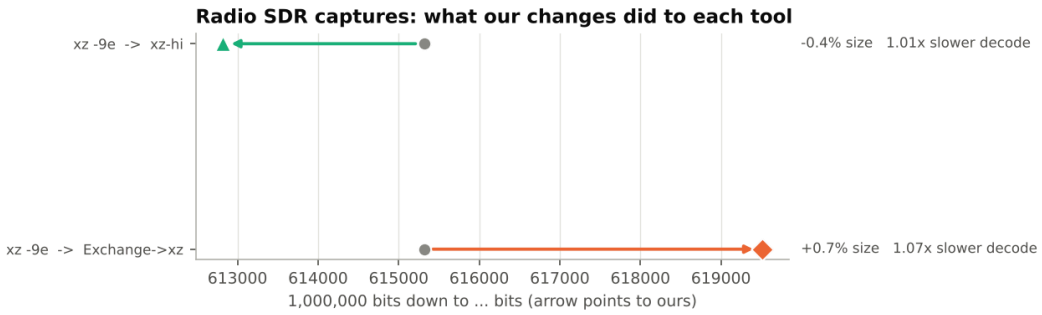



Biased packed data: what our changes did to each tool
Blosc2 (best) -> Exchange->Blosc2 hybrid
-8.3% size 1.28x slower decode
zstd -19 -> Harrington Exchange
-14.4% size 9.26x slower decode
470000 480000 490000 500000 510000 520000 530000 540000
1,000,000 bits down to ... bits (arrow points to ours)

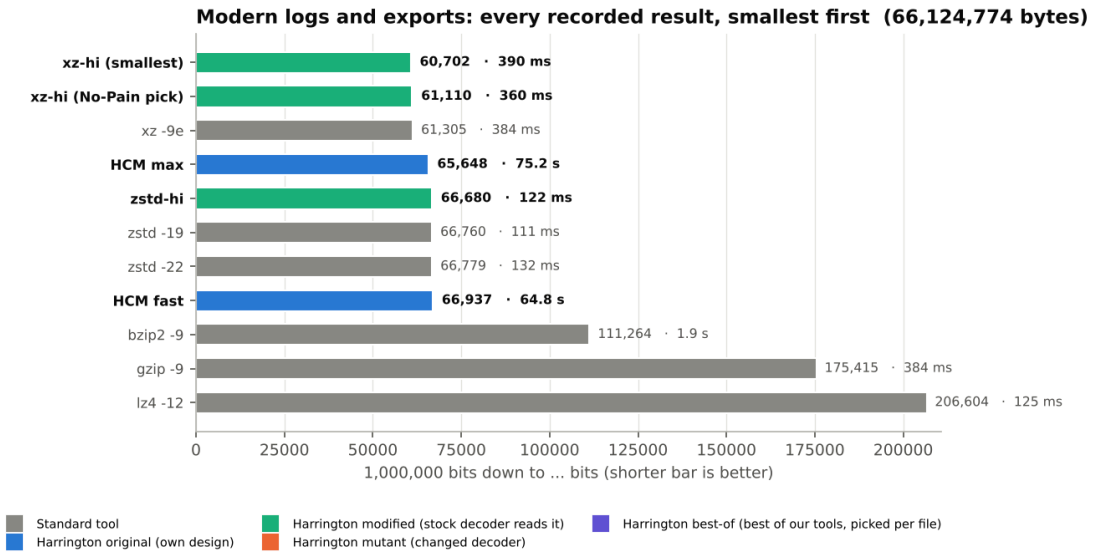
G.7 Radio SDR captures



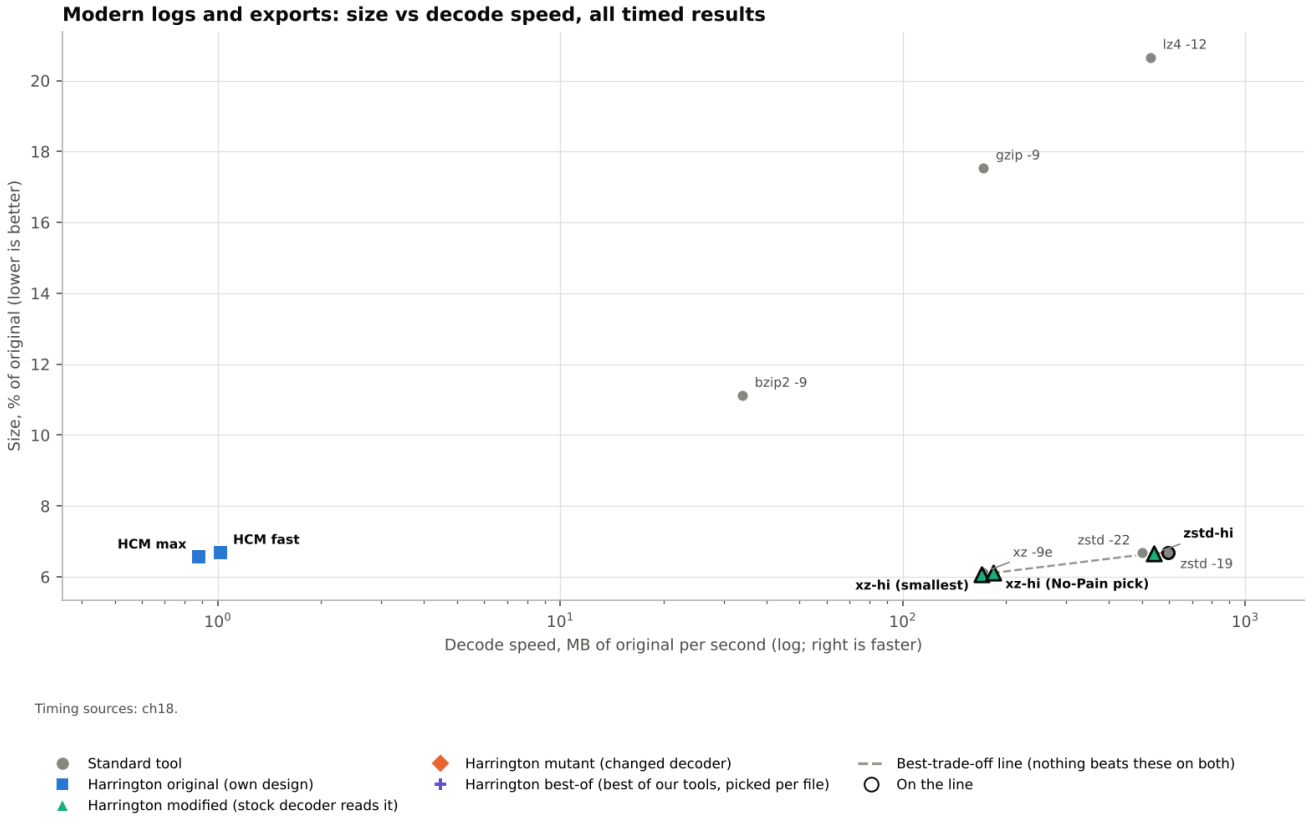


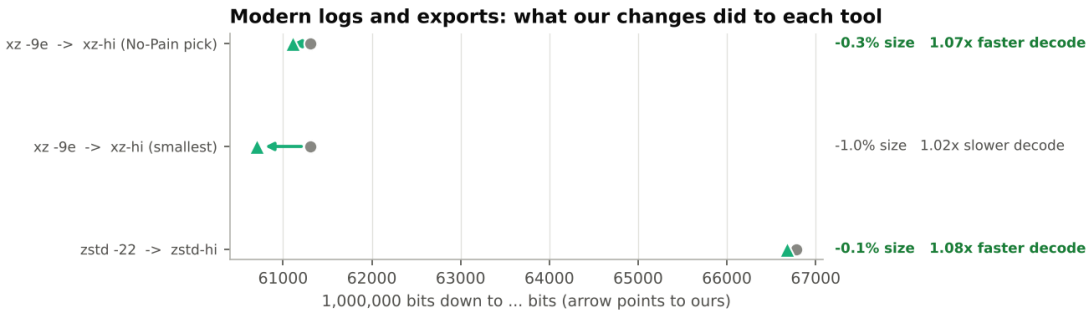

Radio SDR captures: what our changes did to each tool
xz -9e -> xz-hi
-0.4% size 1.01x slower decode
xz -9e -> Exchange->xz
+0.7% size 1.07x slower decode
613000
614000
615000
616000
617000
618000
619000
1,000,000 bits down to ... bits (arrow points to ours)

G.8 Modern logs and exports

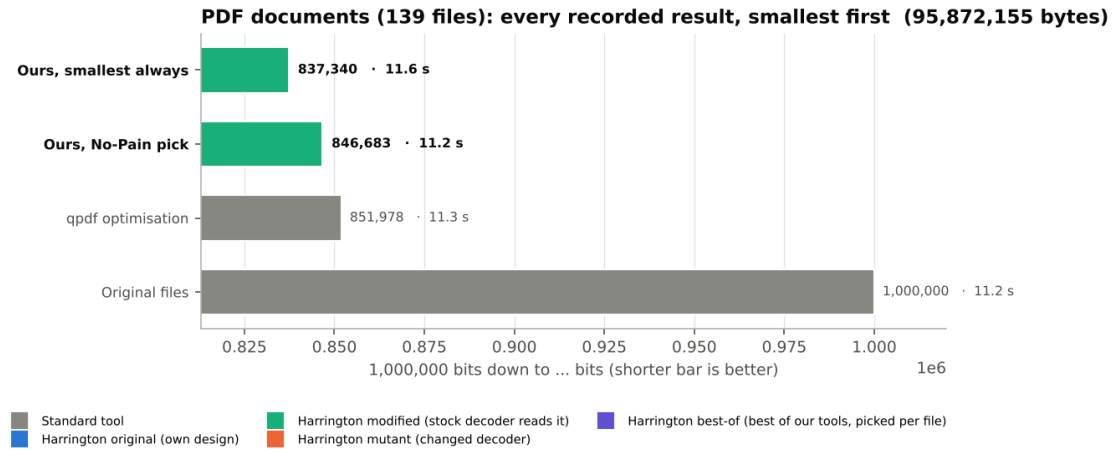


Note: HCM max is the smallest on the four log files (hdfs 42,132 -> 31,068 bytes vs the best stock tool, openstack 38,656 -> 30,192, apache 118,072 -> 80,254, nginx.json 228,132 -> 135,706) but loses on the large exports.

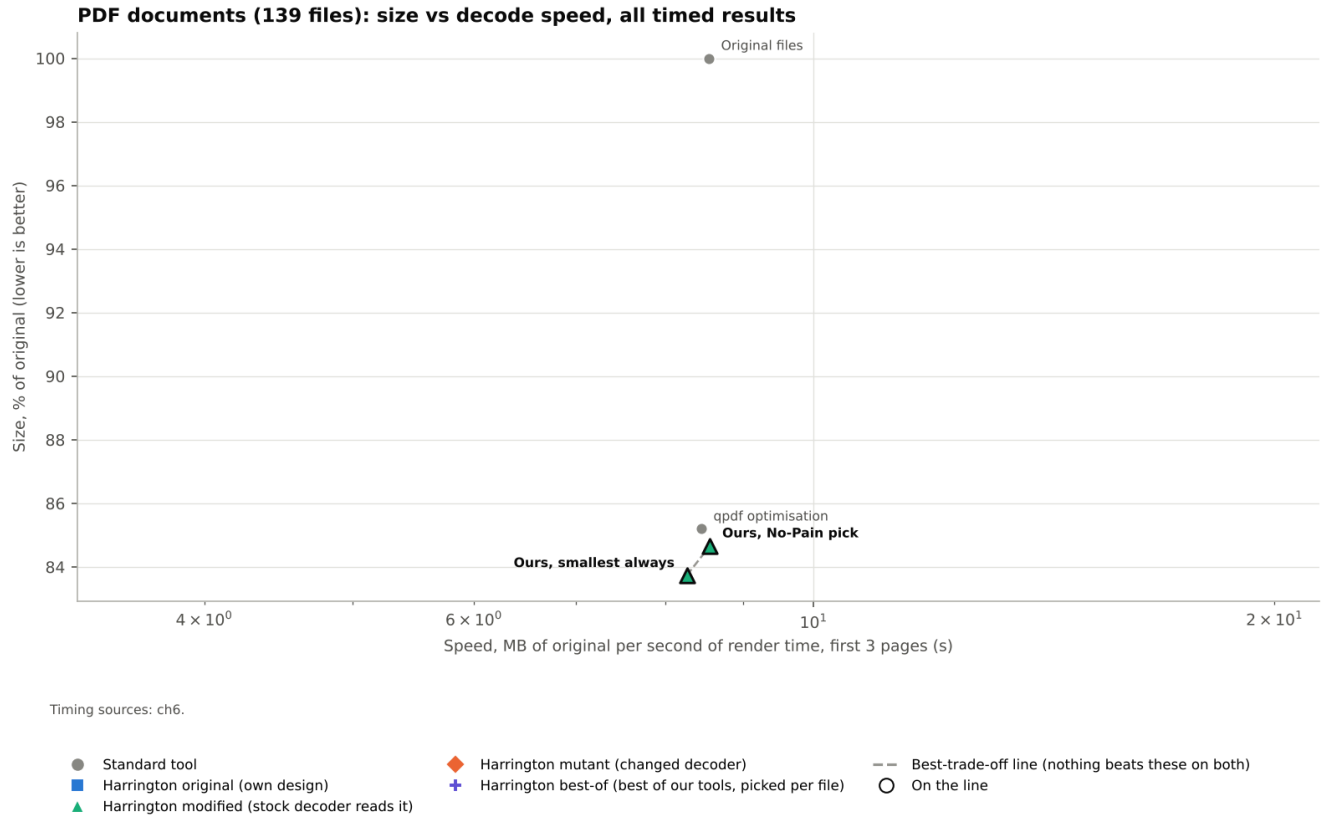



Modern logs and exports: what our changes did to each tool
xz -9e -> xz-hi (No-Pain pick)
xz -9e -> xz-hi (smallest)
zstd -22 -> zstd-hi
-0.3% size 1.07x faster decode
-1.0% size 1.02x slower decode
-0.1% size 1.08x faster decode
61000
62000
63000
64000
65000
66000
67000
1,000,000 bits down to ... bits (arrow points to ours)

G.9 PDF documents (139 files)

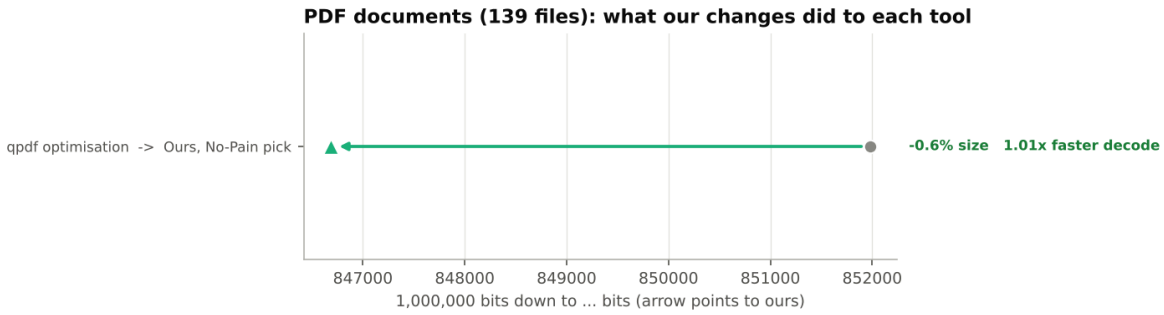


Time shown is render time, first 3 pages (s).

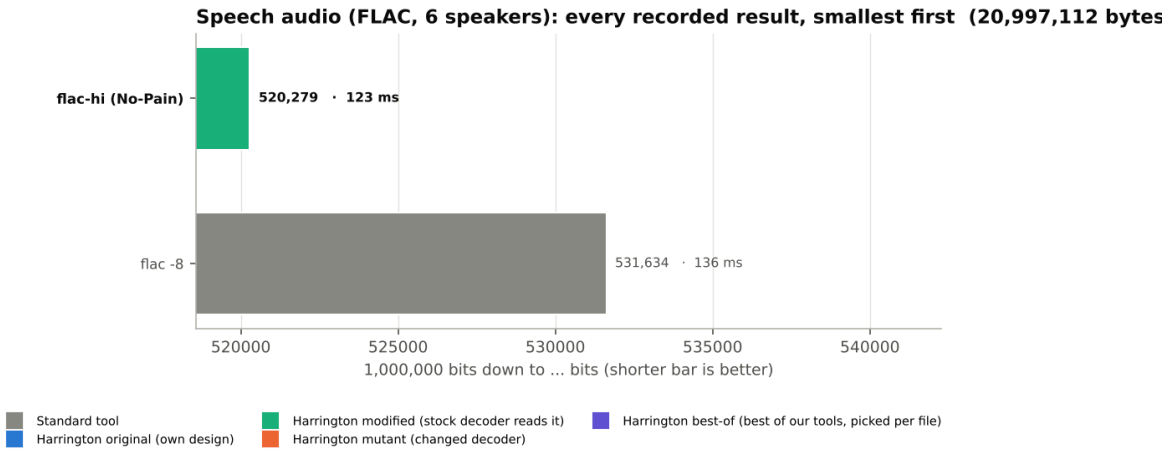


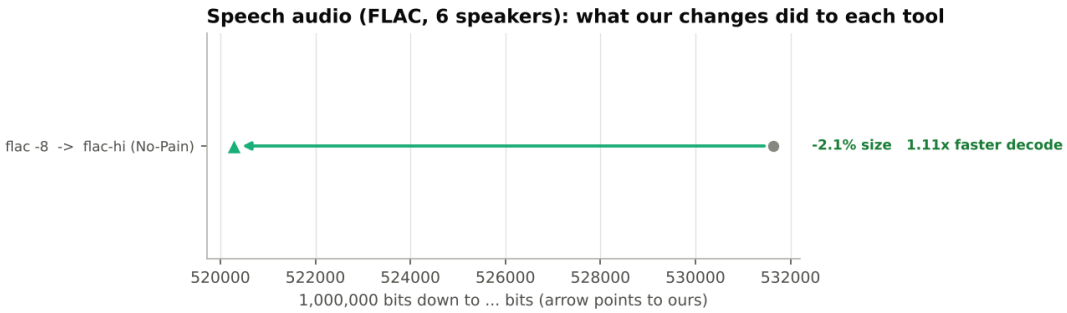
Timing sources: ch6.

Standard tool
Harrington original (own design)
Harrington modified (stock decoder reads it)
Harrington mutant (changed decoder)
Harrington best-of (best of our tools, picked per file)
Best-trade-off line (nothing beats these on both)
On the line

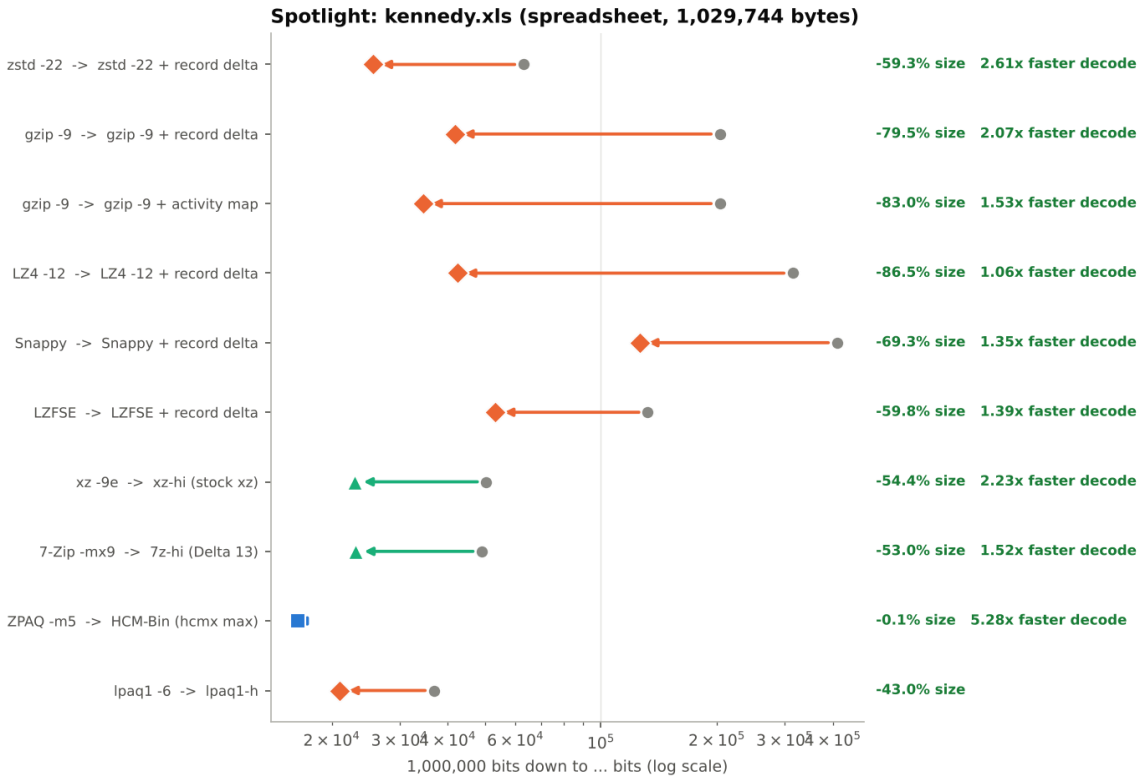

PDF documents (139 files): what our changes did to each tool
qpdf optimisation -> Ours, No-Pain pick
-0.6% size
1.01x faster decode
847000
848000
849000
850000
851000
852000
1,000,000 bits down to ... bits (arrow points to ours)

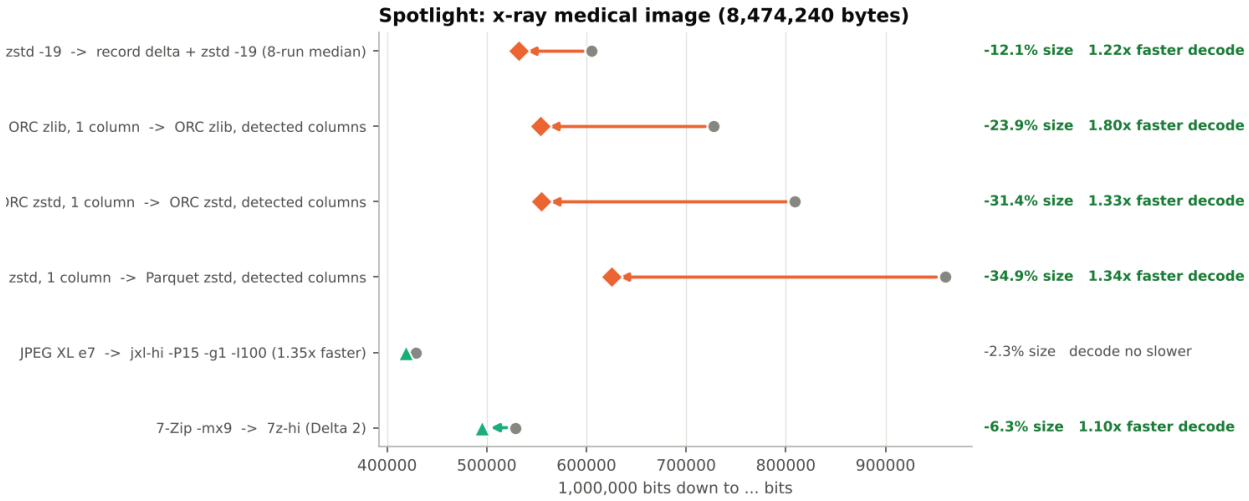
G.10 Speech audio (FLAC)

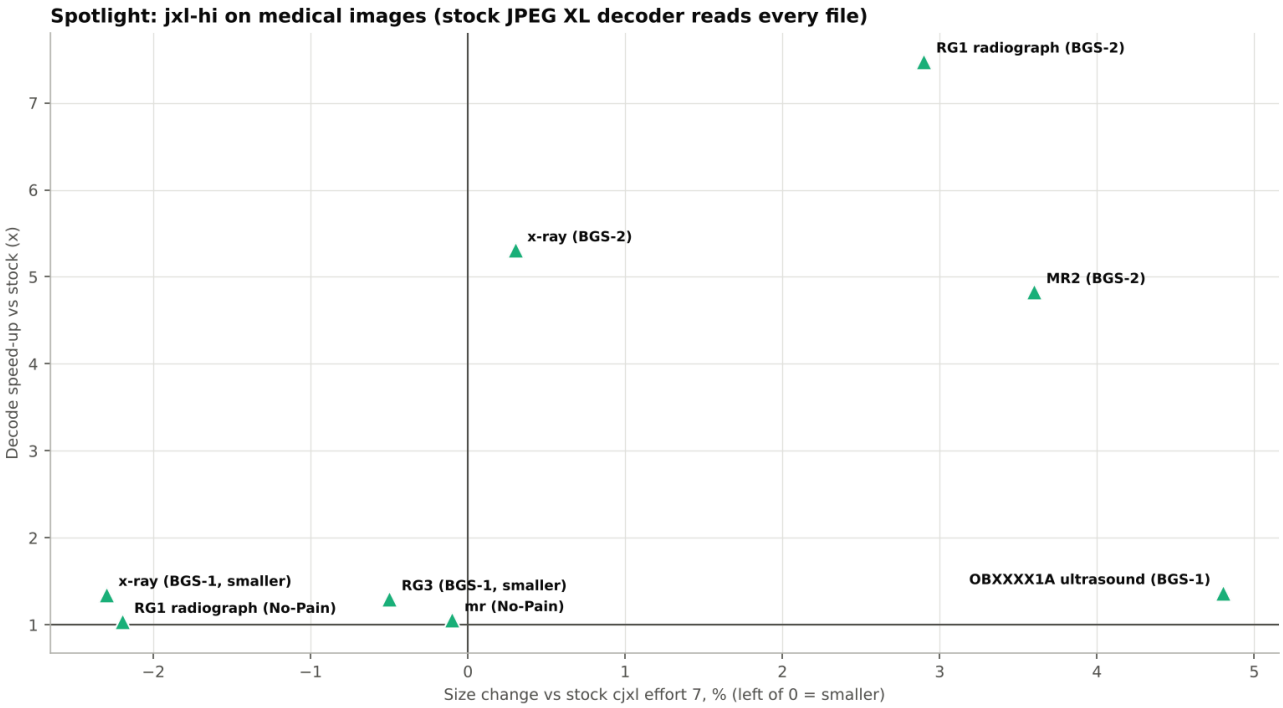




G.11 Spotlights



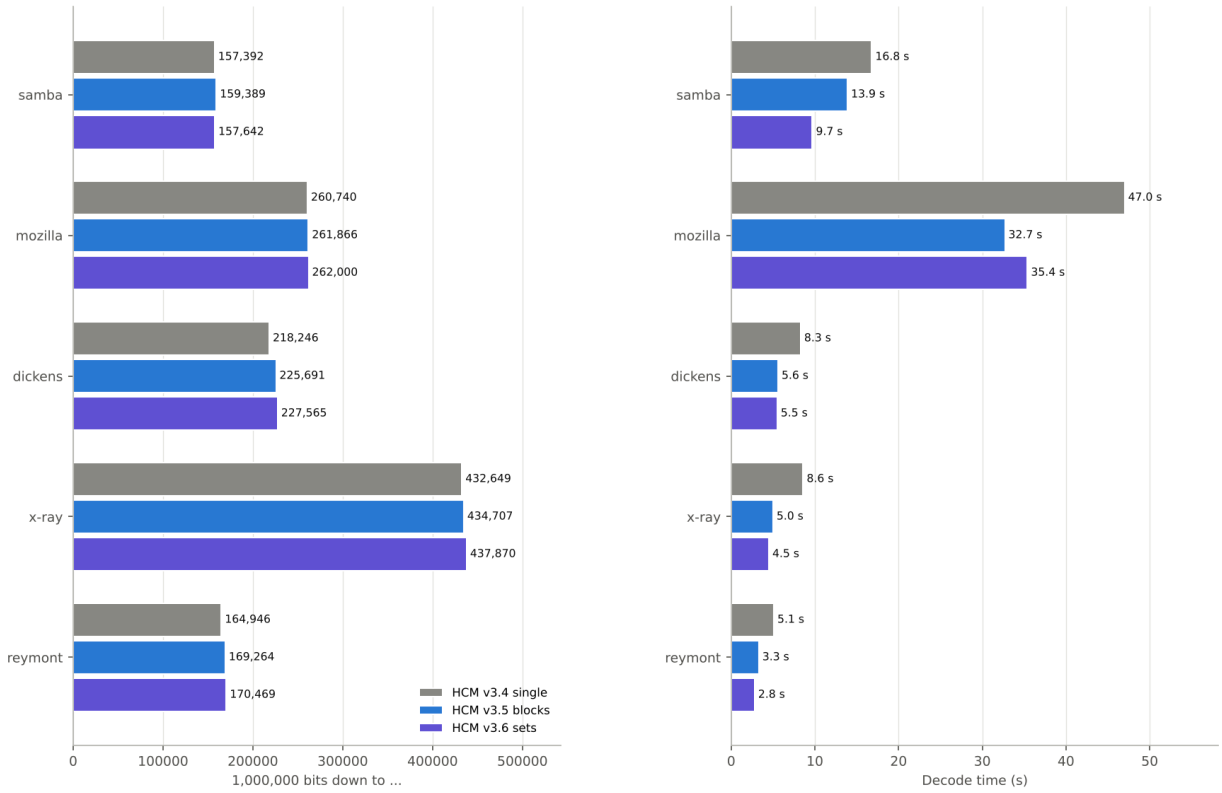



Spotlight: jxl-hi on medical images (stock JPEG XL decoder reads every file)
RG1 radiograph (BGS-2)
x-ray (BGS-2)
MR2 (BGS-2)
x-ray (BGS-1, smaller)
RG1 radiograph (No-Pain)
RG3 (BGS-1, smaller)
mr (No-Pain)
OBXXXX1A ultrasound (BGS-1)
Decode speed-up vs stock (x)
Size change vs stock cjxl effort 7, % (left of 0 = smaller)
7
6
5
4
3
2
1
−2
−1
0
1
2
3
4
5

G.12 Multi-core: one core against parallel blocks and Harrington sets

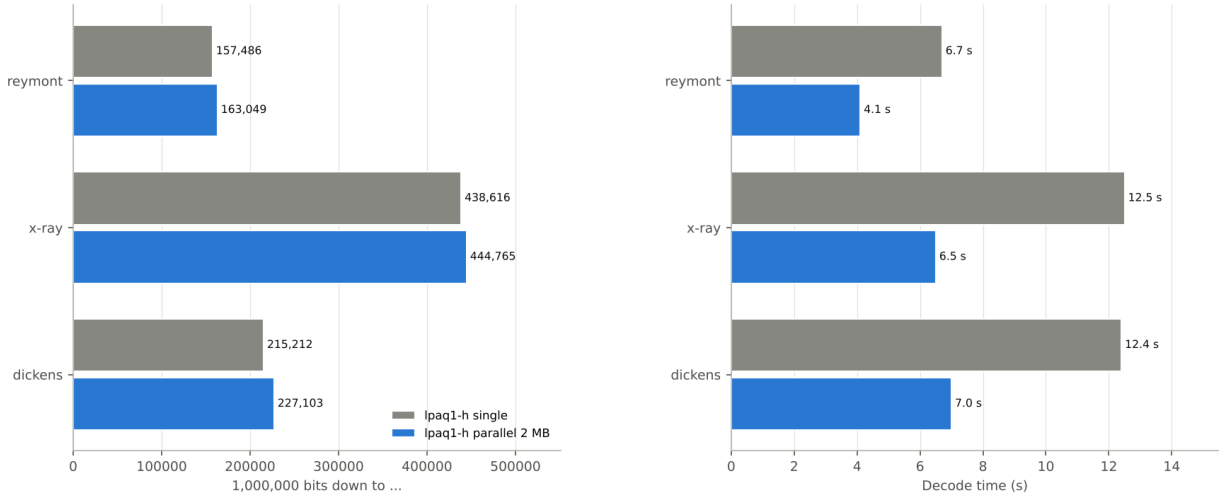
HCM: one core vs multi-core

Left: 1,000,000 bits down to X (shorter is smaller). Right: decode time (shorter is faster). Measured on a 2-core machine; more cores give more speed.
Every file decoded back exactly.



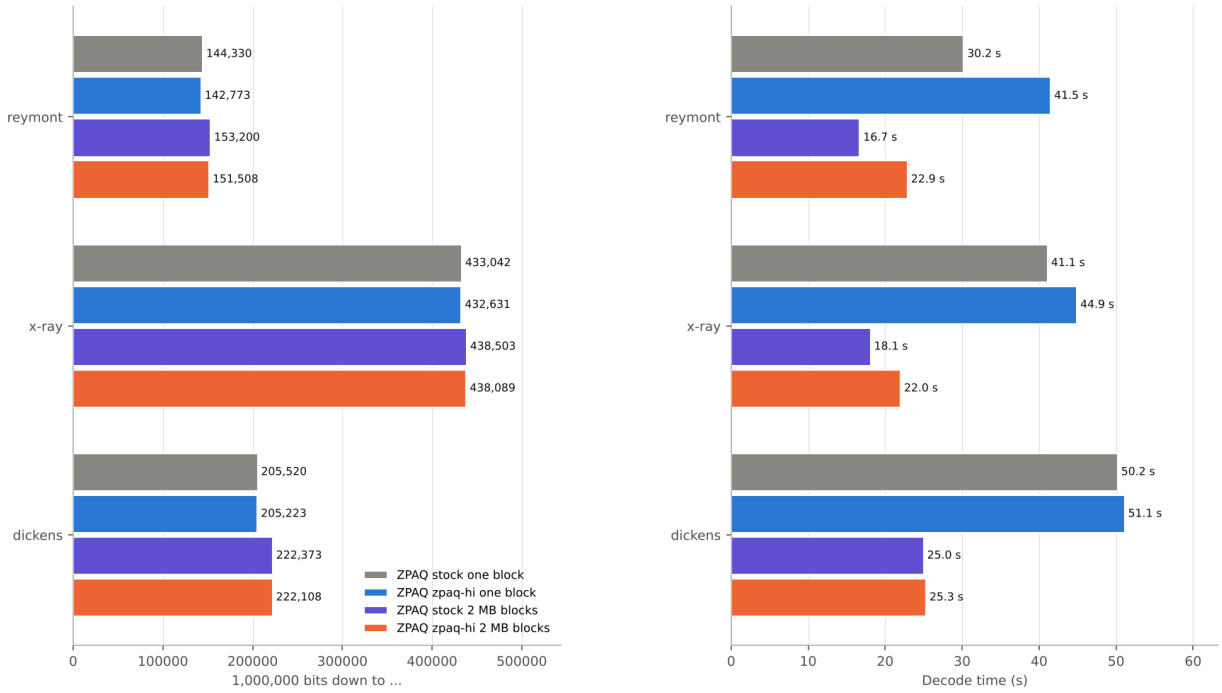
lpaq1-h: one core vs multi-core

Left: 1,000,000 bits down to X (shorter is smaller). Right: decode time (shorter is faster). Measured on a 2-core machine; more cores give more speed.
Every file decoded back exactly.



ZPAQ: one core vs multi-core

Left: 1,000,000 bits down to X (shorter is smaller). Right: decode time (shorter is faster). Measured on a 2-core machine; more cores give more speed.
Every file decoded back exactly.



Contact the Author

Questions, corrections, licensing enquiries and requests for the test tools or data provenance are welcome.

Michael Harrington

- Email: michaelhh@gmail.com
- Phone: 971-533-3116

The scripts and source code used to produce the results in this book are kept by the author and are available on request, so that any result can be reproduced.